



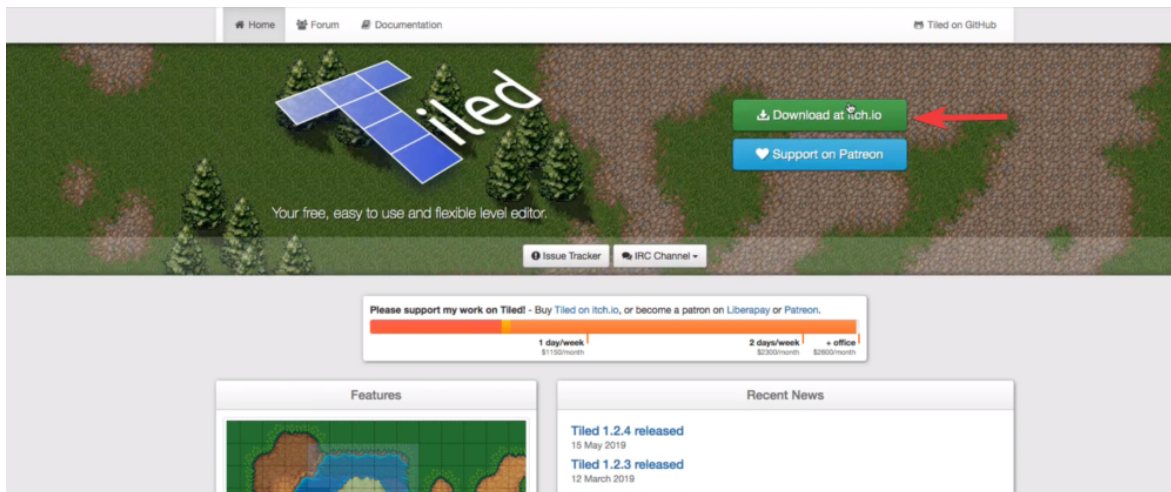
In this lesson, you will learn how to download and install the Tiled Map Editor.

What is Tiled?

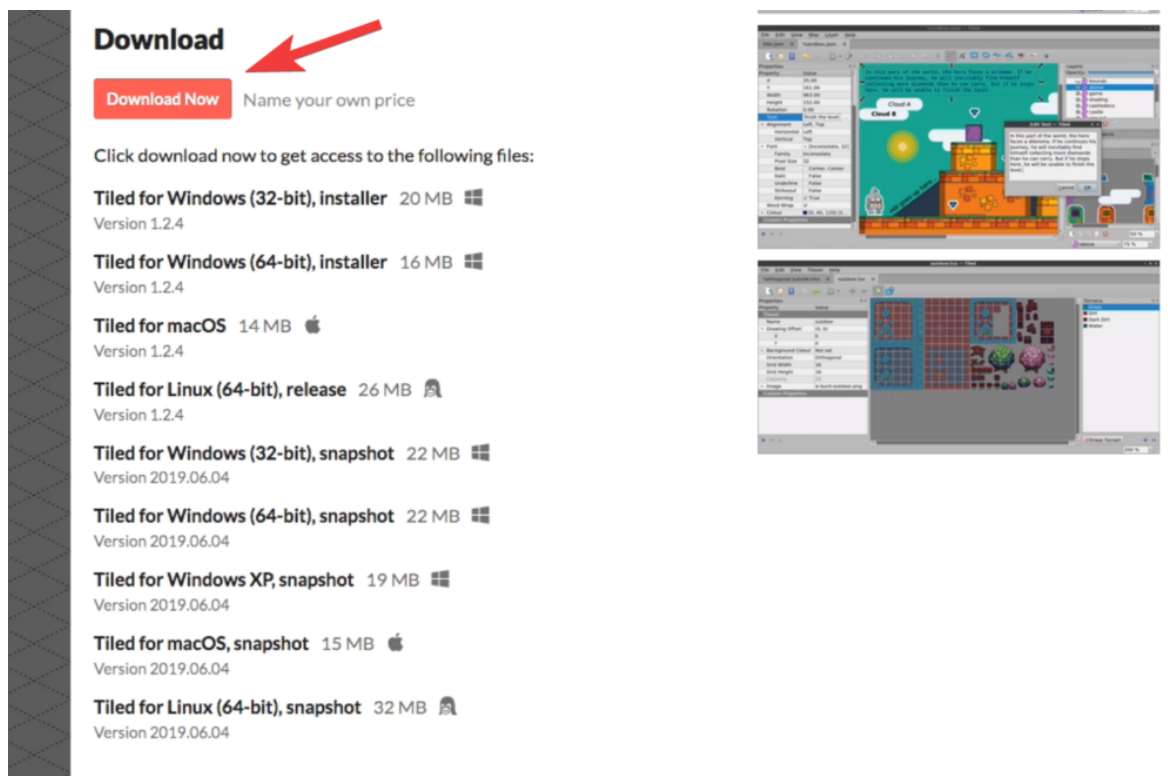
If you haven't encountered Tiled before, it is a general-purpose tile map editor. You can use it for any kind of tile-based games, such as RPGs or platformers. It is highly flexible and can be used to make maps of any size, with no restrictions on the tile size, number of layers, or number of tiles. Maps, layers, tiles, and objects can all be assigned different properties in the editor, which allows you to have a lot of control. Tiled's map format is easy to understand and allows you to utilize multiple tile sets for any of your maps. You can also modify these tile sets at any time, say if you want to change how a particular tile looks.

Downloading Tiled

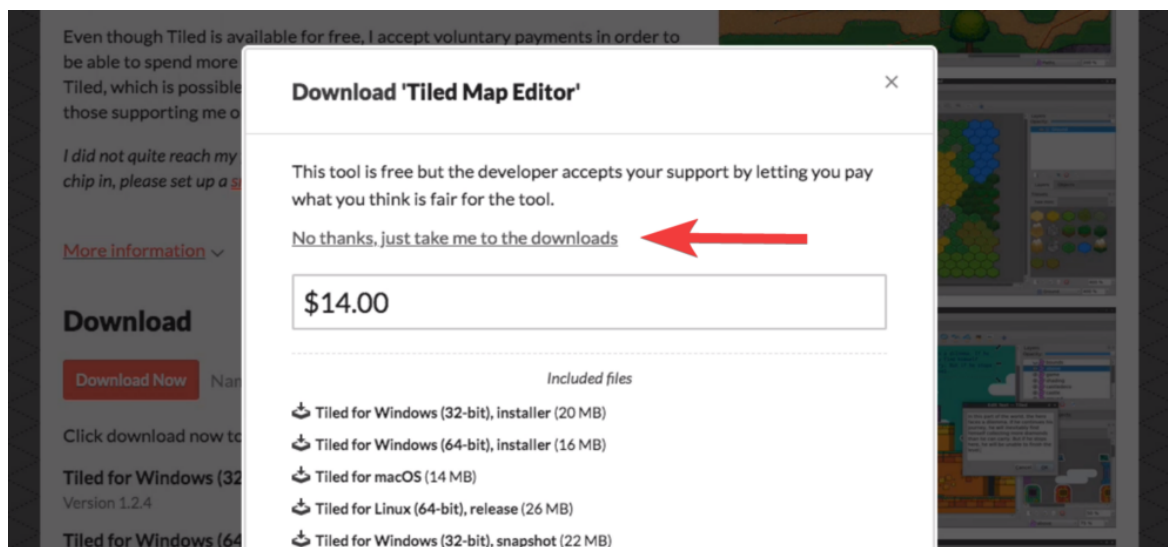
To download Tiled, you will need to go to this website: <https://www.mapeditor.org/>. On this page, you will see a green button that looks like this:



This will take you to the itch.io page. You'll then need to click on another download button.

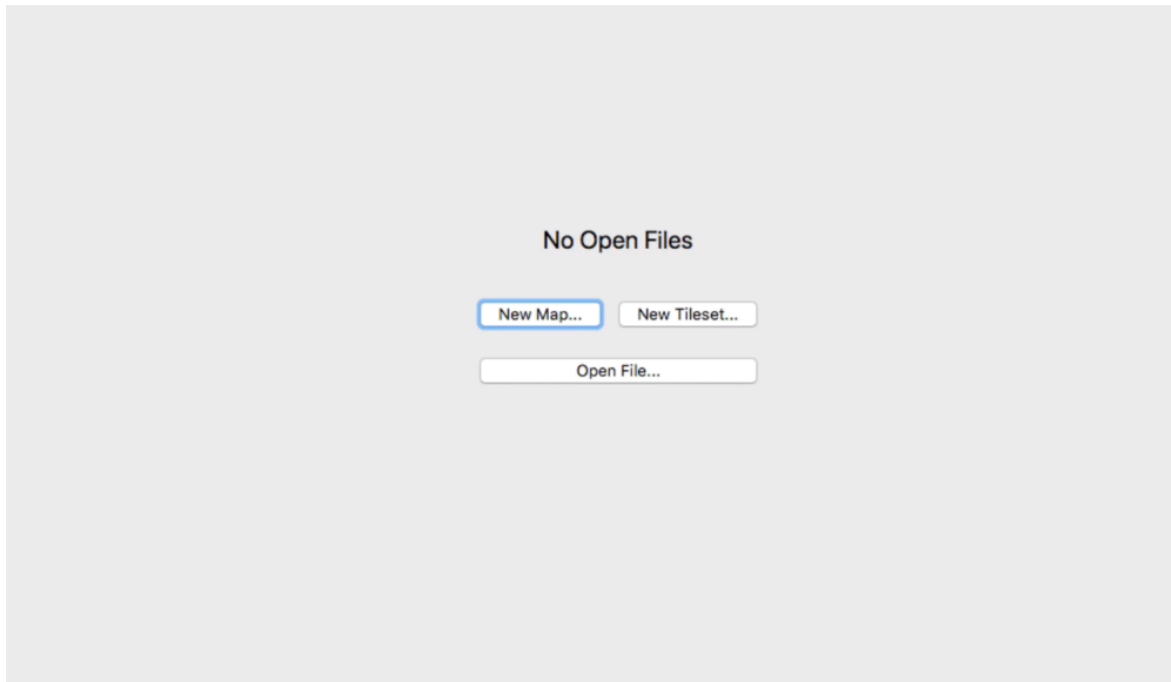


As shown in the image, Tiled is supported on several different platforms. So in the next window, you can choose whether or not you want to support the developer financially. As Tiled is free to use and opensource, you don't have to. Just click on the “No thanks, just take me to the downloads” link shown here:



On this page, download the appropriate version for your OS. Once it is downloaded, install it as you would any other program. The installation process varies based on your OS, but it should be straightforward.

Once it is installed, go ahead and open it. It should look like this:

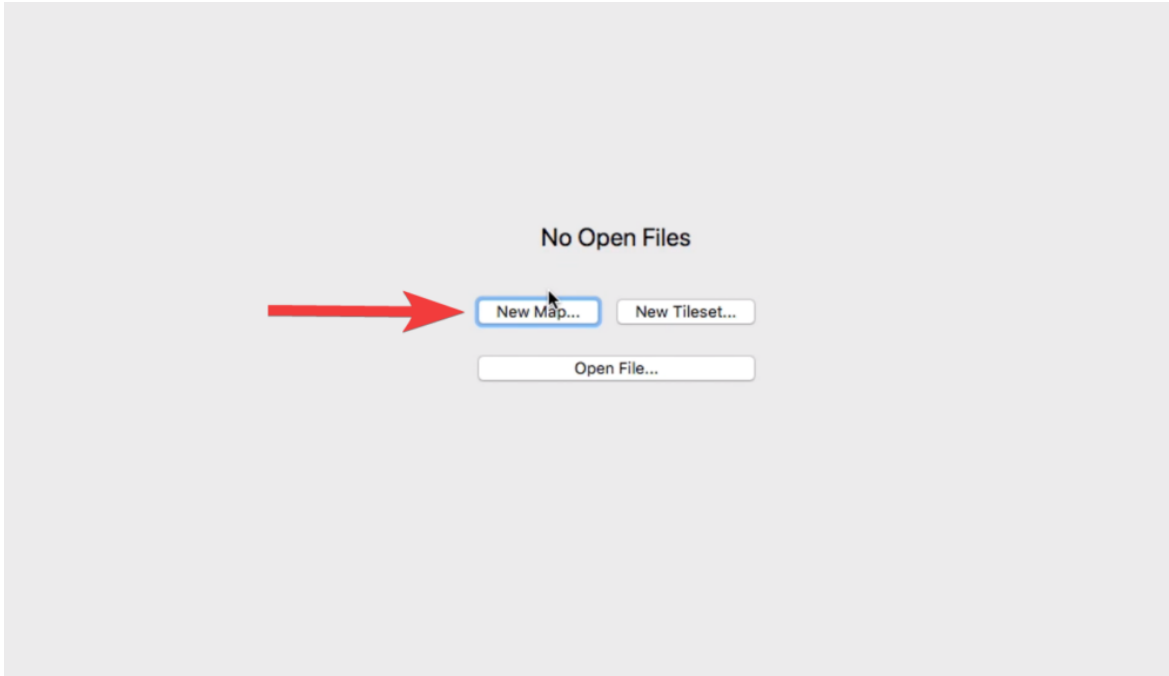


This will be the main page that opens if you don't have a project that you're working on.

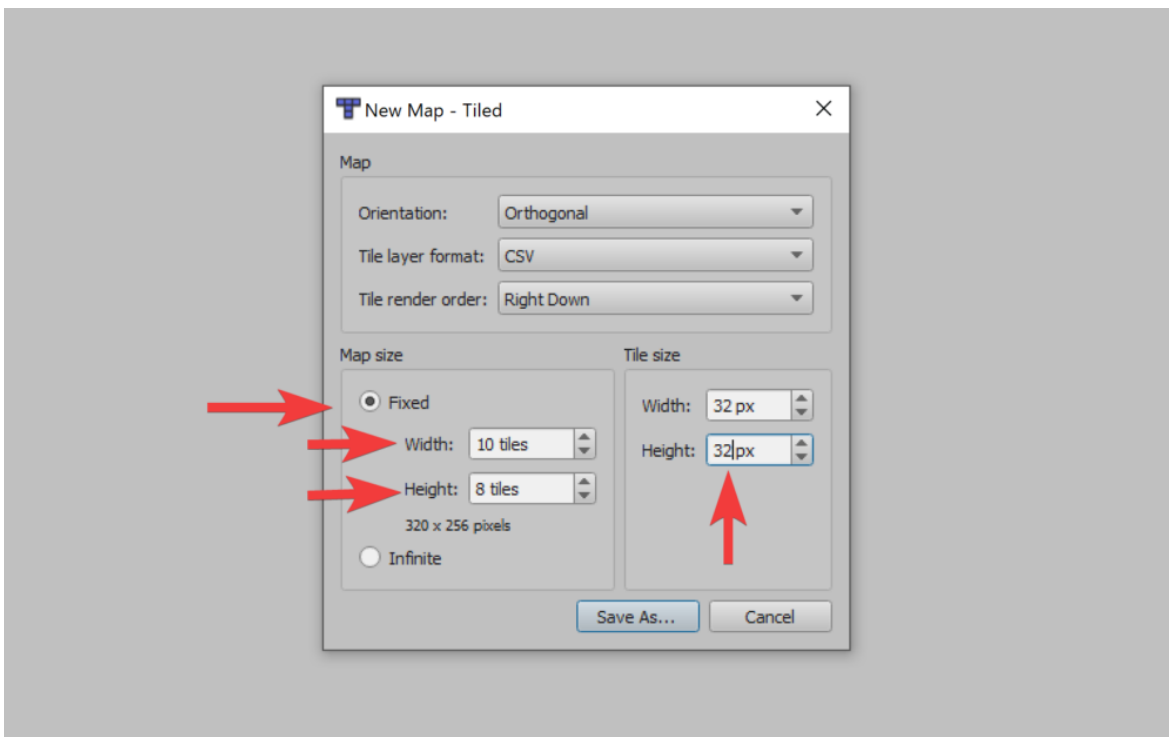
In this lesson, you will learn how to use Tiled to create a map for a level in your game.

Creating a New Map

The first thing you'll need to do is open up the Tiled editor and click on the "New Map..." button.

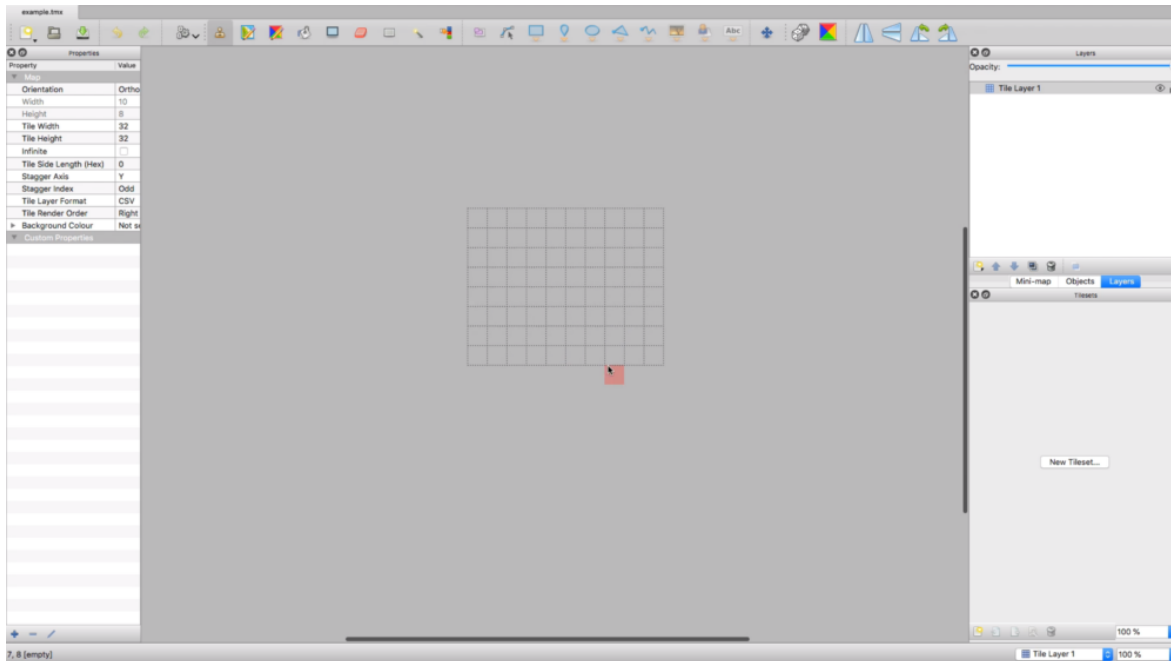


In the next window that opens, you can change some of the properties of the map that will be generated. For this course, you will need to change the **Map size** to use a **Fixed** size. The **Width** will be 10 tiles and the **Height** will be 8 tiles. For the **Tile size**, you'll need to change the **Width** and **Height** to be 32 px. The window should look like this when you are done:



Click on the “Save As...” button. Feel free to name this whatever you want, as this map will be used as an example.

You’ll then be taken to the map editor. It should look something like this:

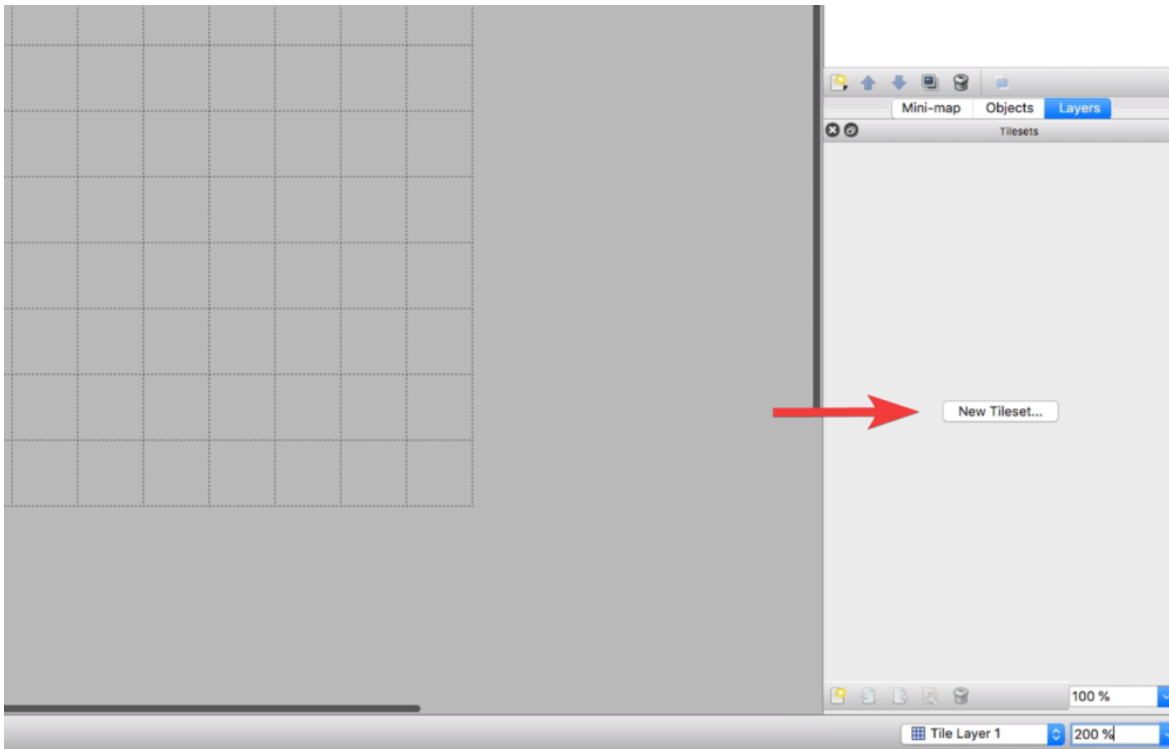


The first thing that you can do to help you see the map a little easier to is to increase the Zoom level. This is in the bottom right, as is common for programs.

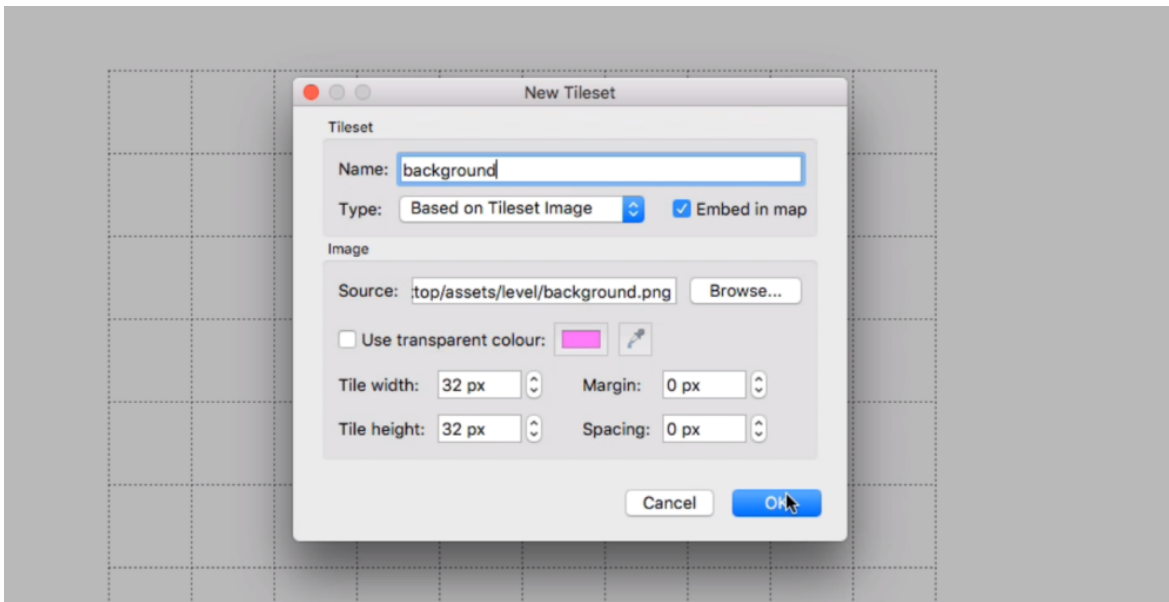
Loading a Tileset

Now, you’ll want to open up the **Assets** folder from the **Course Files**. In it, you will find a **level** folder, which will have four files in it. The two **background** images will be the sprite sheets you’ll be using to create your level in Phaser. The **large_level.json** and **large_level.tmx** will be the maps you’ll be using for the rest of the course.

Back in Tiled, in the bottom right there will be a blank area with a button that says “New Tileset...”.

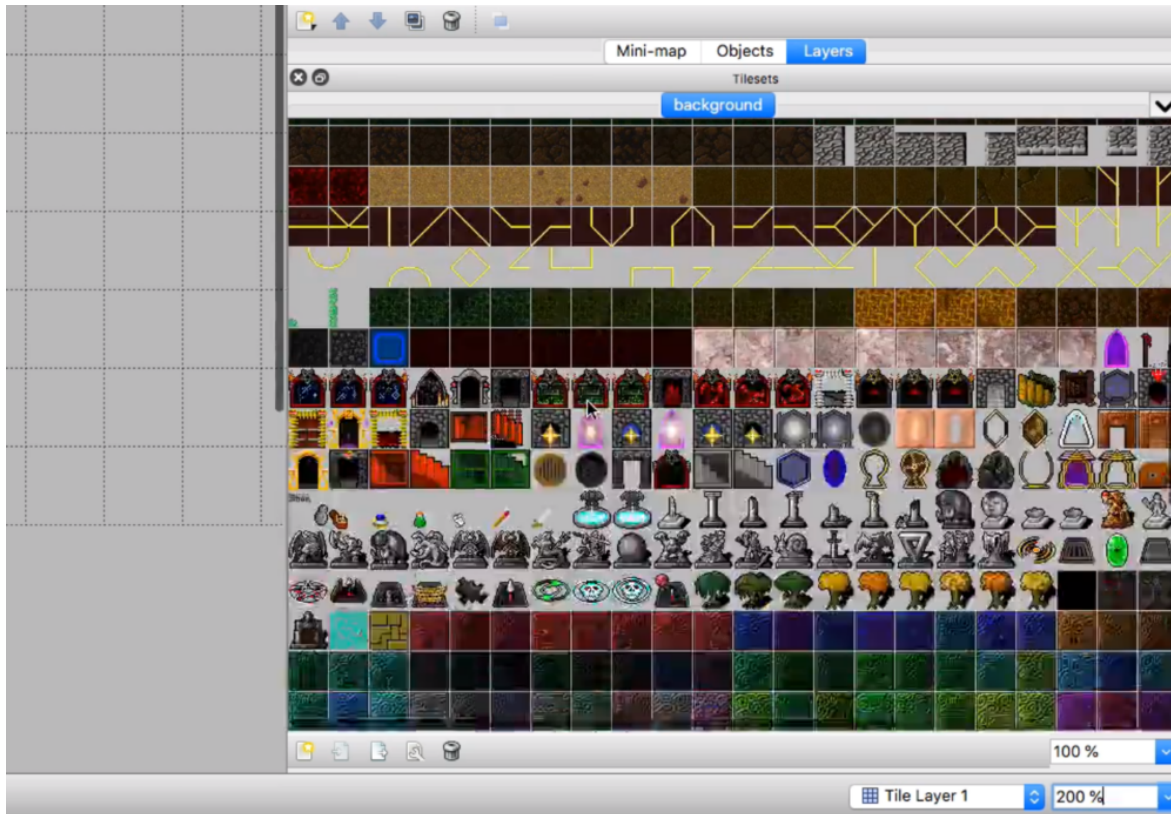


Click on this button, then click on the “Browse...” button in the window that opens. Navigate to and select the **background.png** file from the **level** folder. The **Tile width** and **Tile height** should both be 32 px and you’ll want to make sure that the **Embed in map** checkbox is marked. This saves the metadata in your map file so that when you export it, all of the relevant data will be included.



Click on the **OK** button to upload the tileset.

Now over on the side in the Tileset area, you will see all of the sprites that were included.

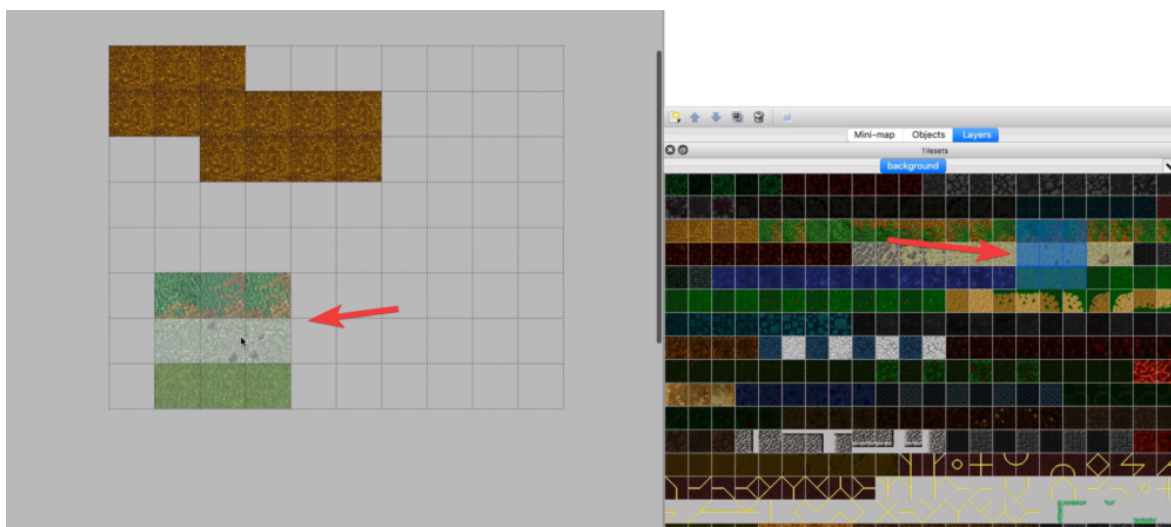


This is how Tiled operates. When you specified the width and height, Tiled split up the tileset into sections of that size. You can then use these sections to build your map.

Painting Tools

Click on one of the sprites and then hover over the map. You'll see that you can place it in any of the squares of the grid. Using the **Stamp Brush** (which has a hotkey of B) you can change single tiles at a time.

If you want to paint a group of tiles at a time, you can select a larger section of the tileset and then use that in your map.

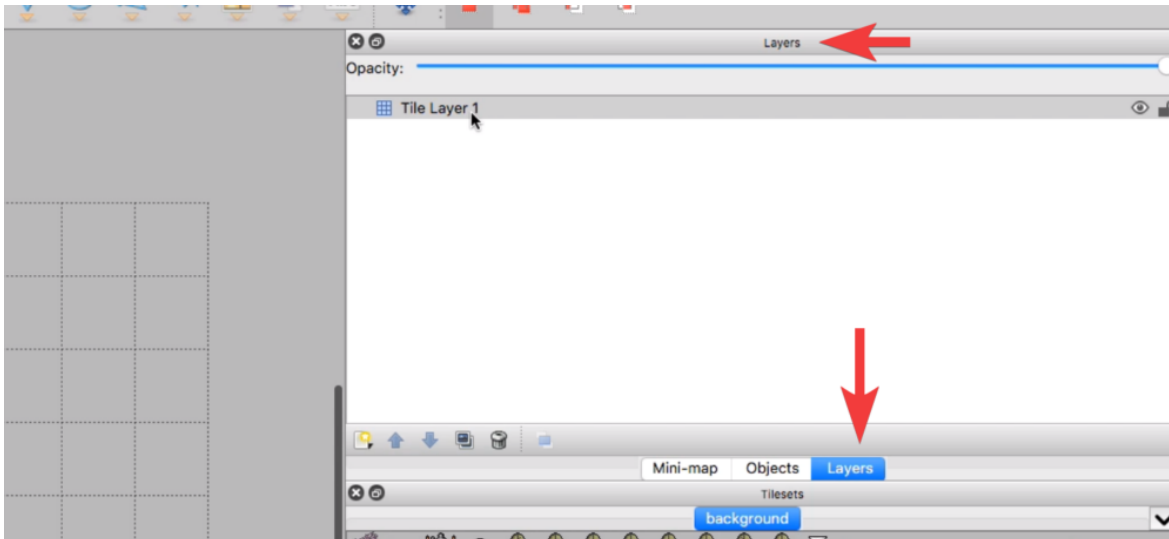


There are a few other tools that you will most likely use frequently. The **Bucket Fill** tool (hotkey: F)

which fills entire areas at a time with the selected tile/s. The **Shape Fill** tool (hotkey: P) is used to draw a rectangular shape and fill that section. The **Eraser** tool (hotkey: E) will erase tiles. The final two commonly used tools are the **Rectangular Select** tool (hotkey: R) and the **Magic Wand** tool (hotkey: w). Both of these tools are used to make selections in your map, which will then allow you to paint in only those sections.

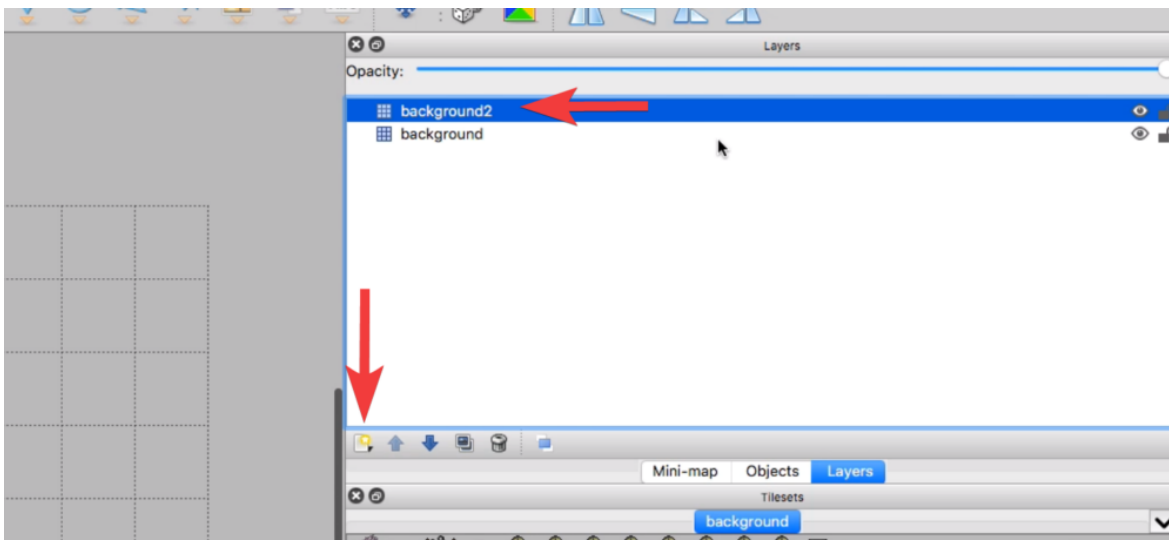
Tile Layer System

The next main feature to learn about in Tiled is the layer system. The layer section in the top right. It should have one layer that says **Tile Layer 1**.



This is the default layer that is generated with any new map. Double click the name and change it to **background**. This is important because any tiles or other metadata used in this layer will be tied to it. Go ahead and paint some tiles onto your map. Then, click on the **eye** icon in the top-right. This will turn off the visibility of that layer. When you have multiple layers, this will be helpful for looking at one layer at a time.

Make another layer now, using the button in the bottom left corner of the layer window. Name it **background2**.



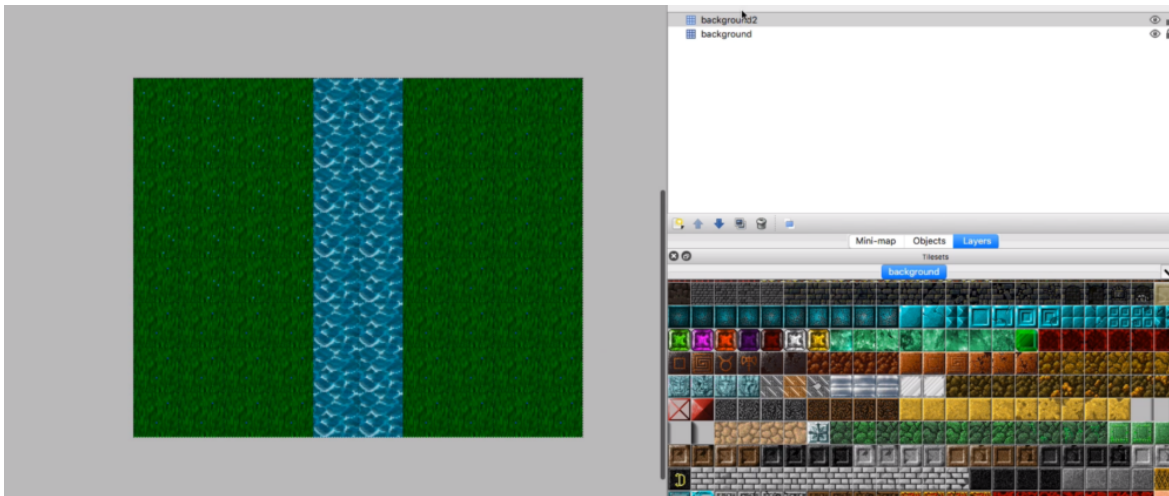
Now with **background2** selected, paint some more tiles on your map, and overlap a little. Then you



can use the **eye** icon again, which will show the **background** layer tiles are still there. The **background2** tiles are simply on top. If you change the order of the layers by clicking and dragging them, you'll change the layering order in your map.

It is easy to create a layer that will block a player's movement. First, delete the tiles you have painted for **background2**. Then, selecting **background** fill the rest of the map with the same tile you used earlier. Then lock that layer so that it can't be edited any further. This isn't super important, but it will help make sure you don't accidentally change anything.

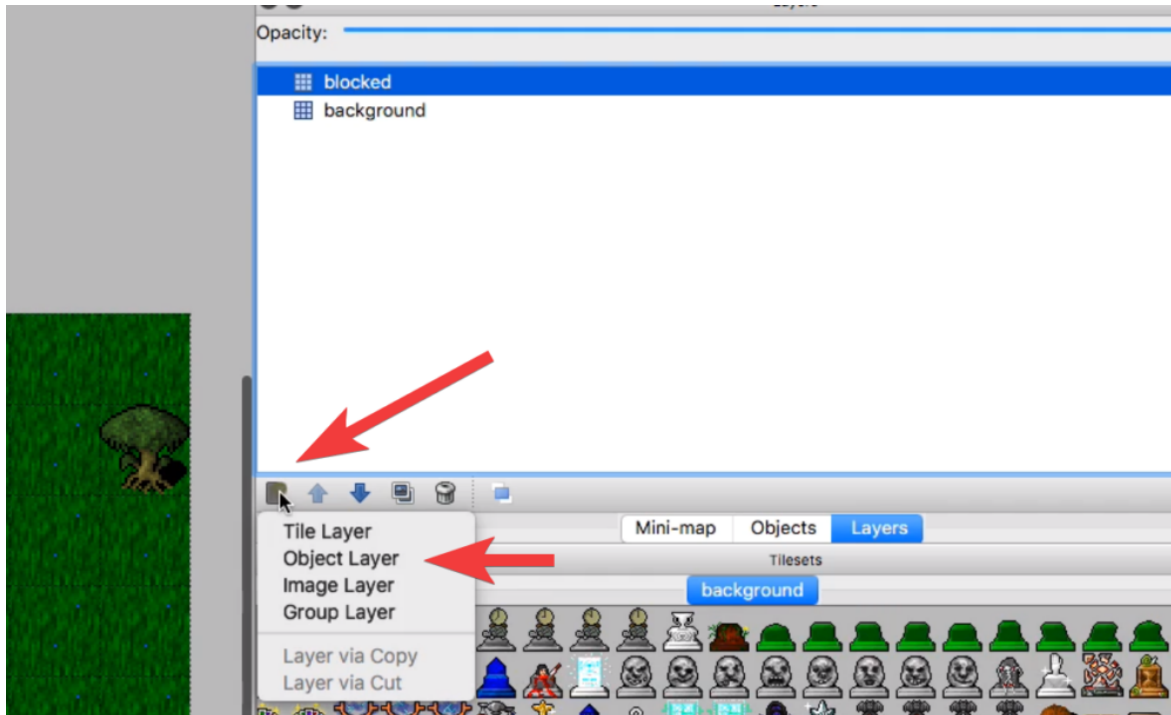
Now reselect **background2** and pick a different tile, such as one of the various water tiles. Paint a section that you want to block the player. For example, you can make a river that would prevent the player from crossing.



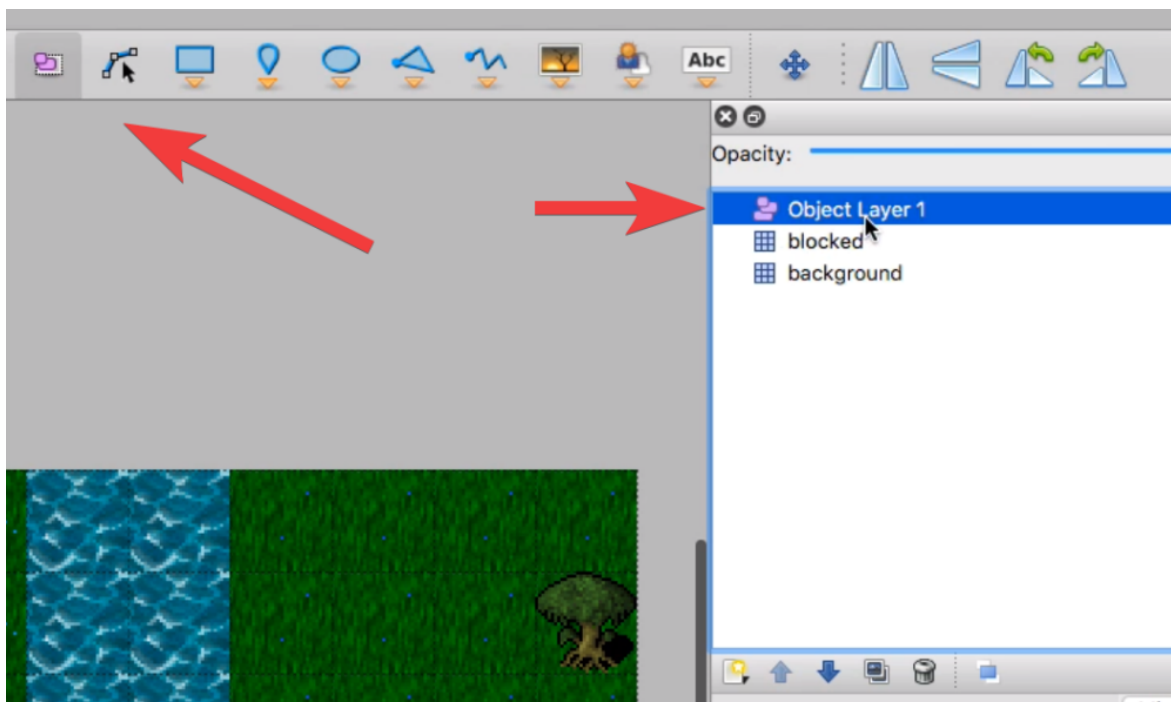
You'll want to rename **background2** to be **blocked** now. You can then add some other objects to your map that you'd want the player to collide with, like columns or trees. As long as you put it in your **blocked** layer, you can easily make it so that the player will collide with it when you load it into Phaser.

Object Layer System

Another type of layer in Tiled is the **Object Layer**. This layer will allow you to add metadata to particular points on your map, which will be used in Phaser to create game objects like the player, monsters, chests, or portals. Click on the **New Layer** button and add an **Object Layer**.

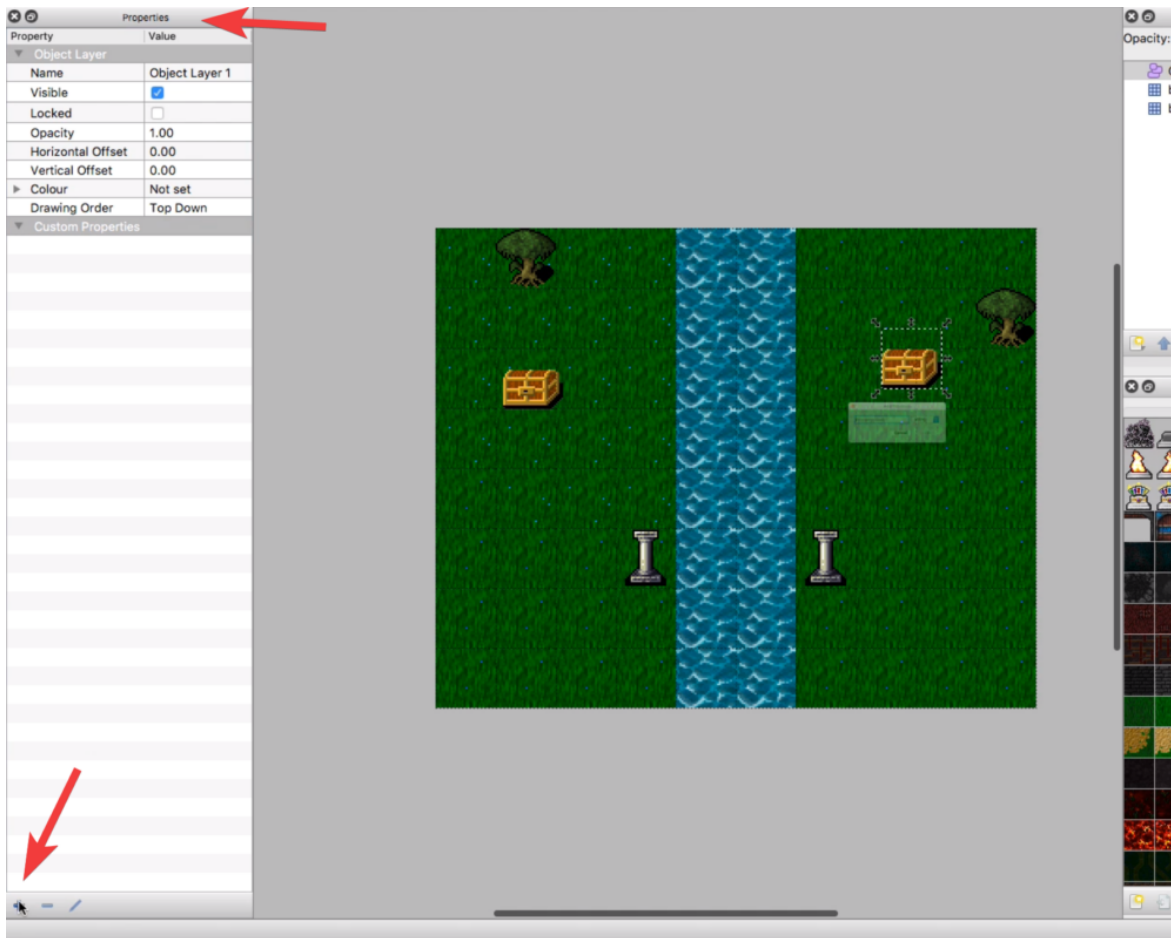


To add objects to the game, you need to use a different toolset. When you have the **Object Layer 1** selected, you will see that the toolset in the top changes so that the right section is highlighted.

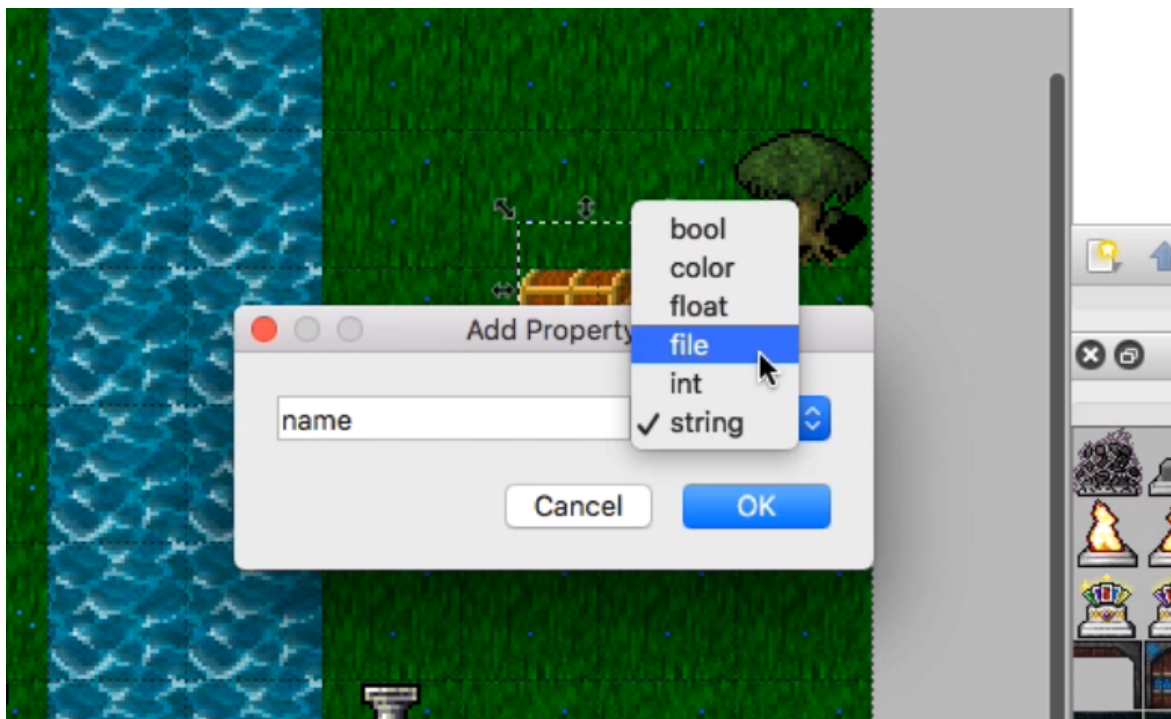


The **Insert Rectangle** (hotkey: R), **Insert Point** (hotkey: I), **Insert Ellipse** (hotkey: C), and **Insert Polygon** (hotkey: P) tools will be what you use to create objects of various shapes. You can also use the **Insert Tile** tool (hotkey: T) to insert a tile sprite as an object. For example, if you wanted to have a chest object, you could insert the chest sprite to represent that object. Just like with the tile layers, you can control the visibility and lock the layer to prevent editing.

One of the helpful features of objects is their **Properties**. This is over on the right-hand side of the screen. You can also add custom properties using the plus in the bottom-left.



When you click on the plus, a new window will open up. This window will allow you to name the new property and choose what kind of variable it is.

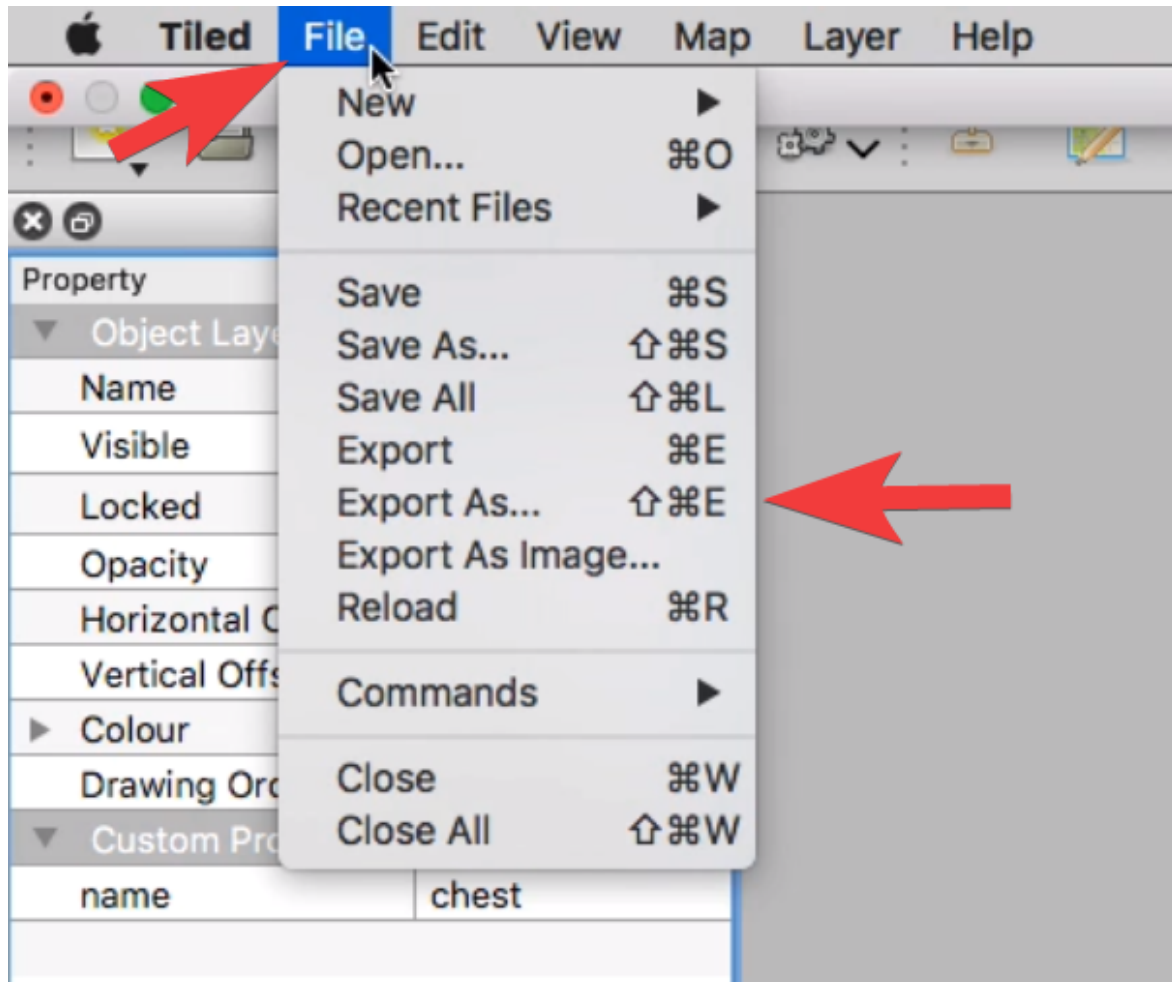


The nice thing about these custom properties is that you can use all of this metadata directly in

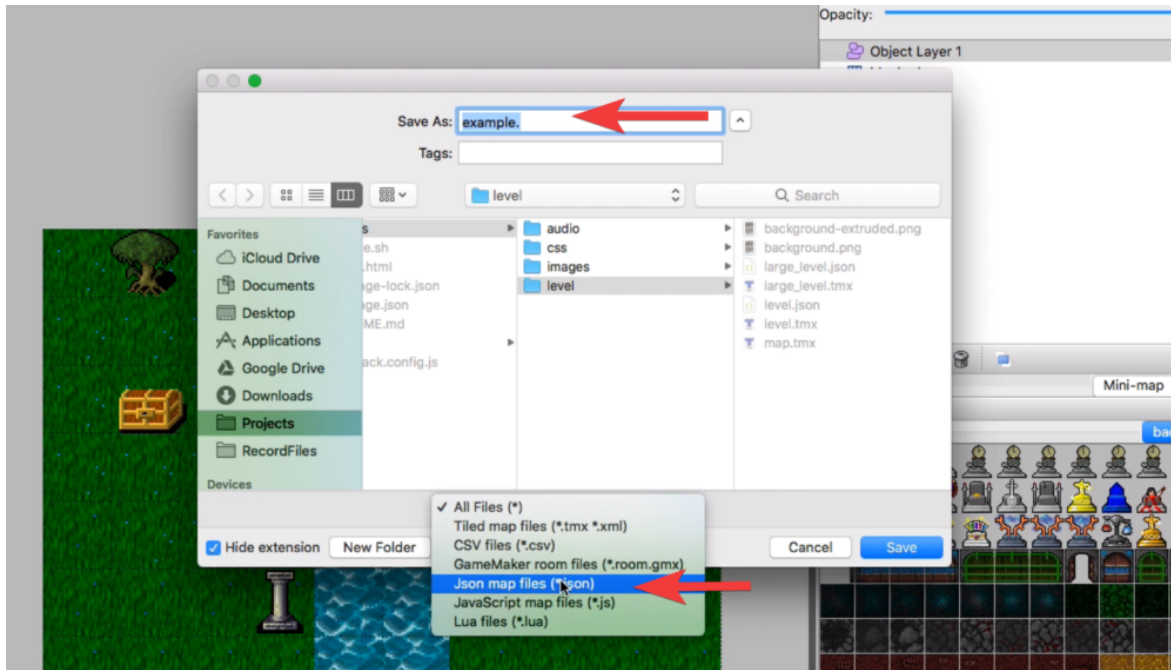
Phaser to do different things in your game.

Exporting The Map

Once you have finished creating a map, the final step is to export it as a JSON file. To do this, go to the **File** tab at the top, and then select **Export As...**. You can also press Ctrl/Cmd + Shift + E to choose how you export the map.



In the browser window that opens up, change the file type to **JSON map files (*.json)**, change the file name to be whatever you want, and then click **Save**.



There are several different file types that you can export Tiled maps to, but for this course, you will need to use JSON to load the map into Phaser.

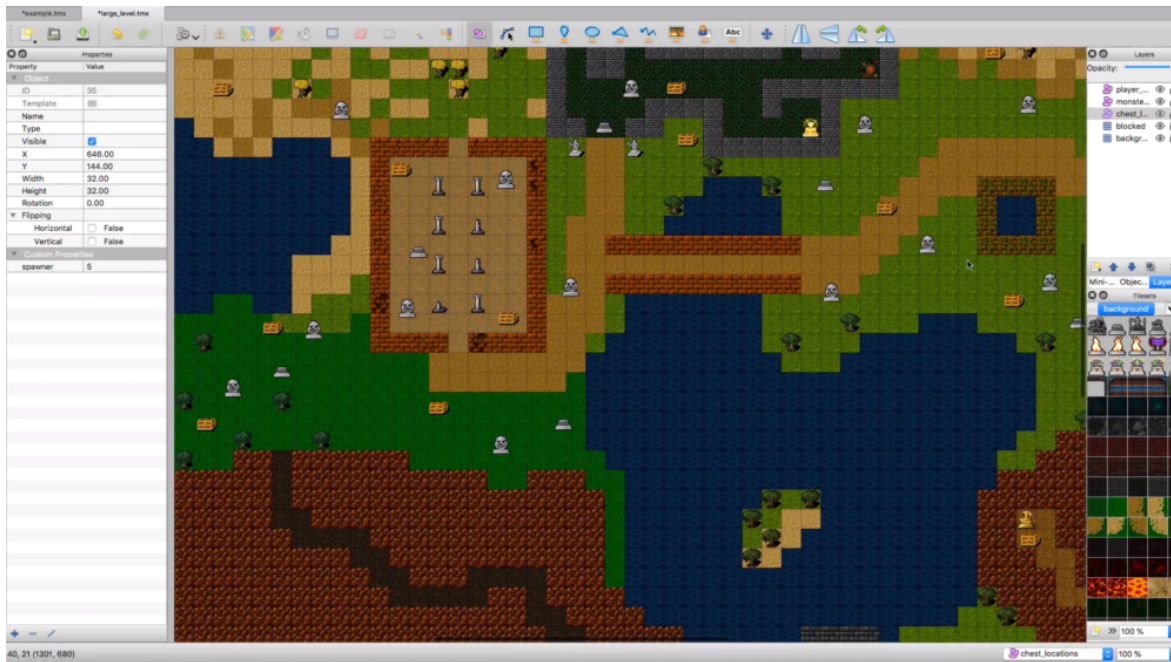
Now you should understand the basics of using Tiled to create levels. As a challenge, try to create a level to use in the game you will be making for this course.

In this lesson, you will learn what a map needs to include to be able to work as a level for your game.

Looking at the Large Level Map

For this course, you can use the provided level, or you can make one yourself. To see an example of what kind of level works for this game, open up the **large_level.tmx** file. You can either open it directly in the Tiled program or through your file explorer.

When you open it, the map will look something like this. You will need to zoom out if you want to see the whole thing.



As you can see, this level is quite large. However, you can easily use this in Phaser without any issues.

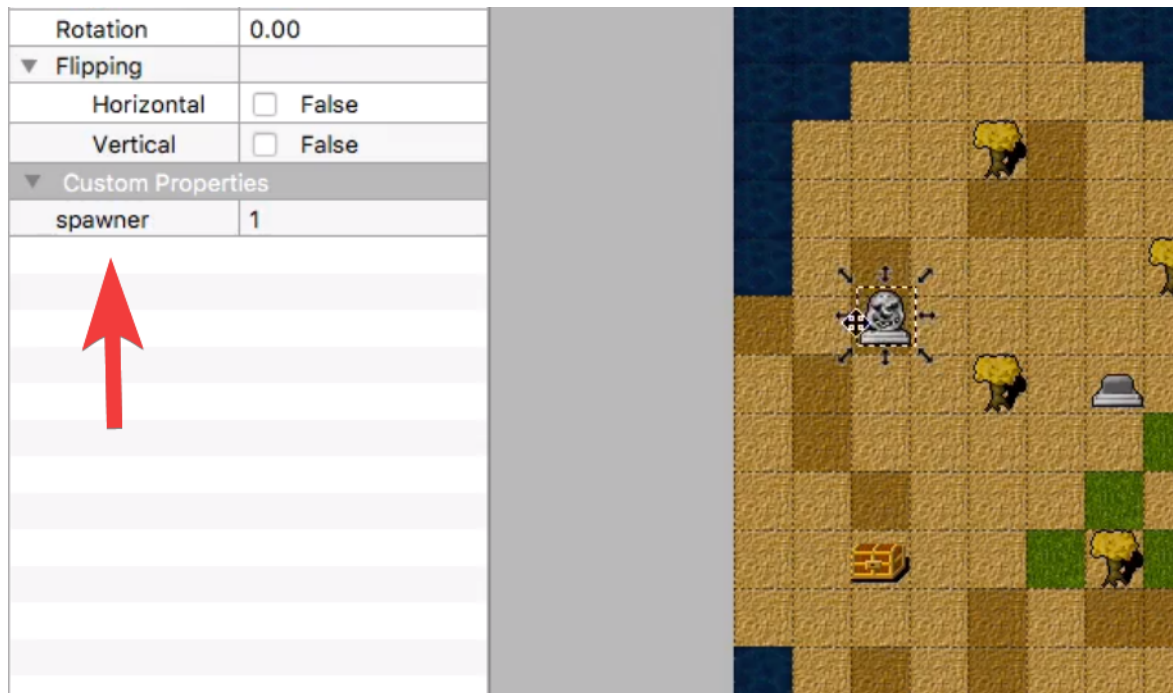
Necessary Elements

For creating your own level, if you want to do that, you will need to include some key features.

You'll need two different tile layers, a **background** layer and a **blocked** layer. The **background** layer will be all of the background details, like the grass, sand, or rocky terrain. The **blocked** layer will need to include any tiles that you want the player to collide with, such as walls, water, or different items in the game, like trees or columns.

You'll also need to include three different object layers. A **chest_locations** layer, a **monster_locations** layer, and a **player_locations** area. These three layers will be used to control the spawn locations of the various elements in your game. You can use the **Insert Tile** tool to add appropriate tiles for each of the different types of spawners. The chest tiles, a monster looking tombstone, and the gravestone can be tiles that you use for the different layers. Feel free to use any tiles or shapes that are meaningful to you.

One thing to note about the monster and chest location objects is that each one has a custom property called **spawner**.



This property will be used to identify which quadrant that spawner is responsible for generating things into. The spawners will spawn things into the map randomly in their assigned quadrant.

This will be the level that will be used for the rest of the course videos, but feel free to create and use your own, as long as you include the necessary elements.

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, you will learn how to load the Tiled level into your Phaser game.

Adding the Assets to the Game

The first step that you need to do is download and unzip the **Course Files** that have been provided, if you have not already done so. In it, you will find a copy of the **level** folder where you got the **background.png** file. If it is not in there, copy it over now. If you are using your own map, copy the JSON file into the **level** folder.

Now, you will be starting the coding process. Open up the **js** folder, then the **scenes** folder, and open the **BootScene.js** file with your preferred code editor. If you use an editor that allows you to open a folder as a working directory, use either the full **assets** folder or the **js** folder.

The first change that you'll need to make is to the **create** method, which is at the bottom of the file. See the code below:

```
create() {  
    this.scene.start('Game');  
}
```

This changes which scene will be loaded during the initial start-up of the game.

Next, you'll need to add a new function call to the **preload** method called **loadTileMap**. See the code below:

```
preload() {  
    // load images  
    this.loadImages();  
    // load spritesheets  
    this.loadSpriteSheets();  
    // load audio  
    this.loadAudio();  
    // load tilemap  
    this.loadTileMap();    // New code  
}
```

Now you can write this new function. It should go below all the other **load...** functions. See the code below:

```
/*
```

```
Other load functions
```

```
*/
```

```
loadTileMap() {  
    // map made with Tiled in JSON format  
    this.load.tilemapTiledJSON('map', 'assets/level/large_level.json');
```

```
}
```

If you are using your own level, write in the appropriate file name here instead of **large_level**. The next step in this process is to load in the tile set that was used in the creation of the map. This is done in the **loadImages** function. See the code below:

```
loadImages() {  
  this.load.image('button1', 'assets/images/ui/blue_button01.png');  
  this.load.image('button2', 'assets/images/ui/blue_button02.png');  
  // load the map tileset image  
  this.load.image('background', 'assets/level/background-extruded.png');  
}
```

Creating the Background Layer

Now that the code to load the assets has been written, you can write the code to create the level in the game. This will be done in the **GameScene.js** file. The first change you'll need to make is to the **create** method. You'll be adding a new method called **createMap**. See the code below:

```
create() {  
  this.createAudio();  
  this.createChest();  
  this.createWalls();  
  this.createPlayer();  
  this.addCollisions();  
  this.createInput();  
  this.createMap();  
}
```

You can add the **createMap** method to the bottom of the file. See the code below:

The following code has been updated, and differs from the video:

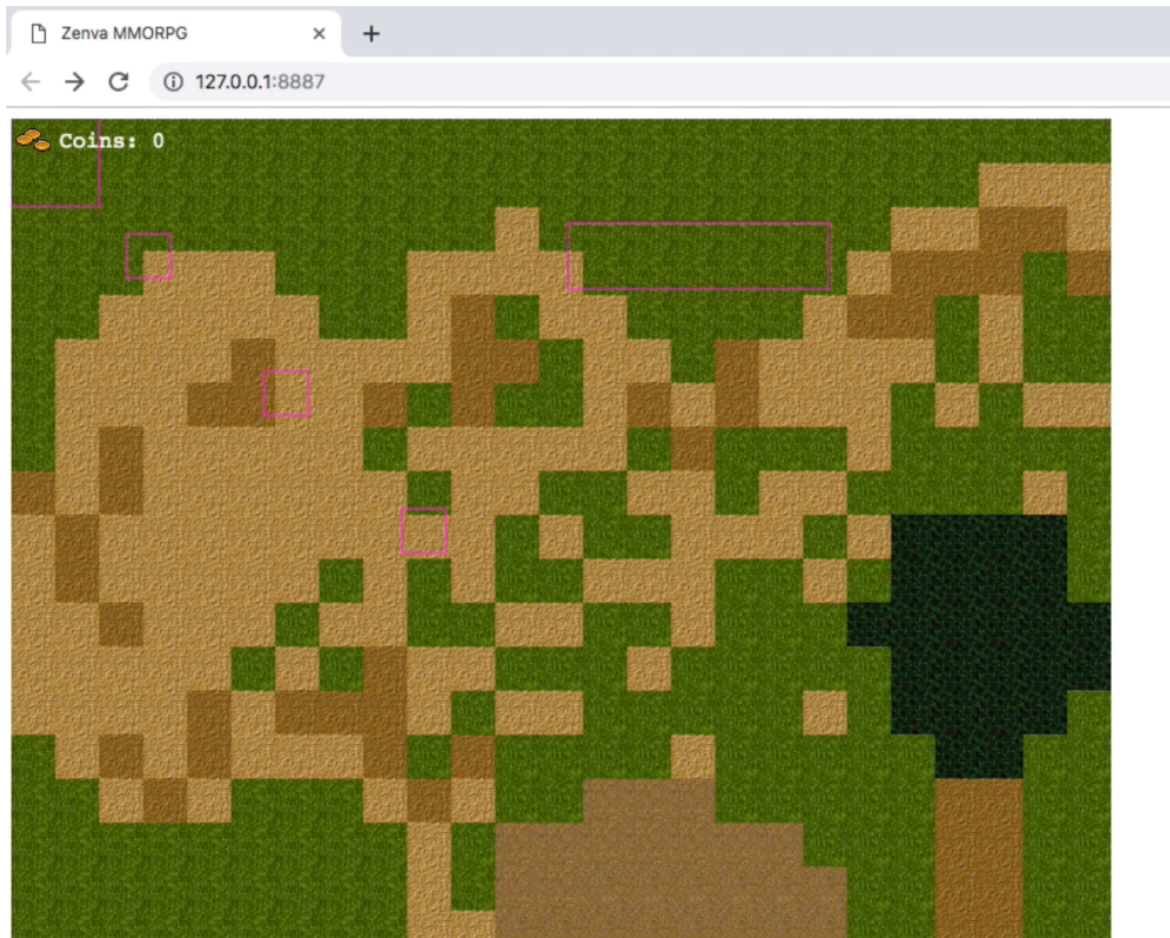
For **newer** versions of Phaser (e.g., v3.87) **use** `createLayer()` instead of `createStaticLayer()` as follows:

```
createMap() {  
  // create the tile map  
  this.map = this.make.tilemap({ key: 'map' });  
  // add the tileset image to our map  
  this.tiles = this.map.addTilesetImage('background', 'background', 32, 32, 1, 2);  
  // create our background  
  this.backgroundLayer = this.map.createLayer('background', this.tiles, 0, 0);  
}
```

To create the tile map, you'll use the **make.tilemap** method, providing an object with a **key** that is the name of the asset you loaded in earlier with the **loadTileMap** function. Next, to add the tiles to

the map, you'll need to use the **map.addTilesetImage** method, which has 6 arguments. The first is the name of the Phaser layer that you are wanting to create. For this layer, it is **background**. The second argument is the name of the tileset image, which is also **background**. The third and fourth arguments are the frame width and frame height of the tiles. The fifth and sixth arguments are the margin and spacing inside of the image. To finalize the background, you'll need to use the **createLayer** method, which takes 4 arguments. The first is the name of the layer, next is the tiles that you loaded, and the final two are the x and y position for the starting point of the layer.

When you finish this, save your file, and refresh your game. You should now see the background image loaded, which should look like the following:



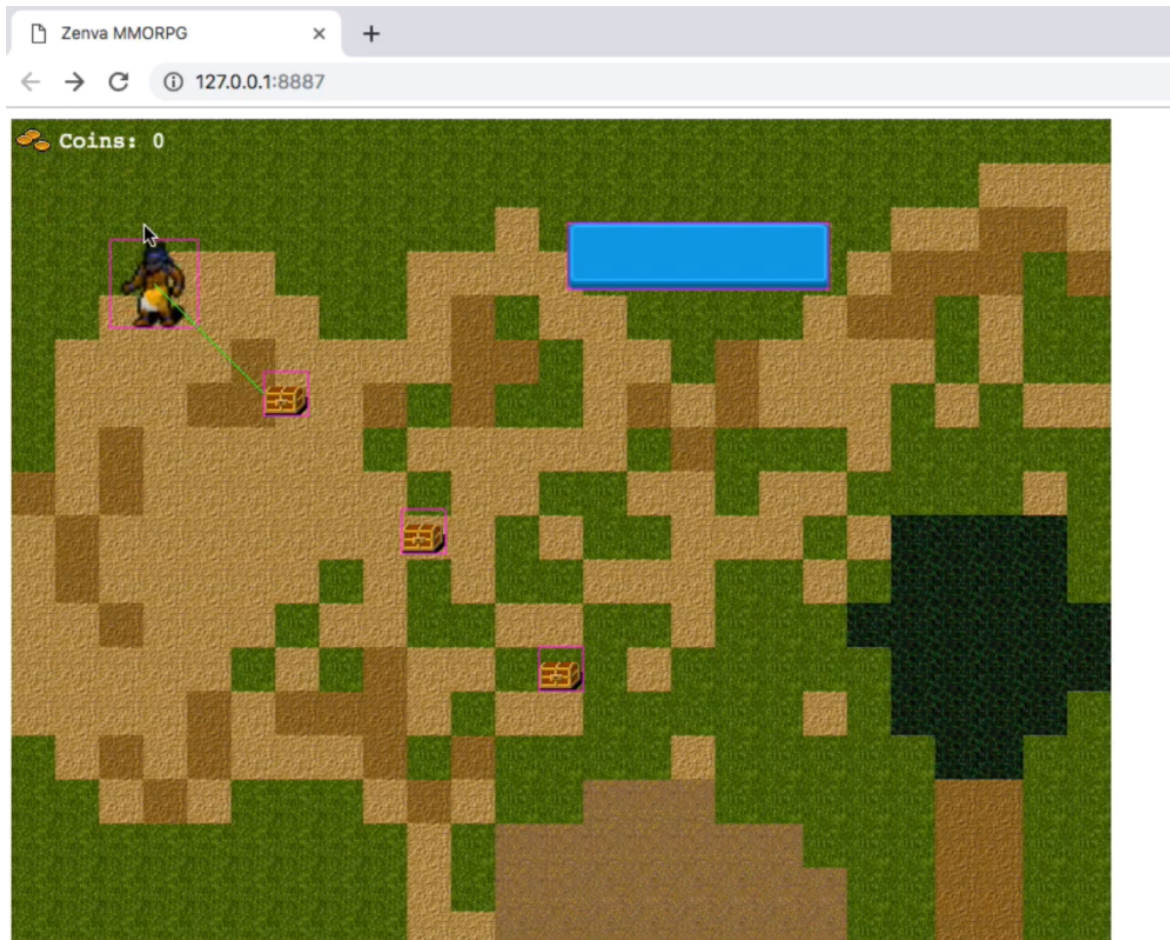
Part of the level has been loaded, specifically the background. As the **blocked** layer has not been added yet, it is not being rendered. There is an issue though. The player and the chests are being rendered over, which is why they are not showing up. The reason for this is that the **createMap()** method is at the bottom of the **create()** method. This means it is performed last in the rendering process. As you want this to be done first, move the **createMap()** method to the top. See the code below:

```
create() {  
  this.createMap();    // moved this method  
  this.createAudio();  
  this.createChests();  
  this.createWalls();  
  this.createPlayer();  
  this.addCollisions();  
}
```




```
this.createInput();  
}
```

Now save the file and refresh your game. The player and chests should now be on top of the background. It should look like this:



Creating the Blocked Layer

The next step in adding the Tiled map is to add the **blocked** layer. This will also be done in the **createMap()** method, using the same **map.createLayer** method that you used earlier. See the code below:

The following code has been updated, and differs from the video:

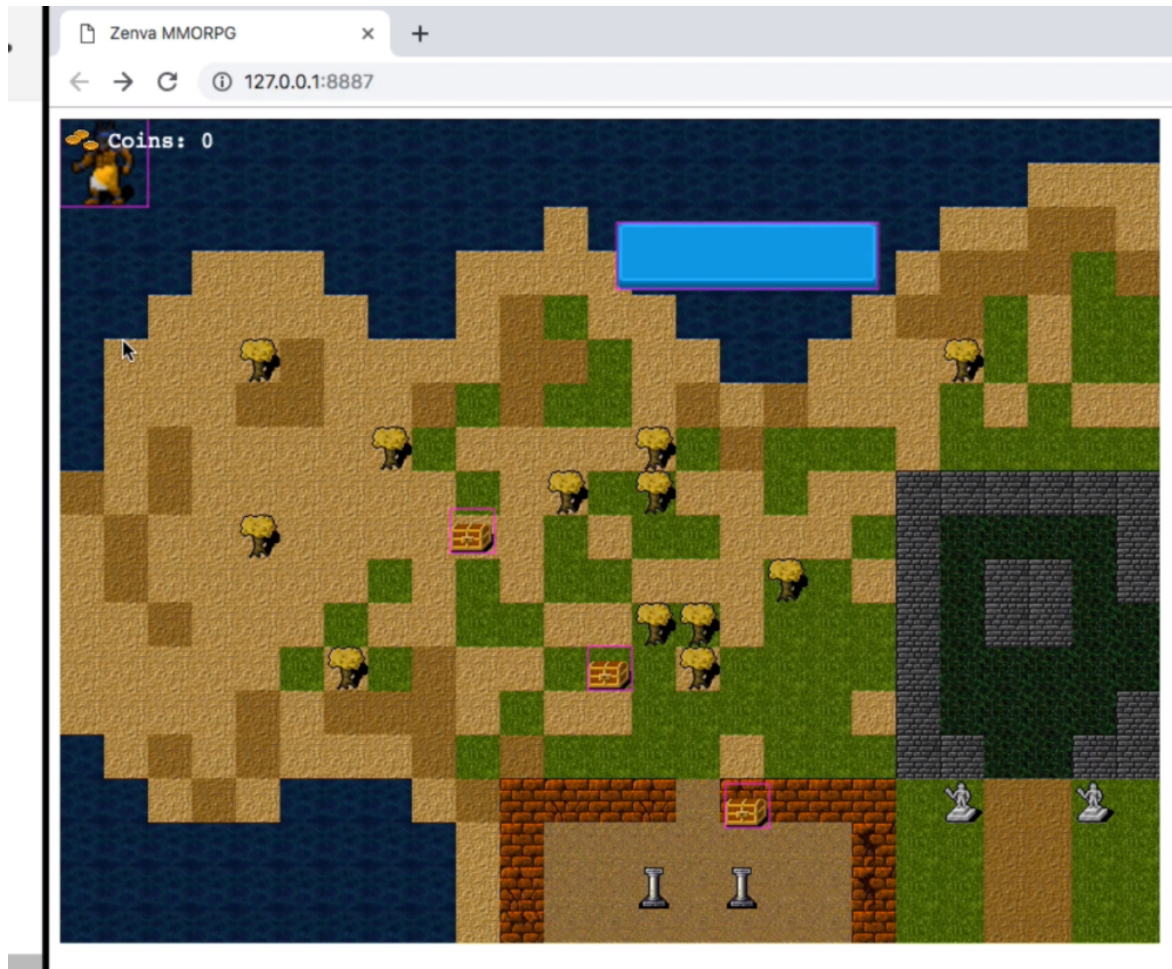
For **newer** versions of Phaser (e.g., v3.87) **use** `createLayer()` instead of `createStaticLayer()` as follows:

```
createMap() {  
    // create the tile map  
    this.map = this.make.tilemap({ key: 'map' });  
    // add the tileset image to our map  
    this.tiles = this.map.addTilessetImage('background', 'background', 32, 32, 1, 2);  
    // create our background
```



```
this.backgroundLayer = this.map.createLayer('background', this.tiles, 0, 0);  
// create blocked layer  
this.blockedLayer = this.map.createLayer('blocked', this.tiles, 0, 0);    // New code  
}  
}
```

Go ahead and save the file and refresh your game. This should update your game so that it will render the **blocked** layer. It should look like this:



Changing the Scale of the Tiles

You may notice that the player is very large on the map. This is because it was scaled up earlier. To correct this issue, you'll need to use the **setScale()** method for both the **background** and **blocked** layer. See the code below:

The following code has been updated, and differs from the video:

For **newer** versions of Phaser (e.g., v3.87) **use** `createLayer()` instead of `createStaticLayer()` as follows:

```
createMap() {  
    // create the tile map  
    this.map = this.make.tilemap({ key: 'map' });  
}
```

```
// add the tileset image to our map
this.tiles = this.map.addTilesetImage('background', 'background', 32, 32, 1, 2);
// create our background
this.backgroundLayer = this.map.createLayer('background', this.tiles, 0, 0);
this.backgroundLayer.setScale(2);    // New code
// create blocked layer
this.blockedLayer = this.map.createLayer('blocked', this.tiles, 0, 0);
this.blockedLayer.setScale(2);    // New code
}
```

Save and refresh again, and your player should not match the save of your map appropriately. It should look like this:



Cleaning Up the Code

The final step for this lesson is to clean up the code a little. You no longer need the **createWalls()** method, as that was just done for testing, so delete both the **createWalls()** method and the **this.createWalls()** method call in the **create()** method. If you save and refresh now, you'll see that the wall is gone.



In summary, the two main methods for adding a Tiled map to a Phaser game are **this.load.tilemapTiledJSON()** and **this.make.tilemap()**. These two methods will allow you to load in any Tiled JSON file that you make and use it in Phaser to create lots of different things.



In this lesson, you'll learn how to adjust the bounds of your game world and fix some issues with the camera.

If you go to your game and move your character around, you'll notice that none of the elements of the **blocked** layer actually prevent the player from moving through them. Also, the camera view is fixed, preventing the player from moving to a different region of the game map. To fix these issues, you'll need to add some new code to your files.

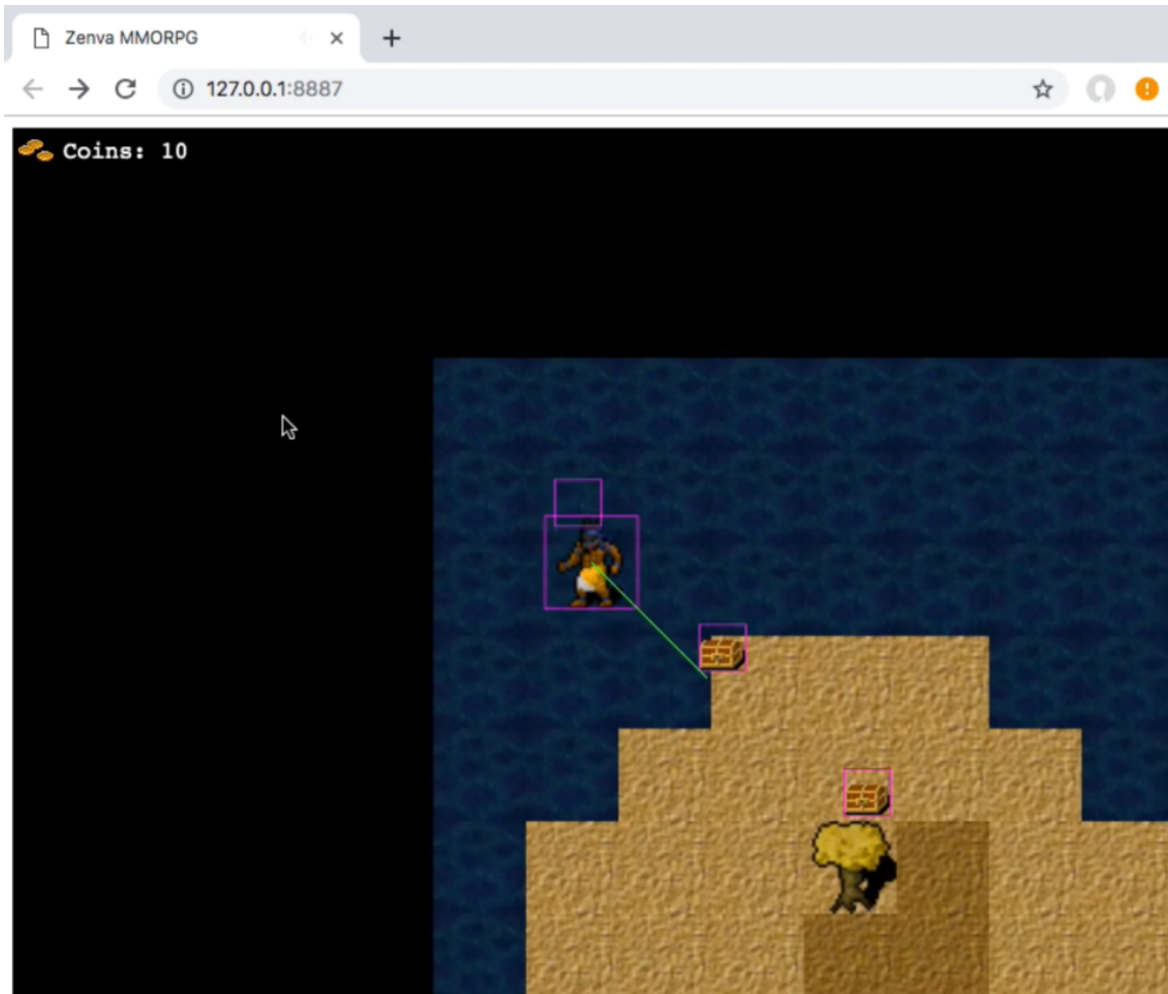
Fixing the Camera and World Bounds

The first step will be to make the camera follow the player. This will allow the player to actually move around the map, instead of being locked into the area shown when the game starts. To do this, you'll need to add a new method to the **class Player**. See the code below:

```
class Player extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);
    this.scene = scene; // the scene this container will be added to
    this.velocity = 160; // the velocity when moving our player

    // enable physics
    this.scene.physics.world.enable(this);
    // set immovable if another object collides with our player
    this.setImmovable(false);
    // scale our player
    this.setScale(2);
    // collide with world bounds
    this.setCollideWorldBounds(true);
    // add the player to our existing scene
    this.scene.add.existing(this);
    // have the camera follow the player
    this.scene.cameras.main.startFollow(this);    // New code
  }
}
```

This method, **startFollow**, needs a game object to track as the argument, which is why you need to pass in **this** so that the **Player** object is tracked. Now save and refresh your game. The camera will be centered on the player. It should look like this:



An odd thing happens now. If you walk the character to the right or down, you'll hit an invisible barrier. This is because the world bounds are still set to the original canvas size, instead of being the full map. To fix this new issue, you'll have to go to the **GameScene.js** file and update the **createMap()** method to update the world bounds. See the code below:

```
createMap() {
  // create the tile map
  this.map = this.make.tilemap({ key: 'map' });
  // add the tileset image to our map
  this.tiles = this.map.addTilesetImage('background', 'background', 32, 32, 1, 2);
  // create our background
  this.backgroundLayer = this.map.createStaticLayer('background', this.tiles, 0, 0);
  this.backgroundLayer.setScale(2);
  // create blocked layer
  this.blockedLayer = this.map.createStaticLayer('blocked', this.tiles, 0, 0);
  this.blockedLayer.setScale(2);

  // New code
  // update the world bounds
  this.physics.world.bounds.width = this.map.widthInPixels * 2;
  this.physics.world.bounds.height = this.map.heightInPixels * 2;
}
```

This updates the physics world bounds directly using the size of the map in pixels. The reason that you need to multiply by 2 is because the game world was scaled up. Save and refresh your game. You should now be able to move past the invisible barriers that were there previously.

The final fix for the camera is to prevent it from showing the areas outside of the map. As Phaser allows for infinite spaces, the game shows a black area in the region outside of the map. To prevent this from happening, the **createMap()** method needs to be updated again. This time, you'll be limiting the camera to the size of the map. See the code below:

```
// limit the camera to the size of our map
this.cameras.main.setBounds(0, 0, this.map.widthInPixels * 2, this.map.heightInPixels * 2);
```

Add this code below the world bounds code. The **setBounds** method takes in 4 arguments. The first two are the starting coordinates, X and Y. The third and fourth arguments are the width and height of the bounding box. Save and refresh your game. You should no longer see the black area outside of your map, like this:



Make sure that you can explore the whole map. The only things your player should run into are the edges of the map. In the next lesson, you'll be learning how to set up the **blocked** layer so that the player will impact more things.

Tips for the Phaser Camera



Here are three helpful tips about the camera system for Phaser:

- By default, the camera is bound to the size of the canvas that is created. It will also be focused on the center of that bounded area.
- Phaser 3 supports running multiple cameras at once.
- The code to get the main camera to follow a game object is the following:

```
this.scene.cameras.main.startFollow('gameObject');
```

Also, remember that the Phaser 3 website has a lot of documentation about the different systems it uses, so if you ever want to learn more in-depth about a particular method you can go there.



In this lesson, you will learn how to set collisions for the **blocked** layer of your map, so that the player will collide with the appropriate objects in your game. You'll also learn more about the configuration properties of Phaser 3, specifically for displaying pixel art.

Setting Up the Collisions

The first step for adding collisions is to add a new method to the **createMap()** method. See the code below:

```
// create blocked layer
this.blockedLayer = this.map.createStaticLayer('blocked', this.tiles, 0, 0);
this.blockedLayer.setScale(2);
this.blockedLayer.setCollisionByExclusion([-1]);    // New code
```

This method, **setCollisionByExclusion**, takes in an array to determine which tiles should be excluded from being checked. Using an array value of **[-1]** means that all of the tiles in the layer will be checked for collisions.

Now you need to check when the player collides with one of the tiles in the **blocked** layer. This will be done in the **addCollisions()** method. You can replace the **this.wall** with **this.blockedLayer**. See the code below:

```
addCollisions() {
    // check for collisions between player and the tiled blocked layer
    this.physics.add.collider(this.player, this.blockedLayer);
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, this);
}
```

Save and refresh your game. Your player will still spawn in the same spot, the upper left in the water, but when you move outside of that area, those tiles will collide with the player and prevent you from going back. You also should collide with the trees now, if you want to check that.

Updating the Player's Starting Position

Now that the **blocked** layer is blocking things, it makes more sense to spawn somewhere else on your map. To fix this, you just need to change the arguments for the **createPlayer()** method from the two **32** values to two **224** values. See the code below:

```
createPlayer() {
    this.player = new Player(this, 224, 224, 'characters', 0);
}
```

When you save and refresh your game, this should now put your character on the map, right next to the water.



Feel free to use other coordinates for this if you want to spawn somewhere else.

Adjusting the Display System

You may notice that in between the tiles, there are blurry lines. This is because the tiles have been zoomed in on, so they appear grainier. To fix this, Phaser has a few options that can be set with the game configurations to help prevent this issue. You will basically be notifying the Phaser system that you are using Sprites so that when scaling happens, they won't be so expanded visually.

You will need to go to the **main.js** file. You'll need to add two new properties, underneath the **physics** property. These properties are **pixelArt** and **roundPixels**, both of which should be set to true. See the code below:

```
var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  scene: [
    BootScene,
    TitleScene,
    GameScene,
    UiScene,
  ],
  physics: {
    default: 'arcade',
    arcade: {
      debug: true,
```




```
        gravity: {
            y: 0,
        },
    },
},
pixelArt: true,    // New code
roundPixels: true,  // New code
};
```

Now save and refresh your game. The tiles should look a little more clear now, like this:



These two properties help out when displaying sprites. The **pixelArt** property is just a flag in Phaser, notifying the system so that it will be more optimized for displaying this kind of image. The **roundPixels** property allows Phaser to display moving pixel characters better. It will round the floating-point number of a pixel's location to the nearest integer so that it can be rendered cleanly.



In this lesson, you will learn how to create and use a new map class.

Making the New Class

To keep the **GameScene.js** file more organized, you will now move some of the logic from the **createMap** method to its own class. This will also allow you to reuse this code for other projects, a helpful feature of using classes.

In your **classes** folder, make a new file and name it **Map.js**. In this file, you'll be making a new class. See the code below:

```
class Map {  
  constructor() {}  
}
```

Before you continue with making the class, you should make sure to update the **index.html** file to include your new class. See the code below:

```
<body>  
  <div id="phaser-game"></div>  
  <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>  
  <script src="assets/js/classes/Player.js" charset="utf-8"></script>  
  <script src="assets/js/classes/Chest.js" charset="utf-8"></script>  
  <script src="assets/js/classes/UIButton.js" charset="utf-8"></script>  
  <script src="assets/js/classes/Map.js" charset="utf-8"></script>    // New code  
  <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>  
  <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>  
  <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>  
  <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>  
  <script src="assets/js/main.js" charset="utf-8"></script>  
</body>
```

Doing this now just makes it so that the class will be loaded, and so you won't have to worry about doing it later.

You'll now want to copy the **createMap** method from the **GameScene.js** file and paste it in your **class Map**, underneath the **constructor**. Then in the **constructor**, you'll call the method. See the code below:

```
class Map {  
  constructor() {  
    this.createMap();    // New code  
  }  
  
  createMap() {  
    // All the copied createMap code goes here  
  }  
}
```



Now you can refactor the code to make it function as an independent class. For the **constructor**, you'll need to pass in a reference to the **scene** that the map will belong to. You'll then need to set that variable. See the code below:

```
constructor(scene) {  
    this.scene = scene; // the scene this map belongs to  
    this.createMap();  
}
```

Next, you'll have to update the **createMap** method to use the new **scene** reference. This will be done for making the tile map, updating the world bounds, and setting the camera size. See the code below:

```
createMap() {  
    // create the tile map  
    this.map = this.scene.make.tilemap({ key: 'map' });    // New code  
  
    // add the tileset image to our map  
    this.tiles = this.map.addTilesetImage('background', 'background', 32, 32, 1, 2);  
  
    // create our background  
    this.backgroundLayer = this.map.createStaticLayer('background', this.tiles, 0, 0);  
    this.backgroundLayer.setScale(2);  
  
    // create blocked layer  
    this.blockedLayer = this.map.createStaticLayer('blocked', this.tiles, 0, 0);  
    this.blockedLayer.setScale(2);  
    this.blockedLayer.setCollisionByExclusion([-1]);  
  
    // update the world bounds  
    this.scene.physics.world.bounds.width = this.map.widthInPixels * 2;    // New code  
    this.scene.physics.world.bounds.height = this.map.heightInPixels * 2;    // New code  
e  
  
    // limit the camera to the size of our map  
    this.scene.cameras.main.setBounds(0, 0, this.map.widthInPixels * 2, this.map.heightInPixels * 2);    // New code  
}
```

You'll now have to go back to the **GameScene.js** file. You can delete all the logic that is currently in the **createMap** method, as you have that in the new **Map** class. Instead, you will need to set the **this.map** variable to a new **Map** object. See the code below:

```
createMap() {  
    // create map  
    this.map = new Map(this);  
}
```

The next step is to change all the hardcoded key values into arguments that will be passed to



the **constructor** of the **Map** class. You'll need to add four parameters to the **constructor** and set all these parameters. See the code below:

```
constructor(scene, key, tileSetName, bgLayerName, blockedLayerName) {  
    this.scene = scene; // the scene this map belongs to  
    this.key = key; // Tiled JSON file key name  
    this.tileSetName = tileSetName; // Tiled Tileset image key name  
    this.bgLayerName = bgLayerName; // the name of the layer created in tiled for the m  
    ap background  
    this.blockedLayerName = blockedLayerName; // the name of the layer created in tiled  
    for the blocked areas  
  
    this.createMap();  
}
```

Now you'll need to update the **createMap** method in the **GameScene.js** file to pass in the new parameters that you'll be using in the **Map** class. These are just the key values you used in the **createMap** method, so feel free to copy and paste them over to speed up your workflow. See the code below:

```
createMap() {  
    // create map  
    this.map = new Map(this, 'map', 'background', 'background', 'blocked');  
}
```

You can now use the new variables in the **createMap** method of the **Map** class to replace all the previously hardcoded key values. This is what will allow you to use this class with other game projects, as you won't need to always go in and manually change the key values. See the code below:

```
createMap() {  
    // create the tile map  
    this.map = this.scene.make.tilemap({ key: this.key });    // New code  
  
    // add the tileset image to our map  
    this.tiles = this.map.addTilesetImage(this.tileSetName, this.tileSetName, 32, 32, 1  
    , 2);    // New code  
  
    // create our background  
    this.backgroundLayer = this.map.createStaticLayer(this.bgLayerName, this.tiles, 0,  
    0);    // New code  
    this.backgroundLayer.setScale(2);  
  
    // create blocked layer  
    this.blockedLayer = this.map.createStaticLayer(this.blockedLayerName, this.tiles, 0  
    , 0);    // New code  
    this.blockedLayer.setScale(2);  
    this.blockedLayer.setCollisionByExclusion([-1]);  
  
    // update the world bounds  
    this.scene.physics.world.bounds.width = this.map.widthInPixels * 2;
```



```
this.scene.physics.world.bounds.height = this.map.heightInPixels * 2;

// limit the camera to the size of our map
this.scene.cameras.main.setBounds(0, 0, this.map.widthInPixels * 2, this.map.heightInPixels * 2);
}
```

Note that even though the key values were the same for the tileset image code and the background layer code, they each have distinct variables. This will again be useful for using this class with other games, in case you use a different naming scheme.

Make sure to save all your files now, then refresh your game to check for changes. Unfortunately, you'll notice that the **blocked** layer elements no longer collide with the player. This is because the **Map** class is separate from the **GameScene.js** file. Luckily it can be fixed very easily. You'll need to update the **addCollisions** method. See the code below:

```
addCollisions() {
    // check for collisions between player and the tiled blocked layer
    this.physics.add.collider(this.player, this.map.blockedLayer);    // New code
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, this);
}
```

Adding in the **map.** before the **blockedLayer** makes it so that the **map** variable will be accessed correctly. Save and refresh your game to verify that the collisions are working again.

It is good practice to check the console when testing your game, as you'll be able to check for any minor errors. If you have some, double-check that you spelled things correctly and consistently, as that is commonly the problem.

In this lesson, you'll learn how to create a Game Manager class, which will be used to input the other elements of your Tiled map into your Phaser game, like the Spawners and chest locations.

Overview of the New Classes

In order to have the elements of your game spawn randomly, as is desired, you'll need to create three new classes.

The first will be a **GameManager** class. This class will parse the JSON file for your Tiled map for the locations of possible player, chest, and monster locations.

Next, you'll have a **Spawner** class. This class will be responsible for generating chests or monsters at specific intervals up to a specified limit. It will utilize the spawner ID that you gave the chest locations in Tiled to generate unique spawners for each location. The spawner object will need to choose random locations around it, up to the limit of three, and keep track of where it spawns objects. That way, the chests will always be spawned at new locations. It will also keep track of what items it spawned so that it can delete them if necessary. The same kind of logic will apply for spawning monsters.

The last class will be a **ChestModel** class. This class will store the x and y coordinates of the object, the amount of gold stored within it, and the ID value of both the chest and the spawner that created it. This class will be similar to the existing class for chests, but this one will not extend any Phaser objects to allow for easier reuse in other projects.

Creating the GameManager Class

Go to your **assets** folder, create a new folder, and name it **game_manager**. In this folder, make 3 new files and name them **GameManager.js**, **Spawner.js**, and **ChestModel.js**. Go ahead and create a quick class outline for the **ChestModel**. See the code below:

```
class ChestModel {  
  constructor() {}  
}
```

Copy this code and paste it to the other two files, changing the name to match.

Now go to the **index.html** file and add the three new classes to your list. You'll need to change the folder from **classes** to **game_manager**. Using copy and paste will make this go much faster. See the code below:

```
<body>  
  <div id="phaser-game"></div>  
  <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>  
  <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>  
  // New code  
  <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>    /  
  / New code  
  <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>    // N  
  ew code  
  <script src="assets/js/classes/Player.js" charset="utf-8"></script>  
  <script src="assets/js/classes/Chest.js" charset="utf-8"></script>  
  <script src="assets/js/classes/UIButton.js" charset="utf-8"></script>  
  <script src="assets/js/classes/Map.js" charset="utf-8"></script>
```



```
<script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
<script src="assets/js/main.js" charset="utf-8"></script>
</body>
```

With this step done, you can begin creating the **GameManager** class. The first step will be to add in the necessary parameters to the **constructor** and create the needed properties for the class. See the code below:

```
constructor(scene, mapData) {
  this.scene = scene;
  this.mapData = mapData;

  this.spawnners = {};
  this.chests = {};

  this.playerLocations = [];
  this.chestLocations = {};
  this.monsterLocations = {};
}
```

The **scene** parameter will be for what **scene** the **GameManager** object is in. The **mapData** parameter will be the array of layers that has the game data needed for this class. The **spawnners** and **chests** properties will be used to keep track of the existing spawnners and chests when they are generated. The other three location properties will be used when the spawnners are stopped.

Next, you'll need to create a new method called **setup**. This method will call a couple of other methods that you will need to create. See the code below:

```
setup() {
  this.parseMapData();
  this.setupEventListener();
  this.setupSpawnners();
  this.spawnPlayer();
}

parseMapData() {
}

setupEventListener() {
}

setupSpawnners() {
}

spawnPlayer() {
```



```
}
```

The **parseMapData** method will be used to parse the layer data that was exported from Tiled, which will be used to generate the three locations. The **setupEventListener** method will be used to create any event listeners that will need to be hooked up to the **GameManager** object, such as when a player picks up a chest. Currently, the chest pick-up event is being handled in the **GameScene.js** file. With this method, you'll set up a system so that the **GameManager** can handle everything. The **setupSpawners** method will use the data from the **parseMapData** method to create the spawners in the appropriate locations on the map. The **spawnPlayer** method will allow the code to spawn the player at a random location that has been designated for a player spawn point.

The next step is to go to the **GameScene.js** file to initialize the **GameManager**. You'll need to add a new method to the **create** method. See the code below:

```
create() {  
    this.createMap();  
    this.createAudio();  
    this.createChests();  
    this.createPlayer();  
    this.addCollisions();  
    this.createInput();  
  
    this.createGameManager();    // New code  
}
```

Now you'll need to create this new method. See the code below:

```
createGameManager() {  
    this.gameManager = new GameManager(this, this.map.map.objects);  
    this.gameManager.setup();  
}
```

The **map.map.objects** parameter will include all of the object layers that you created in Tiled.

Next, to check on your progress, go back to the **GameManager.js** file and update the **parseMapData** method to send the **mapData** to the console. See the code below:

```
parseMapData() {  
    console.log(this.mapData);  
}
```

Save all of your files, and refresh your game. Check on the console, you should see something like this:



```
GameManager.js:22
▼ (3) [initialize, initialize, initialize] ⓘ
  ▶ 0: initialize {name: "chest_locations", opacity: ...
  ▶ 1: initialize {name: "monster_locations", opacity...
  ▶ 2: initialize {name: "player_locations", opacity:...
      length: 3
  ▶ __proto__: Array(0)
```

As you can see, there are three objects. Each of the objects has the name of the layer from Tiled. Inside of each, there is another array containing all of the objects of that layer, with information about the location and the spawner ID.

In the upcoming lessons, you'll be finishing up the methods of the **GameManager** class. Remember that by abstracting the logic of the **GameManager** into a separate class, you can easily re-use it for other games or different situations. Creating these kinds of modular systems makes it easier to create more games with less repeated effort.

In this lesson, you will be updating the **GameManager** to parse the data from the Tiled map and spawn the player.

Parsing the Tiled Map Data

In the **parseMapData** method, you'll be adding some new code to handle the map data, which is an array. You'll need to use a **forEach** loop to go through it. Inside the **forEach**, you'll need to create a new function that will check each **layer.name**. Use the following code:

```
parseMapData() {  
  console.log(this.mapData);  
  this.mapData.forEach((layer)=> {  
    if (layer.name === 'player_locations') {  
  
    } else if (layer.name === 'chest_locations') {  
  
    } else if (layer.name === 'monster_locations') {  
  
    }  
  });  
}
```

For the **'player_locations'** if statement, you'll need to use another **forEach** loop. This will go through all of the possible player spawn points and allow you to store all of them into the **player_locations** array. The code below shows how to do this:

```
if (layer.name === 'player_locations') {  
  layer.objects.forEach((obj) => {  
    this.playerLocations.push([obj.x, obj.y]);  
  });  
}
```

Next, you need to write the code to parse the **'chest_locations'** data. This will be done differently than the previous block of code, as the data will be stored to an object rather than an array. This is so that each key of the object can be mapped to a particular spawner ID, allowing the code to know all of the possible spawn locations associated with it. You'll need an **if/else** statement to handle the data parsing logic. If the particular **spawner** that is being checked already exists in the **chestLocations** object, then the new location information will be stored to that array. If the **spawner** ID does not have an associated key, it will be created and added to the **chestLocations** object. The following code shows how this is done:

```
else if (layer.name === 'chest_locations') {  
  layer.objects.forEach((obj) => {  
    if (this.chestLocations[obj.properties.spawner]) {  
      this.chestLocations[obj.properties.spawner].push([obj.x, obj.y]);  
    } else {  
      this.chestLocations[obj.properties.spawner] = [[obj.x, obj.y]];  
    }  
  });  
}
```


The **'monster_locations'** block will be essentially the same as the **'chest_locations'** one, simply change the **chestLocations** to **monsterLocations**. Use the following code:

```
else if (layer.name === 'monster_locations') {
  layer.objects.forEach((obj) => {
    if (this.monsterLocations[obj.properties.spawner]) {
      this.monsterLocations[obj.properties.spawner].push([obj.x, obj.y]);
    } else {
      this.monsterLocations[obj.properties.spawner] = [[obj.x, obj.y]];
    }
  });
}
```

To validate that everything is working correctly, add a **console.log** for each of the locations that you just updated. The code below shows how to do this. Make sure that it is above the final curly bracket of the **parseMapData** method:

```
});

console.log(this.playerLocations);
console.log(this.monsterLocations);
console.log(this.chestLocations);
}
```

When you run your game and check the console, you should see the following:



If you used a custom Tiled map, your console will look different, but it should be similar, with the **playerLocations** being multiple arrays of 2, and the other two being an object with key values that have arrays of varying sizes.

Now that the Tiled map data is being parsed correctly, you can clean up the **parseMapData** method by removing the **console.log** calls. The finished **parseMapData** method should look like this:

```
parseMapData() {
  this.mapData.forEach((layer) => {
    if (layer.name === 'player_locations') {
      layer.objects.forEach((obj) => {
        this.playerLocations.push([obj.x, obj.y]);
      });
    } else if (layer.name === 'chest_locations') {
      layer.objects.forEach((obj) => {
```



```
        if (this.chestLocations[obj.properties.spawner]) {
            this.chestLocations[obj.properties.spawner].push([obj.x, obj.y]);
        } else {
            this.chestLocations[obj.properties.spawner] = [[obj.x, obj.y]];
        }
    });
} else if (layer.name === 'monster_locations') {
    layer.objects.forEach((obj) => {
        if (this.monsterLocations[obj.properties.spawner]) {
            this.monsterLocations[obj.properties.spawner].push([obj.x, obj.y]);
        } else {
            this.monsterLocations[obj.properties.spawner] = [[obj.x, obj.y]];
        }
    });
}
});
}
```

Spawning the Player

Now that the data from the Tiled map has been parsed and you have an array of potential player spawn points, you can write the code that will spawn the player into the game. For this game, you'll want to spawn the player in at a random spawn location. This will be done by creating a new variable, **location**, and setting it to a randomly selected index value in the **playerLocations** array. This **location** variable will then be used by the **emit** method to trigger the appropriate event and spawn the player at the randomly selected location. The following code shows how to do this:

```
spawnPlayer() {
    const location = this.playerLocations[Math.floor(Math.random() * this.playerLocations.length)];
    this.scene.events.emit('spawnPlayer', location);
}
```

As the Phaser scene has been passed into the **GameManager** class, you can emit events that can be picked up in the **GameScene** class. Go to your **GameScene.js** file. In the **createGameManager** method, you'll be adding the code to listen for the **spawnPlayer** event and trigger the player spawning. You can copy this code from the **setupEvents** method in the **UI.js** file and then edit it, or simply rewrite it for practice. You'll need to make sure that you move the **this.createPlayer** and **this.addCollisions** from the **create** method and delete them. You need to move the **addCollisions** method because it relies on the player already existing in the scene. Use the following code:

```
createGameManager() {
    this.events.on('spawnPlayer', (location) => {
        this.createPlayer();
        this.addCollisions();
    });

    this.gameManager = new GameManager(this, this.map.map.objects);
    this.gameManager.setup();
}
```



The next step is to update the **createPlayer** method so that it will take in the **location** variable, as opposed to being set to a fixed value like it currently is. The following code shows how to do this:

```
createPlayer(location) {  
    this.player = new Player(this, location[0], location[1], 'characters', 0);  
}
```

You'll then need to update the **createGameManager** method again to ensure that **createPlayer** uses the **location** variable. Simply add the variable to the **createPlayer** call, like this:

```
createGameManager() {  
    this.events.on('spawnPlayer', (location) => {  
        this.createPlayer(location);    // New code  
        this.addCollisions();  
    });  
  
    this.gameManager = new GameManager(this, this.map.map.objects);  
    this.gameManager.setup();  
}
```

Now, save and refresh your game. Each time you refresh, the player should spawn in a random location.

A correction needs to be done to the **update** method. As the **createPlayer** method is now only done when the **spawnPlayer** event is emitted, there is a possibility that the player may not exist when the **update** method is called. To prevent this from crashing the game, simply add an **if** statement to check for this. The code below shows how to do this:

```
update() {  
    if (this.player) this.player.update(this.cursors);  
}
```

There is, unfortunately, an issue that needs to be corrected. As you refresh your game, you may notice that at some points, the player spawns outside of the boundaries of the map.



This is because the **x** and **y** values of the **playerLocation** array have not been scaled up to match the current map scale. To fix this, simply multiply the **location** values in the **createPlayer** method by 2, like this:

```
createPlayer(location) {  
    this.player = new Player(this, location[0] * 2, location[1] * 2, 'characters', 0);  
}
```

Save and check your game again. The player should no longer be outside the map area for any of the spawn locations. In the upcoming lessons, you'll be learning how to spawn the other elements of the game.



In more recent versions of Tiled, the JSON format for the map is different than the one used in the course. So, while the course code works for the map file provided in the course files, it might not work if you create your own Tiled maps.

In recent versions of Tiled, the object properties in the JSON format are stored in an array as below. Notice that it is an array of objects, where each object has the name of the property, as well as its type and value.

```
"properties":[
    {
      "name":"spawner",
      "type":"string",
      "value":"1"
    }
  ]
```

To update the course code to work with this different JSON format, the first step is to create a new file in called `utils.js` in `assets/js/game_manager/` with the function below. This function will receive as parameters an object from the Tiled JSON format, a property name, and return the value of this property for this object. This can be done by iterating through the object properties until finding the one with the desired name.

```
function getTiledProperty(obj, property_name) {
  for (var property_index = 0; property_index < obj.properties.length; property_index += 1) {
    var property = obj.properties[property_index];
    if (property.name == property_name) {
      return property.value;
    }
  }
}
```

Now, we need to update the `parseMapData` to make use of this function when creating the Chest and Monster spawners. Notice that the only difference is when indexing the `this.chestLocations` and `this.monsterLocations` arrays, we are now using the return from the `getTiledProperty` function.

```
parseMapData() {
  this.mapData.forEach((layer) => {
    if (layer.name === 'player_locations') {
      layer.objects.forEach((obj) => {
        this.playerLocations.push([obj.x, obj.y]);
      });
    } else if (layer.name === 'chest_locations') {
      layer.objects.forEach((obj) => {
        var spawner = getTiledProperty(obj, 'spawner');
        if (this.chestLocations[spawner]) {
          this.chestLocations[spawner].push([obj.x, obj.y]);
        } else {
          this.chestLocations[spawner] = [[obj.x, obj.y]];
        }
      });
    } else if (layer.name === 'monster_locations') {
```




```
layer.objects.forEach((obj) => {
  var spawner = getTiledProperty(obj, 'spawner');
  if (this.monsterLocations[spawner]) {
    this.monsterLocations[spawner].push([obj.x, obj.y]);
  } else {
    this.monsterLocations[spawner] = [[obj.x, obj.y]];
  }
});
});
}
```

Don't forget to include the `utils.js` file in `index.html`. This file will be used later in the course for other things, so you can use the same file you've created now.

```
<script src="assets/js/game_manager/utils.js" charset="utf-8"></script>
```

Now, your game should work with more recent versions of Tiled. Notice that you only need to apply this fix if you experience problems when using new exported maps.



In this lesson, you'll be learning how to set up the **Spawner** class and use it in the **GameManager** class.

Creating the Spawner Class

The **Spawner** class will be created in the **Spawner.js** file. You can also open up the **ChestModel.js** file if you want, but it won't be necessary at first. The first thing you'll need to do is add the properties that will be necessary to make this class work. The following code shows what you need to add:

```
class Spawner {
  constructor() {
    this.id
    this.spawnInterval
    this.limit
    this.objectType
    this.spawnLocations
    this.addObject
    this.deleteObject

    this.objectsCreated = [];
  }
}
```

The **id** property will be used as a unique identifier for each spawner, to allow it to be tracked in the **spawners** object of the **GameManager**. The **spawnInterval** property will be used to check when it is necessary to respawn game objects. It will check against the **limit** property, which will be the maximum number of objects that the spawner can create. The **objectType** property will store what kind of game objects are to be spawned. This is necessary so that the single **Spawner** class can be used for both chests and monsters. The **spawnLocations** property will store all of the possible locations that the spawner can spawn objects to. This will use the locations that you parsed from the map data in a previous lesson. The three properties, **addObject**, **deleteObject**, and **objectsCreated** are all necessary to keep track of how many objects the spawner has created. These three properties will also be used to communicate between the **GameManager** class and the **Spawner** class.

As both classes have information about the spawners and the object locations, there has to be a way for the two classes to share information. When a chest is created or destroyed, both classes have to be updated. With this in mind, the parameters for the **Spawner** class can be set, and the properties can be updated. You can also add a call to a new method, **start**. The code below shows these changes:

```
class Spawner {
  constructor(config, spawnLocations, addObject, deleteObject) {
    this.id = config.id;
    this.spawnInterval = config.spawnInterval;
    this.limit = config.limit;
    this.objectType = config.objectType;
    this.spawnLocations = spawnLocations;
    this.addObject = addObject;
    this.deleteObject = deleteObject;

    this.objectsCreated = [];
  }
}
```



```
    this.start();  
  }  
}
```

These changes just set the values of the properties using the parameters. The next step is to define the new **start** method. This method will be called when the **Spawner** class is initialized and will be used for creating the interval that will be used to spawn objects. This method uses the **setInterval** function, which allows you to define a function that you want to run at defined intervals, in this case, the **spawnInterval** property. The following code shows how to do this:

```
start() {  
  this.interval = setInterval(() => {  
    if (this.objectsCreated < this.limit) {  
      this.spawnObject();  
    }  
  }, this.spawnInterval);  
}
```

Note the new method **spawnObject**. This method is what will actually handle creating the objects. While you're writing new methods, go ahead and create some more that will be necessary: **spawnChest**, **pickRandomLocation**, **removeObject**. Use the following code:

```
spawnObject() {  
  
}  
  
spawnChest() {  
  
}  
  
pickRandomLocation() {  
  
}  
  
removeObject() {  
  
}
```

The rest of these methods will be filled in later. For now, you can update the **spawnObject** method with a simple **console.log**, like this:

```
spawnObject() {  
  console.log('spawning object');  
}
```

Using the Spawner class in the GameManager



Now that the **Spawner** class has some functionality to it, you can test it out. Go to the **GameManager.js** file. In the **setupSpawners** method, you'll need to do a few things. You'll need to set up a **forEach** loop to go through all of the possible chest spawner locations, iterating over the **key** values. This **forEach** will create a new spawner for each of those locations, creating a **config** variable and using that to create a **new Spawner** object. The last step will be to add this spawner to the **spawners** array. The following code shows how to do this:

```
setupSpawners() {
  // create chest spawners
  Object.keys(this.chestLocations).forEach((key) => {
    const config = {
      spawnInterval: 3000,
      limit: 3,
      objectType: 'CHEST',
      id: `chest-${key}`
    };

    // Feel free to leave the parameters on one line. This is for readability
    const spawner = new Spawner(
      config,
      this.chestLocations[key],
      this.addChest.bind(this),
      this.deleteChest.bind(this)
    );
    this.spawners[spawner.id] = spawner;
  });
}
```

You'll need to add the two methods, **addChest** and **deleteChest**, to the **GameManager** class. These will remain empty for now. Use the following code:

```
addChest() {

}

deleteChest() {

}
```

The final step is to save the two files and check your game. You should see the **spawning objects** text in the console. There will be 6 at first, and every 3 seconds another 6 will be added. This is because the **spawnObject** method does not actually add anything to the **objectsCreated** array, so the **limit** value is never hit and the **start** method just continues to run. If you are getting that behavior, you're right on track for where this lesson is going to stop.

Additional Note

In this lesson, you used the **Object.keys** method to loop through all of the keys in an object. What this method does is return an array of all of the keys in the object passed to it. You can then loop through this array using any array traversal method you prefer, with **forEach** being a recommended method.



In the previous lesson, you set up the **Spawner** class. However, it is still not creating any objects. In this lesson, you will be updating the **Spawner** class to create objects, set up **ChestModel** class, and update **Spawner** class to use **ChestModel** class.

Defining the methods to create objects

To handle the creation of objects, you will update the method **spawnObject** in **Spawner.js** file. Use the following code:

```
spawnObject() {  
  console.log('spawning object')  
  if (this.objectType === 'CHEST') {  
    this.spawnChest();  
  }  
}
```

As the **Spawner** class is responsible for creating either a **Monster** or a **Chest**, it will check the object type first. If the object is of type **Chest**, it will call the **spawnChest** method, and **spawnMonster** method will be called if the object type is a **Monster**. Now, go ahead and update the **spawnChest** method in **Spawner.js** file. Use the code below:

```
spawnChest() {  
  const location = this.pickRandomLocation();  
  const chest = new ChestModel();  
  this.addObject();  
}
```

In this method, the variable **location** is set to a random location picked from the array of possible locations using the **pickRandomLocation** method. Once you have the location, create a **Chest** object using the **ChestModel** class. Then, call the **addObject** method to notify the **GameManager** that you created a new chest. These methods will be implemented later in this lesson.

Creating the ChestModel class

Now that you have the basic layout for creating a **chest** object, you will now implement the **ChestModel** class in **ChestModel.js** file. First, you need to add the properties the class needs. The following code shows what you need to add:

```
class ChestModel {  
  constructor() {  
    this.id  
    this.spawnerId  
    this.x  
    this.y  
    this.gold  
  }  
}
```

The **id** property is the unique identifier created to easily track the chest objects created in the

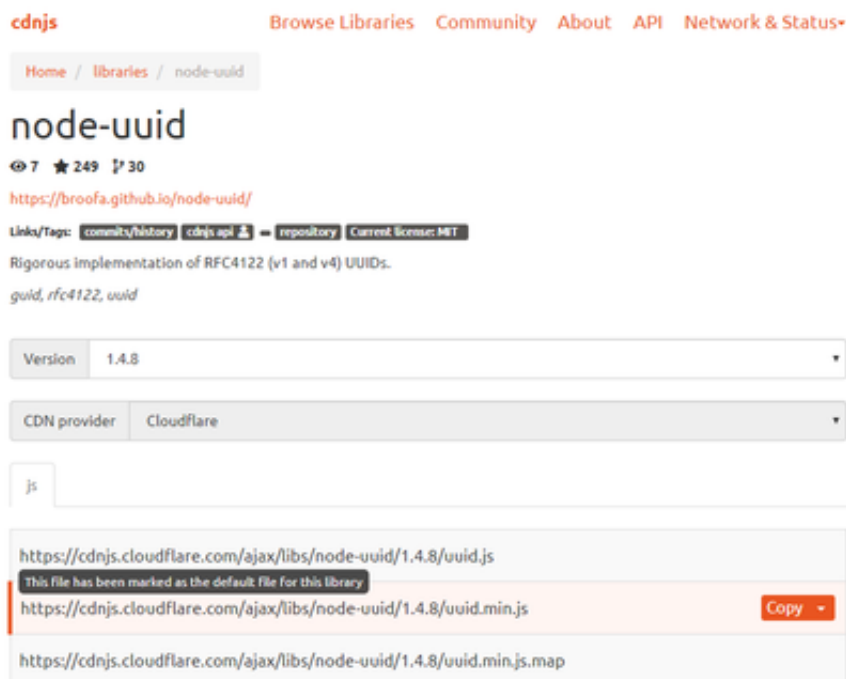
GameManager class. The **spawnerID** property will be used to keep track of which **Spawner** this game object belongs to. The **x** and **y** properties will be used to indicate the position of where this chest object can actually be spawned at, and the **gold** property will hold the amount of gold that is in the chest. The constructor of the **ChestModel** class will take **x** value, **y** value, **gold** and **spawnerID** as arguments. You will update the parameters of the class to their particular properties using the following code:

```
class ChestModel {
  constructor(x, y, gold, spawnerId) {
    this.id = `${spawnerId}-${uuid.v4()}`;
    this.spawnerId = spawnerId;
    this.x = x;
    this.y = y;
    this.gold = gold;
  }
}
```

For the unique **id** of the chest, concatenate the Spawner ID along with a unique identifier using UUIDs. UUIDs are unique identifiers, that you can rely on to be unique every time you generate one. You could normally use something like **DateTime** to generate the IDs, but there's a possibility that the same ID could be generated if the code is fast enough. Therefore you will use the **UUID** library to ensure the uniqueness of the object ID. To use this library, go to the following link:

Link to the node-uuid library: <https://cdnjs.com/libraries/node-uuid>

On the library page, you will see a list of versions as shown in the image below. Select the minified version and click 'Copy'.



Go back to your code repository, and open up the **index.html** page to edit it. Add a new script with the copied link of the **UUID** library. The updated **index.html** is shown below:



```
<body>
  <div id="phaser-game"></div>
  <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>
  <script src="https://cdn.jsdelivr.net/npm/node-
uuid/1.4.8/uuid.min.js"></script>
  <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>
  <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>
  <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>
  <script src="assets/js/classes/player/Player.js" charset="utf-8"></script>
  <script src="assets/js/classes/Chest.js" charset="utf-8"></script>
  <script src="assets/js/classes/UIButton.js" charset="utf-8"></script>
  <script src="assets/js/classes/Map.js" charset="utf-8"></script>
  <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
  <script src="assets/js/main.js" charset="utf-8"></script>
</body>
```

Remember to save the updated **index.html** page.

Updating the methods to create objects

Now that the **ChestModel** class is defined, you can create a chest object using the **spawnChest** method in the **Spawner.js** file. For this, you will update the arguments passed on to the **ChestModel** instance within the **spawnChest** method. Here, you can pass the arguments **location[0]** and **location[1]** for **x** and **y** coordinates of the **location** respectively. This **location** was obtained by calling the **pickRandomLocation** method which will be implemented next. Remaining arguments for the **spawnChest** method are 10 **gold** pieces for the chest and the **Spawner ID** to create the chest object. Next, add the created chest to the **objectsCreated** array. Passing the chest ID and chest object as parameters, use the **addObject** method to notify the **GameManager** of the chest that has been created. This can be done with the following code:

```
spawnChest() {
  const location = this.pickRandomLocation();
  const chest = new ChestModel(location[0], location[1], randomNumber(10, 20), this
.id);
  this.objectsCreated.push(chest);
  this.addObject(chest.id, chest);
}
```

For the **addObject** method to work, you need to update the **addChest** method in **GameManager.js** file. In the **addChest** method, update the chests that are being stored in the **GameManager**. For this, you need the chest ID and the chest object. The code below shows the updated method:

```
addChest(id, chest) {
  this.chests[id] = chest;
}
```



Next, you will implement the logic for the **pickRandomLocation** method in the **Spawner** class. First, create a variable called **location** by setting it to a randomly selected index value in the **spawnLocations** array. This location needs to be checked whether it is being used by any other objects in the **Spawner**. To do this, create a variable **invalidLocation** which will call the **some** method on the **objectsCreated** array. This will execute a callback on the array until it finds an invalid location, returning a **true** value. Once a **true** value has been returned, the method will stop looping. If not, the method will loop through the whole array and return **false**, indicating that the location is not invalid. Then, if the randomly picked location is invalid, recursively call **pickRandomLocation** method until you get a location that is not currently in use. If the location is valid, return the **location** object. See the following code:

```
pickRandomLocation() {
    const location = this.spawnLocations[Math.floor(Math.random() * this.spawnLocations.length)];
    const invalidLocation = this.objectsCreated.some((obj) => {
        if (obj.x === location[0] && obj.y === location[1]) {
            return true;
        }
        return false;
    });

    if (invalidLocation) return this.pickRandomLocation();
    return location;
}
```

Finally, in the **addChest** method in **GameManager**, add a console log and validate the chest that was created as following:

```
addChest(id, chest) {
    this.chests[id] = chest;
    console.log(chest);
}
```

Now when you refresh the game you should see something logged to the console because the chest models are being created.

Unfortunately, there seems to be a mistake in the code as nothing is logged to the console. The bug was in the **Spawner** class constructor where the **objectType** property is taking an **objectType** as the parameter, where as it should be taking a **spawnerType** instead. This is because a **spawnerType** is passed as the argument for the object type in the **setupSpawners** method in the **GameManager**. Notice the **Spawner** class constructor fixed in the code below:

```
class Spawner {
    constructor(config, spawnLocations, addObject, deleteObject) {
        this.id = config.id;
        this.spawnInterval = config.spawnInterval;
        this.limit = config.limit;
        this.objectType = config.spawnerType;
        this.spawnLocations = spawnLocations;
        this.addObject = addObject;
        this.deleteObject = deleteObject;
    }
}
```



```
this.objectsCreated = [];  
  
this.start();  
}
```

Now, if you refresh the game you can see what is logged to the console as below:



In the upcoming lesson, you will be learning how to remove objects of the game.



The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, you will learn how to remove objects from the **Spawner** class and how to use an Enumerated type (enum).

Removing objects from the Spawner class

Before updating the game scene to use the chest that was spawned by the **Spawner** class, logic for removing an object needs to be implemented. This is done by the **removeObject** method in **Spawner.js** file.

Recall that the **objectsCreated** array holds the objects that have been created in the game. In the **removeObject** method, you will go through the **objectsCreated** array and delete **this** particular chest from the **Spawner** class. Pass the **id** of the chest that you will delete as the argument to **removeObject** method. Then, filter the **objectsCreated** array so that it will return a new array that does not contain the object with the **id** you passed. Next, you will call the **deleteObject** method to notify the **GameManager** that this object was removed. The following code shows you how to do this:

```
removeObject(id) {  
    this.objectsCreated = this.objectsCreated.filter(obj => obj.id !== id);  
    this.deleteObject(id);  
}
```

The call to **deleteObject** method is directed to the **deleteChest** method in **GameManager.js** because you are deleting a chest object. The **deleteChest** method takes the **chestId** as the argument and deletes the chest from the **chests** array. Use the code below:

```
deleteChest(chestId) {  
    delete this.chests[chestId];  
}
```

For consistency, replace the variable name **id** with **chestId** in **addChest** method in the **GameManager**, and remove the previous console log. The code below shows the updates:

```
addChest(chestId, chest) {  
    this.chests[chestId] = chest;  
}
```

Go back to the **Spawner** class and update the **spawnObject** method by removing its console log as well as shown below:

```
spawnObject() {  
    if (this.objectType === 'CHEST') {  
        this.spawnChest();  
    }  
}
```



Creating Enums

Notice that you're checking the **objectType** to be 'CHEST' in the **spawnObject** method. In addition, you're passing 'CHEST' as the **spawnerType** in the **GameManager** as well. So, any time that you have a strict value or an enum of possible values you know ahead of time, it's beneficial to keep track of those values. This way, you can reference these values in multiple locations. Enums are also useful when you need to update the types of objects in your game. Then, you would only have to change one location, which is where you defined the enum.

To create an enum, go to the **GameManager** folder in your code base and create a new file **utils.js** in it. Create the **SpawnerType** enum as following:

```
const SpawnerType = {  
  MONSTER: 'MONSTER',  
  CHEST: 'CHEST',  
};
```

Save **utils.js** and open **index.html** file. Now, add a script directing to **utils.js** file as shown below:

```
<body>  
  <div id="phaser-game"></div>  
  <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>  
  <script src="https://cdnjs.cloudflare.com/ajax/libs/node-  
uuid/1.4.8/uuid.min.js"></script>  
  <script src="assets/js/game_manager/utils.js" charset="utf-8"></script>  
  <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>  
  <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>  
  <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>  
  <script src="assets/js/classes/player/Player.js" charset="utf-8"></script>  
  <script src="assets/js/classes/Chest.js" charset="utf-8"></script>  
  <script src="assets/js/classes/UIButton.js" charset="utf-8"></script>  
  <script src="assets/js/classes/Map.js" charset="utf-8"></script>  
  <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>  
  <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>  
  <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>  
  <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>  
  <script src="assets/js/main.js" charset="utf-8"></script>  
</body>
```

Now, you have an enum defined for referencing in multiple locations. To use the enum, you can go to **spawnObject** method in **Spawner** class and replace **objectType** check for 'CHEST' with **SpawnerType.CHEST**. Check the following code:

```
spawnObject() {  
  if (this.objectType === SpawnerType.CHEST) {  
    this.spawnChest();  
  }  
}
```




Apply the same change in **setupSpawners** method in **GameManager.js** as well. This change is shown in the code below:

```
setupSpawners() {
    // create chest spawners
    Object.keys(this.chestLocations).forEach((key) => {
        const config = {
            spawnInterval: 3000,
            limit: 3,
            spawnerType: SpawnerType.CHEST,
            id: `chest-${key}`,
        };

        const spawner = new Spawner(
            config,
            this.chestLocations[key],
            this.addChest.bind(this),
            this.deleteChest.bind(this),
        );
        this.spawners[spawner.id] = spawner;
    });
}
```

Additional Changes

Earlier, you hardcoded a value of 10 for the amount of **gold** in the chest in **spawnChest** method of **Spawner** class. Instead, change this so that the chest will have a random amount of gold. To do this, implement a function **randomNumber** to generate random numbers in **utils.js** file. The generated number will be in the range of maximum (**max**) and minimum (**min**) values as specified. Use the code below. **The following code has been updated, and differs from the video:**

```
function randomNumber(min, max) {
    return Math.floor(Math.random() * (max - min)) + min;
}
```

Update the **spawnChest** method in the **Spawner** class by calling the **randomNumber** method to generate an amount of **gold** between 10 and 20.

```
spawnChest() {
    const location = this.pickRandomLocation();
    const chest = new ChestModel(location[0], location[1], randomNumber(10, 20), this
.id);
    this.objectsCreated.push(chest);
    this.addObject(chest.id, chest);
}
```

Save and refresh the game.

Additional Notes



`Array.filter()` - The filter method will return a new array that has all of the elements pass the test that is in the provided function. If not items match the provided function, then an empty array will be returned.



In this lesson, you will update the Game Scene to use the new Spawner Class logic implemented in the previous lessons.

Updating the Game Scene

Until now, a separate **createChests** method in **GameScene.js** was used to create chest objects for the game. Since now you have implemented the **Spawner** class including the logic for creating objects, you will need to update the methods to create using the **Spawner** class.

Essentially, you need the **GameManager** to emit an event to the **Game Scene** when a new object is spawned. This can be done by updating the **addChest** method in **GameManager.js** as shown below:

```
addChest(chestId, chest) {  
    this.chests[chestId] = chest;  
    this.scene.events.emit('chestSpawned', chest);  
}
```

An event named **chestSpawned** is emitted along with the **chest** object. This event will be received by the **Game Scene** to act upon. For this, you will update the **createGameManager** method in **GameScene.js** to spawn a chest object upon receiving an event. See the code below:

```
createGameManager() {  
    this.events.on('spawnPlayer', (location) => {  
        this.createPlayer(location);  
        this.addCollisions();  
    });  
  
    this.events.on('chestSpawned', (chest) => {  
        this.spawnChest(chest);  
    });  
  
    this.gameManager = new GameManager(this, this.map.map.objects);  
    this.gameManager.setup();  
}
```

Next, rename the **createChests** method as **createGroups** in **GameScene** class and remove all of the logic in it that was used for creating a chest object. See the code below:

```
createGroups() {  
    // create a chest group  
    this.chests = this.physics.add.group();  
}
```

Update the relevant parameter inside the **create** method in **GameScene** class as well. See the code below:

```
create() {  
    this.createMap();  
}
```



```
this.createAudio();
this.createGroups(); #update with the new function name
this.createInput();

this.createGameManager();
}
```

Spawning chests using locations from Tiled level

The location of the spawned chest is stored as a global variable in the Tiled JSON file to make updating easier and to avoid errors. You can use this to update the location in a chest when it is spawned. First, update the **spawnChest** method in the **GameScene** class to receive the **chestObject**. As **chestObject** contains the location of the object (as a global variable), you do not have to create a separate variable **location** to define the position of spawned chest. Therefore, remove that and replace the values passed for location coordinates with **location[0]** corresponding to **chestObject.x * 2** and **location[1]** corresponding to **chestObject.y * 2**. See the code below:

```
spawnChest(chestObject) {
    let chest = this.chests.getFirstDead();
    if (!chest) {
        chest = new Chest(this, chestObject.x * 2, chestObject.y * 2, 'items', 0);
        // add chest to chests group
        this.chests.add(chest);
    } else {
        chest.setPosition(chestObject.x * 2, chestObject.y * 2);
        chest.makeActive();
    }
}
```

Currently, in the game all game objects are being scaled by 2. In addition to this, the positions that are loaded from the Tiled JSON file are also scaled.

Updating the Chest class

You will now update the creation of a chest to take additional parameters for setting up the unique id of the chest and the amount of gold in it. To do this, first go to **Chest.js** in your **classes** folder and update the constructor to take additional parameters **coins** and **id**. Within the constructor, replace the hardcoded value for the amount of coins with the parameter **coins**. Add a property for the **id** as well. Increase the size of the chest so that it's the same size as the other tiles. See the code below:

```
constructor(scene, x, y, key, frame, coins, id) {
    super(scene, x, y, key, frame);
    this.scene = scene; // the scene this game object will be added to
    this.coins = coins; // the amount of coins this chest contains
    this.id = id;

    // enable physics
    this.scene.physics.world.enable(this);
    // add the player to our existing scene
    this.scene.add.existing(this);
    // scale the chest game object
```



```
    this.setScale(2);  
}
```

The reason for passing **id** as a parameter here is to when you pick up this chest, you're going to take the **id** and pass it back to the **GameManager** to let it know that this chest can be deleted.

Back in **GameScene** class, pass the new parameter values when creating a chest object within **spawnChest** method. See the code below:

```
spawnChest(chestObject) {  
    let chest = this.chests.getFirstDead();  
    if (!chest) {  
        chest = new Chest(this, chestObject.x * 2, chestObject.y * 2, 'items', 0, chestObject.gold, chestObject.id);  
        // add chest to chests group  
        this.chests.add(chest);  
    } else {  
        chest.coins = chestObject.gold; // pass the amount of gold  
        chest.id = chestObject.id;      // pass the chest id  
        chest.setPosition(chestObject.x * 2, chestObject.y * 2);  
        chest.makeActive();  
    }  
}
```

Updating the collection event of a chest

With the above changes, you are no longer spawning a chest manually. Instead, it will all be handled by the **GameManager**. However, you still need to notify the **GameManager** that this chest can be deleted when collected. To do this, you need to emit an event named **pickUpChest** at the end of **collectChest** method in **GameScene** class by using the chest **id**. See the code below:

```
collectChest(player, chest) {  
    // play gold pickup sound  
    this.goldPickupAudio.play();  
    // update our score  
    this.score += chest.coins;  
    // update score in the ui  
    this.events.emit('updateScore', this.score);  
    // make chest game object inactive  
    chest.makeInactive();  
  
    this.events.emit('pickUpChest', chest.id);  
}
```

In the **GameManager.js** you need to listen to that event. You will implement this logic in the **setupEventListener** method. For the **pickUpChest** event created in a game scene, **setupEventListener** will take the chest **id** and update the **spawner** object that this chest can be removed. For this, first check to see if the chest still exists. If it does, remove the chest object by



referencing the **spawner** object using the spawner **id** of the object. Use the code shown below:

```
setupEventListener() {  
    this.scene.events.on('pickUpChest', (chestId) => {  
        // update the spawner  
        if (this.chests[chestId]) {  
            // removing the chest  
            this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);  
        }  
    });  
}
```

Save the code changes and refresh the game to test changes. Try and pick up some chests spawned in the game to see if there are any issues. Chests should be spawned at new locations, and at pickup of a chest it should update the score of the player with the collected coins.





You now have the Spawner class working and populating the game with chests. In this lesson, you will extend the Spawner class and the GameManager to also spawn monsters.

Creating monster spawners

In **GameManager**, update **setupSpawners** method to create monster spawners. Similar to what was done to create chest spawners, create a configuration for each **monsterLocations** with **spawnerType** being **MONSTER**. With the configuration, create a new **Spawner** for the monster. See the code below:

```
setupSpawners() {  
  
    // create chest spawners  
    Object.keys(this.chestLocations).forEach((key) => {  
        const config = {  
            spawnInterval: 3000,  
            limit: 3,  
            spawnerType: SpawnerType.CHEST,  
            id: `chest-${key}`;;  
        };  
  
        const spawner = new Spawner(  
            config,  
            this.chestLocations[key],  
            this.addChest.bind(this),  
            this.deleteChest.bind(this)  
        );  
        this.spawners[spawner.id] = spawner;  
    });  
  
    // create monster spawners  
    Object.keys(this.monsterLocations).forEach((key) => {  
        const config = {  
            spawnInterval: 3000,  
            limit: 3,  
            spawnerType: SpawnerType.MONSTER,  
            id: `monster-${key}`;;  
        };  
  
        const spawner = new Spawner(  
            config,  
            this.monsterLocations[key],  
            this.addMonster.bind(this),  
            this.deleteMonster.bind(this),  
        );  
        this.spawners[spawner.id] = spawner;  
    });  
}
```

An optimization to shorten the above code will be presented later in the lesson.

You can see that new methods **addMonster** and **deleteMonster** are required in **setupSpawners** to create a new monster spawner. See the code below to implement these methods in

**GameManger:**

```
addMonster(monsterId, monster) {
    this.monsters[monsterId] = monster;
    this.scene.events.emit('monsterSpawned', monster);
}

deleteMonster(monsterId) {
    delete this.monsters[monsterId];
}
```

Upon adding a monster object, the **GameManager** will emit an event named **monsterSpawned** along with the **monster** object. In the **GameScene**, you will update the **createGameManager** method to listen to the **monsterSpawned** event and spawn a monster. See the code below:

```
createGameManager() {
    this.events.on('spawnPlayer', (location) => {
        this.createPlayer(location);
        this.addCollisions();
    });

    this.events.on('chestSpawned', (chest) => {
        this.spawnChest(chest);
    });

    // listen to the event to spawn a monster
    this.events.on('monsterSpawned', (monster) => {
        this.spawnMonster(monster);
    });

    this.gameManager = new GameManager(this, this.map.map.objects);
    this.gameManager.setup();
}
```

Upon receiving a **monsterSpawned** event in **GameScene**, you will need to call the **spawnMonster** method. For now, implement the **spawnMonster** method in **GameScene** with only a **console.log** statement to log the spawned monster. See the code below:

```
spawnChest(monster) {
    console.log(monster);
}
```

Optimizing setupSpawners method

In the previous code for **setupSpawners**, you can see that both **config** objects used for chest and monster spawners are similar. Instead of declaring it twice, you can have a single configuration object outside the **foreach** loops. This can be referenced to update any unique values for each spawner object. Since only **id** and **spawnerType** are different for each **Spawner**, update only those



two variables when creating each spawner. The general configuration will have an empty string for the **id** which will be updated at each spawner creation. In addition, the default **spawnerType** will be **CHEST** in the configuration object, which can be assigned to **MONSTER** spawnerType when creating monster spawners. In addition, declare a variable **spawner** and remove the **const** keyword for the spawner creation for both spawner types. See the code below:

```
setupSpawners() {
    const config = {
        spawnInterval: 3000,
        limit: 3,
        spawnerType: SpawnerType.CHEST,
        id: '',
    };

    let spawner; // initialize variable

    // create chest spawners
    Object.keys(this.chestLocations).forEach((key) => {
        config.id = `chest-${key}`; // update the unique id for the chest

        spawner = new Spawner(
            config,
            this.chestLocations[key],
            this.addChest.bind(this),
            this.deleteChest.bind(this)
        );
        this.spawners[spawner.id] = spawner;
    });

    // create monster spawners
    Object.keys(this.monsterLocations).forEach((key) => {
        config.id = `monster-${key}`; // update the unique id for the monster
        config.spawnerType = SpawnerType.MONSTER; // update the spawner type

        spawner = new Spawner(
            config,
            this.monsterLocations[key],
            this.addMonster.bind(this),
            this.deleteMonster.bind(this)
        );
        this.spawners[spawner.id] = spawner;
    });
}
```

Spawning monster objects

Next, you will update the **spawnObject** method in **Spawner.js** file to check for a monster spawner. If the object to be created is of type **MONSTER**, the **spawnMonster** method needs to be called. See the code below:

```
spawnObject() {
    if (this.objectType === SpawnerType.CHEST) {
```



```
        this.spawnChest();
    } else if (this.objectType === SpawnerType.MONSTER) {
        this.spawnMonster();
    }
}
```

You will implement the new method **spawnMonster** in the **Spawner** class with similar logic as used in **spawnChest** method. See the code below:

```
spawnMonster() {
    const location = this.pickRandomLocation();
    const monster = new MonsterModel();
    this.objectsCreated.push(monster);
    this.addObject(monster.id, monster);
}
```

You can see that in the `spawnMonster` method a new class **MonsterModel** is called to create a monster object. In the **gamemanager** folder of your code base, create a new file **MonsterModel.js** with the class constructor as following:

```
class MonsterModel {
    constructor(x, y, gold, spawnerId, frame, health, attack) {
        this.id = `${spawnerId}-${uuid.v4()}`;
        this.spawnerId = spawnerId;
        this.x = x * 2;
        this.y = y * 2;
        this.gold = gold;
        this.frame = frame;
        this.health = health;
        this.maxHealth = health;
        this.attack = attack;
    }
}
```

The monster will have **x** and **y** values to indicate its position. Also, each monster will have an amount of **gold** that will be rewarded to the player when they defeat the monster. The **spawnerId** is used to help generate the unique ID for the monster. You will also need the constructor to have **frame**, **health** and **attack** values. The **frame** will be used to dictate which monster sprite is being used in the game. A monster sprite sheet will be used for this, from which a random frame will be generated. This way, you can have a random monster appear in the game. Each monster will have a **health** value to indicate whether the monster needs to die or not. It will also have a random **attack** value for when it's attacking a player.

Back in the **Spawner** class, you will need to update the call to **MonsterModel** in **spawnMonster** method with relevant arguments. Random numbers can be generated for **gold**, **frame**, **health** and **attack** values. See the code below:

```
spawnMonster() {
    const location = this.pickRandomLocation();
```



```
const monster = new MonsterModel(  
    location[0],  
    location[1],  
    randomNumber(10, 20),  
    this.id,  
    randomNumber(0, 20),  
    randomNumber(3, 5),  
    1,  
);  
this.objectsCreated.push(monster);  
this.addObject(monster.id, monster);  
}
```

You will also need to update the **GameManager** constructor with the object for monsters as below:

```
constructor(scene, mapData) {  
    this.scene = scene;  
    this.mapData = mapData;  
  
    this.spawners = {};  
    this.chests = {};  
    this.monsters = {};  
  
    this.playerLocations = [];  
    this.chestLocations = {};  
    this.monsterLocations = {};  
}
```

Finally, In your **index.html** file, add in a new **script** tag for the model. See the code below:

```
<body>  
    <div id="phaser-game"></div>  
    <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>  
    <script src="https://cdnjs.cloudflare.com/ajax/libs/node-  
uuid/1.4.8/uuid.min.js"></script>  
    <script src="assets/js/game_manager/utils.js" charset="utf-8"></script>  
    <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>  
    <script src="assets/js/game_manager/MonsterModel.js" charset="utf-8"></script>  
>  
    <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>  
    <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>  
    <script src="assets/js/classes/player/Player.js" charset="utf-8"></script>  
    <script src="assets/js/classes/Chest.js" charset="utf-8"></script>  
    <script src="assets/js/classes/UiButton.js" charset="utf-8"></script>  
    <script src="assets/js/classes/Map.js" charset="utf-8"></script>  
    <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>  
    <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>  
    <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>  
    <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>  
    <script src="assets/js/main.js" charset="utf-8"></script>  
</body>
```



Save everything and refresh the game. You should see some monsters being logged on your console. Observe the values populating the **MonsterModel** instances of each monster.

```
MonsterModel {id: "monster-6-3a19ba5b-5d0c-419a-a45b-ab74ac080c35", spawnId: "monster-6", x: 893, y: 109, gold: 22, ...}
MonsterModel {id: "monster-1-4fe516a4-7017-4f7c-8a1a-08e3ef967861", spawnId: "monster-1", x: 365, y: 775, gold: 14, ...}
MonsterModel {id: "monster-2-9aeba536-eed8-4371-9dc8-110c342a0773", spawnId: "monster-2", x: 1113, y: 474, gold: 23, ...}
  attack: 1
  frame: 10
  gold: 23
  health: 3
  id: "monster-2-9aeba536-eed8-4371-9dc8-110c342a0773"
  maxHealth: 3
  spawnId: "monster-2"
  x: 1113
  y: 474
  __proto__: Object
MonsterModel {id: "monster-3-9616fbce-4287-4fb3-a165-4145a02a0e85", spawnId: "monster-3", x: 1836, y: 678, gold: 17, ...}
MonsterModel {id: "monster-4-7af55cc7-9fbc-4330-a8e6-6666cd324259", spawnId: "monster-4", x: 1531, y: 915, gold: 23, ...}
```




The anchor point of an object is the point of its sprite where its location points to. For example, in Tiled, the location of an object refers to the location of its bottom left corner. On the other hand, in Phaser 3, the default anchor point is the middle of the object.

Because of that, when creating a new object from a Tiled map, you need to handle this difference in order to create the object visually in the same location as in the Tiled map.

An easy way to fix that is changing the `parseMapData` method to adjust the locations accordingly, as below. Notice that, instead of using `(obj.x, obj.y)` we use `(obj.x + (obj.width / 2), obj.y - (obj.height / 2))`.

```
parseMapData() {
  this.mapData.forEach((layer) => {
    if (layer.name === 'player_locations') {
      layer.objects.forEach((obj) => {
        this.playerLocations.push([obj.x + (obj.width / 2), obj.y - (obj.height / 2)
      ]]);
    }
    } else if (layer.name === 'chest_locations') {
      layer.objects.forEach((obj) => {
        if (this.chestLocations[obj.properties.spawner]) {
          this.chestLocations[obj.properties.spawner].push([obj.x + (obj.width / 2)
, obj.y - (obj.height / 2)]);
        } else {
          this.chestLocations[obj.properties.spawner] = [[obj.x + (obj.width / 2),
obj.y - (obj.height / 2)]];
        }
      });
    } else if (layer.name === 'monster_locations') {
      layer.objects.forEach((obj) => {
        if (this.monsterLocations[obj.properties.spawner]) {
          this.monsterLocations[obj.properties.spawner].push([obj.x + (obj.width /
2), obj.y - (obj.height / 2)]);
        } else {
          this.monsterLocations[obj.properties.spawner] = [[obj.x + (obj.width / 2)
, obj.y - (obj.height / 2)]];
        }
      });
    }
  });
}
```



The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

Now that you have the **GameManager** creating the monsters, you need to actually create the **Monster** game object in the Phaser game. In this lesson, you will create the **Monster** class and update the **GameScene** to use the new class.

Implementing the Monster class

In your **classes** folder, create a new class file **Monster.js**. Implement the constructor for the **Monster** class by extending the **Phaser.Physics.Arcade.Image** class. The constructor will take the parameters **scene**, **x** and **y** value for location, **key**, and **frame**. You will also provide the **id**, **health**, and **maxHealth** values of the monster as parameters. Set the references to the provided parameters. See the code below:

```
class Monster extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame, id, health, maxHealth) {
    super(scene, x, y, key, frame);
    this.scene = scene;
    this.id = id;
    this.health = health;
    this.maxHealth = maxHealth;
  }
}
```

In addition, you will need to enable the physics on the monster game object and scale up the object so that it is the same size as the other tiles. You also need to make sure that the object collides with the world bounds as you don't want the monsters to exit the game. And then you need to add the existing monster to the game. See the updated constructor below:

```
class Monster extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame, id, health, maxHealth) {
    super(scene, x, y, key, frame);
    this.scene = scene;
    this.id = id;
    this.health = health;
    this.maxHealth = maxHealth;

    // enable physics
    this.scene.physics.world.enable(this);
    // set immovable if another object collides with our monster
    this.setImmovable(false);
    // scale our monster
    this.setScale(2);
    // collide with world bounds
    this.setCollideWorldBounds(true);
    // add the monster to our existing scene
    this.scene.add.existing(this);
  }
}
```



To control the status of the monster objects, you will need to implement the **makeActive** and **makeInactive** methods in the **Monster** class. See the code below:

```
makeActive() {
    this.setActive(true);
    this.setVisible(true);
    this.body.checkCollision.none = false;
}

makeInactive() {
    this.setActive(false);
    this.setVisible(false);
    this.body.checkCollision.none = true;
}
```

When a monster is created in the game, it should be made active so that collisions with a player can be implemented. Once a monster dies, you will want to make it inactive and turn off collisions. The above methods can be used for that purpose. That way, you can create a pool of monsters and reuse those game objects as the players are playing the game.

Add the new **Monster** class to your **index.html** file as a new script. See the code below:

```
<body>
    <div id="phaser-game"></div>
    <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/node-
uuid/1.4.8/uuid.min.js"></script>
    <script src="assets/js/game_manager/utils.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/MonsterModel.js" charset="utf-8"></script>
>
    <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>
    <script src="assets/js/classes/player/Player.js" charset="utf-8"></script>
    <script src="assets/js/classes/Monster.js" charset="utf-8"></script>
    <script src="assets/js/classes/Chest.js" charset="utf-8"></script>
    <script src="assets/js/classes/UiButton.js" charset="utf-8"></script>
    <script src="assets/js/classes/Map.js" charset="utf-8"></script>
    <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
    <script src="assets/js/main.js" charset="utf-8"></script>
</body>
```

Now that you have the **Monster** class in place, you should be able to create monsters in the game. In the **GameScene.js** file, first add the logic to create a monster group in the **createGroups** method as shown below:

```
createGroups() {
    // create a chest group
```



```
this.chests = this.physics.add.group();
// create a monster group
this.monsters = this.physics.add.group();
}
```

Next, update the **spawnMonster** method to create **Monster** objects. Remove the previous **console.log** that was used to log the monsters. If a monster object does not exist, create a new monster by passing relevant parameters. If the monster already exists, you don't need to create a new monster. Simply update the existing object's values and make the object active. See the code below:

```
spawnMonster(monsterObject) {
    let monster = this.monsters.getFirstDead();
    if (!monster) {
        const monster = new Monster(
            this,
            monsterObject.x * 2,
            monsterObject.y * 2,
            'monsters',
            monsterObject.frame,
            monsterObject.id,
            monsterObject.health,
            monsterObject.maxHealth,
        );
        // add monster to monsters group
        this.monsters.add(monster);
    } else {
        monster.id = monsterObject.id;
        monster.health = monsterObject.health;
        monster.maxHealth = monsterObject.maxHealth;
        monster.setTexture('monsters', monsterObject.frame);
        monster.setPosition(monsterObject.x * 2, monsterObject.y * 2);
        monster.makeActive();
    }
}
```

At this point, there seems to be an issue in the **spawnMonsters** method. The monster object declared with the **let** keyword and the monster object created with the **const** keyword are going to be different variables and will have different scopes. This is a redeclaration of the same object. What was meant to be done was to reuse the same object. Therefore, only declare the object with the **let** keyword and remove the **const** keyword when creating the object by passing parameters. See the code below;

The following code has been updated, and differs from the video. After adding the monster to the group, we need to set its `collideWorldBounds` property again:

```
spawnMonster(monsterObject) {
    let monster = this.monsters.getFirstDead(); // declare the monster variable only here
    if (!monster) {
        monster = new Monster( // the const keyword is removed here
```



```
        this,
        monsterObject.x * 2,
        monsterObject.y * 2,
        'monsters',
        monsterObject.frame,
        monsterObject.id,
        monsterObject.health,
        monsterObject.maxHealth,
    );
    // add monster to monsters group
    this.monsters.add(monster);
    monster.setCollideWorldBounds(true);
} else {
    monster.id = monsterObject.id;
    monster.health = monsterObject.health;
    monster.maxHealth = monsterObject.maxHealth;
    monster.setTexture('monsters', monsterObject.frame);
    monster.setPosition(monsterObject.x * 2, monsterObject.y * 2);
    monster.makeActive();
}
}
```

Do the same change in **spawnChest** method as well and remove redeclaration of the chest object.

You can see that in the above **spawnMonsters** method, a new method **setTexture** is called. This is used to update the texture of an existing game object. This method will take two arguments; the asset **monsters** and the frame if it is a Sprite sheet. The default frame is the first frame. if you dont specify the frame when you create a Sprite or image game object from a Sprite sheet asset, Phaser will automatically use the first frame of the sprite sheet.

Finally, you need to load the monster's asset. In your **BootScene.js** file update the **loadSpriteSheets** method to load the monster assets from the relevant Sprite sheet. See the code below:

```
loadSpriteSheets() {
    this.load.spritesheet('items', 'assets/images/items.png', { frameWidth: 32, frameHeight: 32 });
    this.load.spritesheet('characters', 'assets/images/characters.png', { frameWidth: 32, frameHeight: 32 });
    this.load.spritesheet('monsters', 'assets/images/monsters.png', { frameWidth: 32, frameHeight: 32 });
}
```

Now you should be able to save and refresh the game to test the changes. You will see that monsters are being spawned at different locations.





Currently, the monsters aren't doing anything and you are not checking for any type of collisions. In this lesson, you will update the monster group to collide with map block layers.

Colliding with a blocked layer

First, you need to add a collider between the monster and the blocked layer. That way the monsters won't be moving over the blocked tiles. In **GameScene.js**, update the **addCollisions** method by adding the logic for checking collision between the monster group and blockedLayer. See the code below:

```
addCollisions() {
    // check for collisions between the player and the tiled blocked layer
    this.physics.add.collider(this.player, this.map.blockedLayer);
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, this);
;
    // check for collisions between the monster group and the tiled blocked layer
    this.physics.add.collider(this.monsters, this.map.blockedLayer);
}
```

By putting the monsters in a group, Phaser will automatically check for collisions between all of the child game objects in that group and the other matching game object/group as well.

Colliding with the player

Now you will also add an overlap in **addCollisions** method to destroy the monster if the player goes over it. See the code below.

```
addCollisions() {
    // check for collisions between the player and the tiled blocked layer
    this.physics.add.collider(this.player, this.map.blockedLayer);
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, this);
;
    // check for collisions between the monster group and the tiled blocked layer
    this.physics.add.collider(this.monsters, this.map.blockedLayer);
    // check for overlaps between the player and monster game objects
    this.physics.add.overlap(this.player, this.monsters, this.enemyOverlap, null, this);
};
}
```

You can see that a new method **enemyOverlap** is called when checking for collisions between the player and monsters. Create this new method in **GameScene** class. Two arguments are received by the **enemyOverlap** method; the **player** and **enemy** game objects. First, make the enemy inactive. Then emit an event named **destroyEnemy** along with the **id** of the enemy object. See the code below:

```
enemyOverlap(player, enemy) {
    enemy.makeInactive();
    this.events.emit('destroyEnemy', enemy.id);
}
```

```
}
```

In the **GameManager**, update the **setupEventListener** method to listen to the **destroyEnemy** event. Upon receiving the event, remove the monster object from the group. See the code below:

```
setupEventListener() {  
    this.scene.events.on('pickUpChest', (chestId) => {  
        // update the spawner  
        if (this.chests[chestId]) {  
            this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);  
        }  
    });  
  
    this.scene.events.on('destroyEnemy', (monsterId) => {  
        // update the spawner  
        if (this.monsters[monsterId]) {  
            this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);  
        }  
    });  
}
```

Now, save and refresh the game. You should be able to test the monsters spawning. When you collide with a monster, it should be removed from the game.



Additional notes



- To create a collider:

```
this.physics.add.collider();
```

- To create an overlap:

```
this.physics.add.overlap();
```



Now that you have monsters in your game, you'll work on adding logic to allow the player to attack a monster instead of just checking for an overlap between the player and the monster.

Creating a new container class

The first thing to do is to create a new container class. The reason for that is, you are going to switch your player from an arcade image to a container. A container is preferred because now your player objects will be made of two separate sprites. The first will be the player itself, and the second will be the weapon. Instead of having to keep track of these separately, you can easily put them in the container and have that be your player. That way, you can reference both of those game objects relatively easily.

In your **classes** folder, make a new folder called **player**. Then, drag and drop your previously implemented **Player** class into the new **player** folder. Also create a new file **PlayerContainer.js** in the **player** folder. Copy all the logic from **Player.js** and paste into **PlayerContainer.js**. Update the class name in the **PlayerContainer.js** file to **PlayerContainer** class (previously, the class name in **PlayerContainer.js** was **Player** as you copied the logic from **Player** class), and change the extending class into **Phaser.GameObjects.Container** from **Phaser.Physics.Arcade.Image**.

Now you have two classes **Player** and **PlayerContainer** that you can use to control your player. Any logic that's related to the player game object itself, you will implement it in the **Player** class. And any logic that has to do with the container itself (such as moving the player) you will be keeping that in the high level which is the **PlayerContainer** class.

Update the Player class

In the **Player** class, remove the **update** method entirely. The reason for this is you're going to have the container be the one moving around, and the **update** logic will be implemented in **PlayerContainer** class later. In the constructor of the **Player** class, you will remove and change some unnecessary logic as well. First, as you don't need the camera to follow the player game object, remove the code lines for that logic. The world bounds don't need to be checked either because that will be implemented in the **PlayerContainer**. In addition, remove the parameter set for the **velocity** as well. Change the argument passed to **setImmovable** method to true. The updated code in **Player** class will look like following:

```
class Player extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);
    this.scene = scene; // the scene this container will be added to

    // enable physics
    this.scene.physics.world.enable(this);
    // set immovable if another object collides with our player
    this.setImmovable(true);
    // scale our player
    this.setScale(2);
    // add the player to our existing scene
    this.scene.add.existing(this);
  }
}
```

Update the PlayerContainer class



In the constructor of the **PlayerContainer** class, remove the call to **setImmovable** method as well as the logic for scaling since you do not need to scale the container. However, you still need the container to collide with the world bounds so change that logic to call the **setColliderWorldBounds** method on the **body**. In addition, you need to set a size on the container. The reason for that is, by default a container won't have any size until objects are added to it. And because you are creating the object first and then adding the game objects to the container, the container won't have a size. Therefore, set the size to 64x64 since that's the size of the player game object. See the code below for the updates to the constructor in **PlayerContainer** class:

```
class PlayerContainer extends Phaser.GameObjects.Container {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y);
    this.scene = scene; // the scene this container will be added to
    this.velocity = 160; // the velocity when moving our player

    // set a size on the container
    this.setSize(64, 64);
    // enable physics
    this.scene.physics.world.enable(this);
    // collide with world bounds
    this.body.setCollideWorldBounds(true);
    // add the player container to our existing scene
    this.scene.add.existing(this);
    // have the camera follow the player
    this.scene.cameras.main.startFollow(this);
  }
}
```

(Please note that the **PlayerContainer** class also has the **update** method implementation which was copied from the **Player** class, but is not shown in the above code snippet for clarity)

Next, you will need to create the player game object and add it to this container. For this, create the player first by calling a **Player** class instance with proper arguments within the **PlayerContainer** constructor. The position of the player will be set to (0,0) because the position of any game object that is placed inside a container is relative to that particular container. Therefore, by passing (0,0), this game object should be right where the container is. Then, add the created player object to the container by calling the **add** method and passing that game object as an argument. Another change to be made in the **PlayerContainer** is you need to change the **super** call in the constructor. Because it's a container, you don't actually need to pass in a key or a frame to the super method. Therefore, remove those and only pass **scene**, **x**, and **y** to the super call. See the updated code below:

```
class PlayerContainer extends Phaser.GameObjects.Container {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y);
    this.scene = scene; // the scene this container will be added to
    this.velocity = 160; // the velocity when moving our player

    // set a size on the container
    this.setSize(64, 64);
    // enable physics
    this.scene.physics.world.enable(this);
    // collide with world bounds
```



```
this.body.setCollideWorldBounds(true);
// add the player container to our existing scene
this.scene.add.existing(this);
// have the camera follow the player
this.scene.cameras.main.startFollow(this);

// create the player
this.player = new Player(this.scene, 0, 0, key, frame);
this.add(this.player);
}
}
```

Update the GameScene class

In the **GameScene** class, change the creation of player object in createPlayer method by calling **PlayerContainer** instead of a **Player** instance. See the code below:

```
createPlayer(location) {
    this.player = new PlayerContainer(this, location[0] * 2, location[1] * 2, 'characters', 0);
}
```

Finally, go to the **index.html** page and add in the new classes as new script tags. See the code below:

```
<body>
    <div id="phaser-game"></div>
    <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>
    <script src="https://cdn.jsdelivr.net/npm/node-
uuid/1.4.8/uuid.min.js"></script>
    <script src="assets/js/game_manager/utils.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/MonsterModel.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>
    <script src="assets/js/classes/player/Player.js" charset="utf-8"></script>
    <script src="assets/js/classes/player/PlayerContainer.js" charset="utf-8"></script>
    <script src="assets/js/classes/Monster.js" charset="utf-8"></script>
    <script src="assets/js/classes/Chest.js" charset="utf-8"></script>
    <script src="assets/js/classes/UIButton.js" charset="utf-8"></script>
    <script src="assets/js/classes/Map.js" charset="utf-8"></script>
    <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
    <script src="assets/js/main.js" charset="utf-8"></script>
</body>
```




Save all changes and refresh the game. You will see that players are still being created in the game and you're still able to move them around. They will collide with the blocked layer.





In this lesson, you will create a player weapon game object, and update that game object to rotate as the player moves around the game.

Creating a player weapon game object

In the previous lesson you refactored the **Player** class to be two separate classes. Now you will update the **PlayerContainer** to have a weapon that the player will be able to swing at the monsters. You will need to track and update a few different things to do this:

1. Track the direction the player was last headed towards. That way, when the player swings their weapon it will be in the direction the player was facing last. For example, if the last direction was 'Left' you will have the sword swing in the left direction.
2. Update the player game object so that the player sprite is flipped towards the direction of movement. For an example, if the player is moving to the right of the game scene, he will be facing the right.
3. Track when the player is attacking a monster.
4. Track if the weapon actually hit an enemy game object. The reason for doing this is because the sword is going to hit the enemy game object multiple times when it's swinging. But you only want to check for collision on the first hit. Then, you want to wait until after the player has done attacking before you check for collisions again. Otherwise, for every frame that the sword is touching that enemy's collider, it will keep deducting health from our enemy.

To make the above changes, you will be editing the **PlayerContainer.js** file. First, add some new properties to the **PlayerContainer** constructor. Set default current direction as **RIGHT** as denoted by **currentDirection**, and have the player not attacking by default using the **playerAttacking** property. Next, set the property **flipX** to true to indicate the player sprite to be flipped in the direction of movement, and **swordHit** to check if the weapon hit the enemy game object. See the code below:

```
constructor(scene, x, y, key, frame) {
    super(scene, x, y);
    this.scene = scene; // the scene this container will be added to
    this.velocity = 160; // the velocity when moving our player
    this.currentDirection = 'RIGHT';
    this.playerAttacking = false;
    this.flipX = true;
    this.swordHit = false;

    // set a size on the container
    this.setSize(64, 64);
    // enable physics
    this.scene.physics.world.enable(this);
    // collide with world bounds
    this.body.setCollideWorldBounds(true);
    // add the player container to our existing scene
    this.scene.add.existing(this);
    // have the camera follow the player
    this.scene.cameras.main.startFollow(this);

    // create the player
    this.player = new Player(this.scene, 0, 0, key, frame);
    this.add(this.player);
}
```



To make future references easier, you can create an enum variable for the directions you will be using to control the movement. Add it to the top of the **PlayerContainer.js** file (before the **PlayerContainer** constructor). See the code below:

```
const Direction = {  
  RIGHT: 'RIGHT',  
  LEFT: 'LEFT',  
  UP: 'UP',  
  DOWN: 'DOWN',  
};
```

Now, update the **currentDirection** variable in the constructor to use the new **Direction** object. See the code line below:

```
this.currentDirection = Direction.RIGHT;
```

Next, you will create the weapon game object. For this, you can use the values 40 and 0 as **x** and **y** positions, 'items' as the **asset**, and 4 as the **frame** number. Add the created weapon object to the scene. Also, scale up the weapon by a factor of 1.5 enable the physics of the weapon game object. Next, add the weapon to the container by calling the **add** method. In addition, set the **alpha** value of the weapon to zero. The reason for doing this is because you only want the weapon to be displayed when the player is swinging it. See the updates to the constructor code below:

```
constructor(scene, x, y, key, frame) {  
  super(scene, x, y);  
  this.scene = scene; // the scene this container will be added to  
  this.velocity = 160; // the velocity when moving our player  
  this.currentDirection = Direction.RIGHT;  
  this.playerAttacking = false;  
  this.flipX = true;  
  this.swordHit = false;  
  
  // set a size on the container  
  this.setSize(64, 64);  
  // enable physics  
  this.scene.physics.world.enable(this);  
  // collide with world bounds  
  this.body.setCollideWorldBounds(true);  
  // add the player container to our existing scene  
  this.scene.add.existing(this);  
  // have the camera follow the player  
  this.scene.cameras.main.startFollow(this);  
  
  // create the player  
  this.player = new Player(this.scene, 0, 0, key, frame);  
  this.add(this.player);  
  
  // create the weapon game object  
  this.weapon = this.scene.add.image(40, 0, 'items', 4);  
  this.scene.add.existing(this.weapon);  
  this.weapon.setScale(1.5);
```



```
this.scene.physics.world.enable(this.weapon);  
this.add(this.weapon);  
this.weapon.alpha = 0;  
}
```

To test the weapon movement, set **alpha** to 1 temporarily. Now, save and refresh the game. You should be able to see the player holding a weapon.



Updating the movement of the weapon game object

The next thing to do is change the direction of the weapon sprite as the player moves towards a new direction. In the **update** method in **PlayerContainer** class, update the direction of our player. Remember to set up correct directions according to the cursor direction. See the code below:

```
update(cursors) {  
    this.body.setVelocity(0);  
  
    if (cursors.left.isDown) {  
        this.body.setVelocityX(-this.velocity);  
        this.currentDirection = Direction.LEFT;  
    } else if (cursors.right.isDown) {  
        this.body.setVelocityX(this.velocity);  
        this.currentDirection = Direction.RIGHT;  
    }  
  
    if (cursors.up.isDown) {  
        this.body.setVelocityY(-this.velocity);  
    }  
}
```



```
        this.currentDirection = Direction.UP;
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
    }
}
```

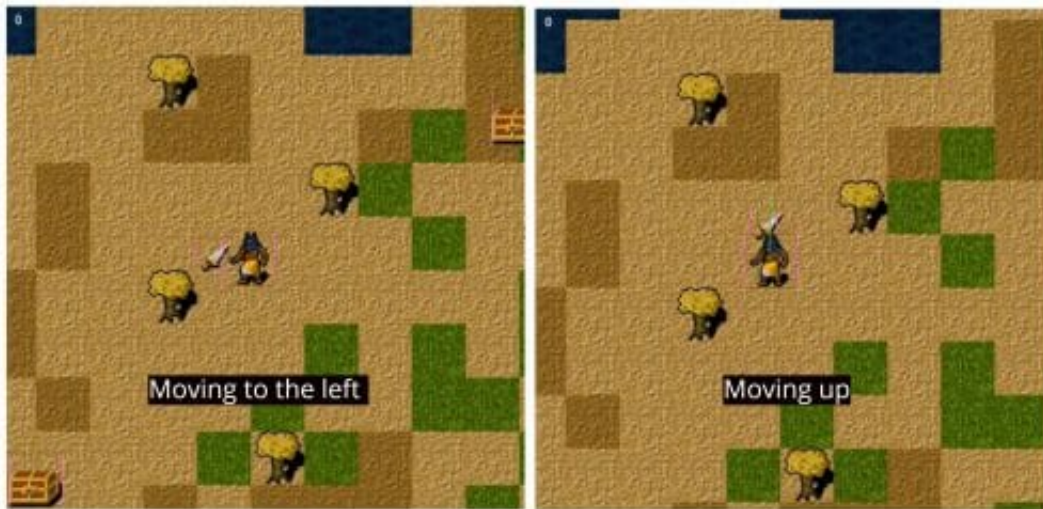
Next, update the position of the weapon in the **update** method itself. Do this by calling **setPosition** method on the weapon object. Notice the different positions set for each direction in the updated code below:

```
update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }
}
```

Save and refresh the game. You should see that as the player is moving, the weapon also moves with the player and faces towards the appropriate direction.



However, you can see that the weapon is not rotating in the correct direction as the player moves in different directions. You can fix this by checking if the player is currently attacking. If they are attacking, the sword will be slowly rotated to look like it's slashing the enemy. If the player is not attacking, the weapon should be angled in one particular direction. To implement this logic, add a new if-else block in the **update** method. See the code below:

```
update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }

    if (this.playerAttacking) {
    } else {
        if (this.currentDirection === Direction.DOWN) {
            this.weapon.setAngle(-270);
        } else if (this.currentDirection === Direction.UP) {
            this.weapon.setAngle(-90);
        } else {
            this.weapon.setAngle(0);
        }
    }
}
```




```
}  
  }  
}
```

According to each direction, use the **setAngle** method to define the angle the weapon is at. The **setAngle** method expects only one argument which is the number of degrees that you want to rotate the weapon game object.

Save changes and test the game. You will see that as the player rotates in different directions, the weapon is now in the proper position.





In this lesson, you will animate the weapon for when the player is attacking, and update the rotation of the weapon when player changes direction. You will also update the attacking logic of the player.

* All changes are made in the **update** method of **PlayerContainer** class if not otherwise stated.

Updating the direction of the weapon

In the previous lesson, you changed the position and the direction of the weapon as the player moved. However, you might have noticed that the weapon was not flipped towards the correct direction when the player was facing left. To flip the weapon game object when the player is facing to the left, you will add some logic to the **if-else** block at the bottom of the **update** method in **PlayerContainer**. See the code below (lines 35-39):

```
update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }

    if (this.playerAttacking) {

    } else {
        if (this.currentDirection === Direction.DOWN) {
            this.weapon.setAngle(-270);
        } else if (this.currentDirection === Direction.UP) {
            this.weapon.setAngle(-90);
        } else {
            this.weapon.setAngle(0);
        }

        // flip the weapon when player is facing left
        this.weapon.flipX = false;
        if (this.currentDirection === Direction.LEFT) {
            this.weapon.flipX = true;
        }
    }
}
```

Have the weapon not flipped by default, but if the player direction is LEFT, flip the weapon by setting **flipX** to true. The **flipX** property tells the Phaser to flip that game object on the x-axis. There is also a **flipY** property where you can flip it on the y-axis, which you will not be using here.

Save and refresh the game to see the weapon rotated in the right direction.



Updating the player to attack

Now, you will update the logic of the player to allow them to attack. First, you need to check to see if the spacebar key has been pressed down. This is to ensure the player is currently not attacking before allowing them to attack again. Add a new **if-else** block for this logic in the **update** method as shown below (lines 24-33):

```
update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }

    // allow the player to attack
    if (Phaser.Input.Keyboard.JustDown(cursors.space) && !this.playerAttacking) {
```



```
this.weapon.alpha = 1;
this.playerAttacking = true;
this.scene.time.delayedCall(150, () => {
    this.weapon.alpha = 0;
    this.playerAttacking = false;
    this.swordHit = false;
}, [], this);
}

if (this.playerAttacking) {

} else {
    if (this.currentDirection === Direction.DOWN) {
        this.weapon.setAngle(-270);
    } else if (this.currentDirection === Direction.UP) {
        this.weapon.setAngle(-90);
    } else {
        this.weapon.setAngle(0);
    }

    this.weapon.flipX = false;
    if (this.currentDirection === Direction.LEFT) {
        this.weapon.flipX = true;
    }
}
}
```

If the spacebar is pressed and if the player is not attacking currently, first set **alpha** to 1 as you want to make the weapon visible. Next, set **playerAttacking** to true as the player is now in attacking state. You need to reset the state of the **weapon** after a small amount of time has passed. To do this, use the Phaser's **time.delayedCall** method, which will run a callback function after a set amount of time has passed. Within the callback function, reset the **alpha** on the **weapon**, and set the state of player as not attacking. Reset the **swordHit** to false, because once attacking is done and the animation is complete, you want the player to be able to attack again.

In the above **if-else** block, you used the **JustDown** method from the **Phaser.Input.Keyboard** class. The **JustDown** method allows you to specify a particular key as the input, and it will detect if that key was just pressed down one time. If the player is holding the particular button down, it will only detect the keypress when it was pressed down the first time. Therefore, the player won't keep attacking repeatedly.

Save and refresh the game. If you press the spacebar, you will see that the weapon appears and disappears.

Animating the weapon

Lastly, you will animate the weapon when the player is actually attacking. To do this, complete the **if** statement that was left empty of logic when player is attacking. See the code below (lines 35-40):

```
update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
```



```
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }

    if (Phaser.Input.Keyboard.JustDown(cursors.space) && !this.playerAttacking) {
        this.weapon.alpha = 1;
        this.playerAttacking = true;
        this.scene.time.delayedCall(150, () => {
            this.weapon.alpha = 0;
            this.playerAttacking = false;
            this.swordHit = false;
        }, [], this);
    }

    if (this.playerAttacking) {
        // animating the weapon
        if (this.weapon.flipX) {
            this.weapon.angle -= 10;
        } else {
            this.weapon.angle += 10;
        }
    } else {
        if (this.currentDirection === Direction.DOWN) {
            this.weapon.setAngle(-270);
        } else if (this.currentDirection === Direction.UP) {
            this.weapon.setAngle(-90);
        } else {
            this.weapon.setAngle(0);
        }

        this.weapon.flipX = false;
        if (this.currentDirection === Direction.LEFT) {
            this.weapon.flipX = true;
        }
    }
}
```

First, you will check if the weapon's **flipX** property is set to true. If it is flipped, update the **angle** on



the weapon to deduct 10 degrees so that it rotates counter clockwise. Otherwise, you want to rotate the weapon clockwise by adding 10 degrees. Here you are increasing the **angle** value directly as if calling the **setAngle** method.

Save and refresh the game. If you press the spacebar you can now see that the weapon is rotating in the proper direction.

Finally, go back to the place where you first created the weapon in the constructor of the **PlayerContainer** class, and change the **alpha** value to zero so that the weapon is not visible by default.

```
this.weapon.alpha = 0;
```

Save and refresh the game. Now, you can see that the weapon is not visible by default, but will appear when you attack an enemy.





With the updates in the previous lesson, you now have a working weapon in the game. In this lesson, you will add the logic to allow a player to attack an enemy, and update monster game objects to become inactive when they lose all of their health.

Attacking monsters

Earlier, you had the enemy disappear when the player overlaps with them. Instead, you will update the collision to destroy the monster when the player's weapon is overlapping with the monster, and if the player is attacking. To do this, update the overlap between the player and monster objects in the **addCollisions** method in the **GameScene** class. Replace the player game object with the player's weapon game object so that the overlap is checked between the weapon and the monster instead of the player and the monster. See the code below:

```
addCollisions() {
    // check for collisions between the player and the tiled blocked layer
    this.physics.add.collider(this.player, this.map.blockedLayer);
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, this)
;
    // check for collisions between the monster group and the tiled blocked layer
    this.physics.add.collider(this.monsters, this.map.blockedLayer);
    // check for overlaps between the player's weapon and monster game objects
    this.physics.add.overlap(this.player.weapon, this.monsters, this.enemyOverlap, null, this);
}
```

Next, in the **enemyOverlap** method in **GameScene** class, add an **if** statement to check if the player is currently attacking and that they have not already hit the monster with the weapon. Use **playerAttacking** and **swordHit** properties of the player to do this check. If the above conditions are met, first set the **swordHit** property to **true** to indicate that the player has hit the monster with the weapon. Then, make the enemy inactive upon attacking by calling the **makeInactive** method and emit an event '**destroyEnemy**'. See the code below:

```
enemyOverlap(player, enemy) {
    if (this.player.playerAttacking && !this.player.swordHit) {
        this.player.swordHit = true;
        enemy.makeInactive()
        this.events.emit('destroyEnemy', enemy.id);
    }
}
```

Save and refresh the game. Wait for a monster to spawn and test the new collider logic. When the player overlaps with a monster, nothing should happen. But if you press the Space key and the weapon attacks, you will see that the monster disappears.

Updating player movement

Currently, the **flipX** property of the weapon is updated so that it is flipped in the correct direction. But you can see that the player is still facing one direction no matter what direction they are moving towards. To change this, you will change the **update** method in **PlayerContainer** class. When checking if the Left arrow key is pressed down, you will not flip the player as he is already facing left



by default. Otherwise, if the Right key is pressed down, set **flipX** value to true so that the player is turned to the right. See the code below (line 8 and line 13):

```
update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
        this.player.flipX = false;    // do not flip the player
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
        this.player.flipX = true;    // flip the player
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }

    if (Phaser.Input.Keyboard.JustDown(cursors.space) && !this.playerAttacking) {
        this.weapon.alpha = 1;
        this.playerAttacking = true;
        this.scene.time.delayedCall(150, () => {
            this.weapon.alpha = 0;
            this.playerAttacking = false;
            this.swordHit = false;
        }, [], this);
    }

    if (this.playerAttacking) {
        if (this.weapon.flipX) {
            this.weapon.angle -= 10;
        } else {
            this.weapon.angle += 10;
        }
    } else {
        if (this.currentDirection === Direction.DOWN) {
            this.weapon.setAngle(-270);
        } else if (this.currentDirection === Direction.UP) {
            this.weapon.setAngle(-90);
        } else {
            this.weapon.setAngle(0);
        }
    }

    this.weapon.flipX = false;
    if (this.currentDirection === Direction.LEFT) {
```



```

        this.weapon.flipX = true;
    }
}

```

Save and refresh the game. Press Left and Right arrow keys to test if the player is facing the correct way.



Updating the health of the monster

Once you attack a monster, you don't want them to die instantly. Instead, you first want to reduce the health of the monster from the amount of health they currently have. Once they lose all of their health, then you actually want to destroy them.

To do this, first update the **enemyOverlap** method in **GameScene** class. Remove the **makeInactive** call called upon the **enemy** object. Then update the event that **enemyOverlap** method is emitting to be named '**monsterAttacked**'. See the code below:

```

enemyOverlap(weapon, enemy) {
    if (this.player.playerAttacking && !this.player.swordHit) {
        this.player.swordHit = true;
        this.events.emit('monsterAttacked', enemy.id, this.player.id);
    }
}

```

Next, in the **GameManager** class, replace '**destroyEnemy**' event name in **setEventListener** with '**monsterAttacked**' event. Then, update the **if** statement by first subtracting the health from the monster model. For this, you will call a new method **loseHealth** on the monster object which will be implemented later. Next, check to see if the monster is dead. If it is, its **health** will be less than or equal to zero and you want to remove that monster. After removing the monster object, add an event '**monsterRemoved**' to be emitted along with the **monsterId**. This is done because in the **GameScene** you removed the logic for making the enemy inactive, and you need to add that logic back so that you don't keep creating new monster game objects over and over again. See the code below:



```
setupEventListener() {
  this.scene.events.on('pickUpChest', (chestId, playerId) => {
    // update the spawner
    if (this.chests[chestId]) {
      // update the spawner
      this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
    }
  });

  this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
    // update the spawner
    if (this.monsters[monsterId]) {
      // subtract health monster model
      this.monsters[monsterId].loseHealth();

      // check the monsters health, and if dead remove that object
      if (this.monsters[monsterId].health <= 0) {
        // removing the monster
        this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
        this.scene.events.emit('monsterRemoved', monsterId);
      }
    }
  });
}
```

You will then implement the new **loseHealth** method in the **MonsterModel** class. Add a console log to log the current health of the monster first. Then, have an attack deduct 1 health point from the monster for the time being. See the code below:

```
loseHealth() {
  console.log(this.health)
  this.health -= 1;
}
```

Now, you need to listen to the new event emitted by the **GameManager**. This will be done in the **createGameManager** method of the **GameScene** class. Upon receiving the event **'monsterRemoved'** and the **monsterID**, loop through the **monsters** group and check if any monster game object ID matches the one you just provided. If it matches, make that game object inactive. See the code below:

```
createGameManager() {
  this.events.on('spawnPlayer', (playerObject) => {
    this.createPlayer(playerObject);
    this.addCollisions();
  });

  this.events.on('chestSpawned', (chest) => {
    this.spawnChest(chest);
  });

  this.events.on('monsterSpawned', (monster) => {
```



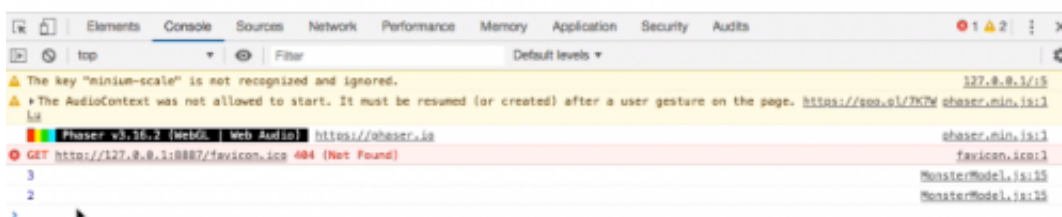
```
        this.spawnMonster(monster);
    });

    // make monster inactive on event monsterRemoved
    this.events.on('monsterRemoved', (monsterId) => {
        this.monsters.getChildren().forEach((monster) => {
            if (monster.id === monsterId) {
                monster.makeInactive();
            }
        });
    });

    this.gameManager = new GameManager(this, this.map.map.objects);
    this.gameManager.setup();
}
```

The **getChildren** method called above on the **monsters** group will return an array of child game objects. You can use **forEach** to loop through them.

Save and refresh the game. Wait for a monster to spawn and check to see if the logic is working properly. If you attack the monster, you should see the health of the monster being reduced logged to the console. When it reaches zero, the monster should become inactive and spawned somewhere else.







In the previous lesson, you implemented the logic to allow a player to attack a monster. In this lesson, you will add a health bar for each monster and update it to change dynamically as the monster loses health.

Adding the health bar graphics

Currently, the player may keep attacking a monster and there's no indicator of how much health the monster has left. Therefore, you will add a health bar to display the remaining health of the monster. As the player is attacking that monster, you'll start having the health bar decrease.

To do this, first you will implement a new method **createHealth** in **Monster.js** file. Phaser graphics will be used to draw some rectangles on the screen. The built-in graphics of Phaser allows you to draw basic shapes and you can fill them in with solid colors, gradients, etc. These graphics can be very useful when creating UIs, health bars, progress bars, and such. And it's all supported out of the box.

In the **createHealth** method, first you need to add a graphic scheme object named **healthBar** to the game by calling **scene.add.graphics** method. This method does not require any arguments to be passed to it. Next, you have to specify what it is that you want to draw, and how you want to fill and style that graphic. To set the initial fill style, call the **fillStyle** method on the **healthBar** object. This is going to determine the color that gets filled in the graphic. Two arguments will be passed into the **fillStyle** method; the hexadecimal code for the color white (0xffffffff), and the opacity of the color. The opacity is set to 1 so that the graphic would be filled with a solid color. Next, you will specify that you want the **healthBar** graphic to be a rectangle. For this, call the **fillRect** method on the **healthBar**. As arguments, specify the position of where this rectangle will be, as well as the width and height of that rectangle. Adjust the rectangle **y** position a little so that it's positioned a little bit above the monster game object, instead of lying right on top of the monster. Width can be 64 and height can be 5. See the code below:

```
createHealthBar() {  
    this.healthBar = this.scene.add.graphics();  
    this.healthBar.fillStyle(0xffffffff, 1);  
    this.healthBar.fillRect(this.x, this.y - 8, 64, 5);  
}
```

At the bottom of the **Monster** class constructor, call the new method **createHealth**. See the code below (line 20):

```
constructor(scene, x, y, key, frame, id, health, maxHealth) {  
    super(scene, x, y, key, frame);  
    this.scene = scene;  
    this.id = id;  
    this.health = health;  
    this.maxHealth = maxHealth;  
  
    // enable physics  
    this.scene.physics.world.enable(this);  
    // set immovable if another object collides with our monster  
    this.setImmovable(false);  
    // scale our monster  
    this.setScale(2);  
    // collide with world bounds  
    this.setCollideWorldBounds(true);
```



```
// add the monster to our existing scene
this.scene.add.existing(this);

//create the health bar
this.createHealthBar();
}
```

Save and refresh the game. Wait for a monster to spawn, and when it appears you will see that the health bar is now appearing.



Updating the position of the health bar

You can see that the health bar is still obstructing the monster graphic. The reason for this is, currently the origin of the container is in the dead center. If you want the health bar to be above the game object, you will have to update the origin of the container. To do this, update the constructor of the **Monster** class to change the origin. Use the **setOrigin** method for this and pass 0 as the argument. See the code below (line 20):

```
constructor(scene, x, y, key, frame, id, health, maxHealth) {
    super(scene, x, y, key, frame);
    this.scene = scene;
    this.id = id;
    this.health = health;
    this.maxHealth = maxHealth;

    // enable physics
    this.scene.physics.world.enable(this);
    // set immovable if another object collides with our monster
```

```
this.setImmovable(false);  
// scale our monster  
this.setScale(2);  
// collide with world bounds  
this.setCollideWorldBounds(true);  
// add the monster to our existing scene  
this.scene.add.existing(this);  
  
// update the origin  
this.setOrigin(0);  
  
this.createHealthBar();  
}
```

Save and refresh the game. Now when a monster is spawned you will see that the health bar is appearing above the monster.



Displaying the current health on the health bar

Currently the rectangle is solid white, and it looks like an empty health bar. You will use the white bar as the background of the health bar, and you will draw another rectangle on top of it that will be a gradient. That will signify how much health the monster currently has. To do this, update the **createHealthBar** method by calling the **fillGradientStyle** method on the **healthBar** object. The color of this gradient would be red and white, so pass the proper hex codes for those colors as arguments. Next, create a new rectangle at the same position as the previous rectangle so that they will align correctly. Update the width of the new rectangle to be proportional to the amount of health the monster has out of its maximum health. See the code below:


```
createHealthBar() {
    this.healthBar = this.scene.add.graphics();
    this.healthBar.fillStyle(0xffffffff, 1);
    this.healthBar.fillRect(this.x, this.y - 8, 64, 5);

    // creating a gradient rectangle
    this.healthBar.fillGradientStyle(0xff0000, 0xffffffff, 4);
    this.healthBar.fillRect(this.x, this.y - 8, 64 * (this.health / this.maxHealth),
5);
}
```

Save the game and refresh. When you test your changes you will see that now there's a gradient health bar over the monster. If you attack the monster, the monster will disappear.



Updating the health bar to change dynamically

You can see that there are still a few issues. The first issue is, the health bar is not updating when the player attacks the monster. The second issue is, once the monster becomes inactive the health bar stays in position even though the monster disappeared. It doesn't move with the monster.

Anytime a monster is attacked, you will want to update the health bar so that it will reflect the current health of the monster. To do this, go to **GameManager** class, and inside the **monsterAttacked** event in **setUpEventListener** method, add an **else** statement after the **if** statement. If the monster has not died yet, you want to admit an event that the monster has lost health. For this, emit a new event **updateMonsterHealth** along with the **monsterId** and **health** value. See the code below (lines 20-23):

```
setUpEventListener() {
    this.scene.events.on('pickUpChest', (chestId) => {
        // update the spawner
        if (this.chests[chestId]) {
            this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
        }
    });

    this.scene.events.on('monsterAttacked', (monsterId) => {
```



```
// update the spawner
if (this.monsters[monsterId]) {
  // subtract health monster model
  this.monsters[monsterId].loseHealth();

  // check the monsters health, and if dead remove that object
  if (this.monsters[monsterId].health <= 0) {
    // removing the monster
    this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
    this.scene.events.emit('monsterRemoved', monsterId);
  } else {
    // update the monsters health
    this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);
  }
}
});
}
```

Next, in the **GameScene** class, listen to the **updateMonsterHealth** event inside the **createGameManager** method. Upon receiving the event, a new method **updateHealth** is called on the monster. See the code below:

```
createGameManager() {
  this.events.on('spawnPlayer', (location) => {
    this.createPlayer(location);
    this.addCollisions();
  });

  this.events.on('chestSpawned', (chest) => {
    this.spawnChest(chest);
  });

  this.events.on('monsterSpawned', (monster) => {
    this.spawnMonster(monster);
  });

  this.events.on('monsterRemoved', (monsterId) => {
    this.monsters.getChildren().forEach((monster) => {
      if (monster.id === monsterId) {
        monster.makeInactive();
      }
    });
  });

  this.events.on('updateMonsterHealth', (monsterId, health) => {
    this.monsters.getChildren().forEach((monster) => {
      if (monster.id === monsterId) {
        monster.updateHealth(health);
      }
    });
  });
}
```



Implement the new method **updateHealth** in the **Monster** class. Set the health of the monster to be equal to the **health** value that was passed to the method as the argument. See the code below:

```
updateHealth(health) {  
    this.health = health;  
}
```

Next, you want to clear the existing graphics and redraw the rectangles so that you have the correct health of the monster being displayed. For this, create a new method **updateHealthBar** in **Monster** class. Inside **updateHealthBar**, first call the **clear** method on the **healthBar** object. The **clear** method clears all of the graphics that have been drawn so that they can be redrawn. Next, remove all the logic, except creating the **healthBar** object, from **createHealthBar** method. Put the removed logic inside the **updateHealthBar** method instead. Call **updateHealthBar** method inside the **createHealthBar** method and inside the **updateHealth** method. This way, the logic for creating the health bar is separated from updating it. See the code below for the changes:

```
createHealthBar() {  
    this.healthBar = this.scene.add.graphics();  
    this.updateHealthBar();  
}  
  
updateHealthBar() {  
    this.healthBar.clear();  
    this.healthBar.fillStyle(0xffffffff, 1);  
    this.healthBar.fillRect(this.x, this.y - 8, 64, 5);  
    this.healthBar.fillGradientStyle(0xff0000, 0xffffffff, 4);  
    this.healthBar.fillRect(this.x, this.y - 8, 64 * (this.health / this.maxHealth),  
5);  
}  
  
updateHealth(health) {  
    this.health = health;  
    this.updateHealthBar();  
}
```

Save and refresh the game. Wait for a monster to spawn and test changes. When you attack a monster, you will see its health bar starting to decrease.



Next, you want to clear the health bar when the monster becomes inactive. For this, update the **makeInactive** method in **Monster** class by calling the **clear** method on the **healthBar**. This will clear all health bar graphics. See code below:

```
makeInactive() {  
    this.setActive(false);  
    this.setVisible(false);  
    this.body.checkCollision.none = true;  
    // clear the health bar graphics  
    this.healthBar.clear();  
}
```

When the monster game object becomes active again, you want to update the health bar. State this logic in the **makeActive** method in the **Monster** class by calling the **updateHealthBar** method. See the code below:

```
makeActive() {  
    this.setActive(true);  
    this.setVisible(true);  
    this.body.checkCollision.none = false;  
    // update the health bar  
    this.updateHealthBar();  
}
```

Save and test the game. Now, the game should have the desired effect. If you attack a monster, it will die and the health bar will disappear with it. When a monster spawns, they have a full health bar.



Finally, in the **MonsterModel** class, remove the console log inside the **loseHealth** method. See the code below:

```
loseHealth() {  
    this.health -= 1;  
}
```

In this lesson, you will create a new **PlayerModel** class, and update the **GameManager** to use it.

Creating the PlayerModel class

When a monster dies, you want to take the gold that particular monster is worth and give it to the player. In addition, you can have the monsters attack the player back, so that the game is a little more interesting. To do these, you will add a new model for the player. That way, you can have all the logic for the player's health and the amount of gold they have in the same location where you're tracking monster's health, etc.

In your **gamemanager** folder, create a new file called **PlayerModel.js**. Build the constructor as shown below:

```
class PlayerModel {
  constructor() {
    this.health = 10;
    this.maxHealth = 10;
    this.gold = 0;
  }
}
```

The player needs **health** and **maxHealth** properties, both of which will be set to 10 by default. You also need to keep track of how much **gold** that player has, which will be set to 0 initially.

Next, add the class file to your **index.html** file. See the code below (line 8):

```
<body>
  <div id="phaser-game"></div>
  <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/node-
  uuid/1.4.8/uuid.min.js"></script>
  <script src="assets/js/game_manager/utils.js" charset="utf-8"></script>
  <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>
  <script src="assets/js/game_manager/MonsterModel.js" charset="utf-8"></script>
  >
  <script src="assets/js/game_manager/PlayerModel.js" charset="utf-8"></script>
  <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>
  <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>
  <script src="assets/js/classes/player/Player.js" charset="utf-8"></script>
  <script src="assets/js/classes/player/PlayerContainer.js" charset="utf-8"></s
  cript>
  <script src="assets/js/classes/Monster.js" charset="utf-8"></script>
  <script src="assets/js/classes/Chest.js" charset="utf-8"></script>
  <script src="assets/js/classes/UIButton.js" charset="utf-8"></script>
  <script src="assets/js/classes/Map.js" charset="utf-8"></script>
  <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
  <script src="assets/js/main.js" charset="utf-8"></script>
</body>
```



Updating the GameManager class

Next, update the constructor of the **GameManager** class so that the **player** is stored in the **players** object. See the code below (line 8):

```
constructor(scene, mapData) {
    this.scene = scene;
    this.mapData = mapData;

    this.spawnners = {};
    this.chests = {};
    this.monsters = {};
    this.players = {};

    this.playerLocations = [];
    this.chestLocations = {};
    this.monsterLocations = {};
}
```

Because you're storing the player's information here, you will need a way to track that player. Therefore, add an **id** to the **PlayerModel** constructor. The UUID library is used to generate the **id** of the player (similar to how the **id** was generated for a monster in **MonsterModel** class). See the code below:

```
constructor() {
    this.health = 10;
    this.maxHealth = 10;
    this.gold = 0;
    this.id = `player-${uuid.v4()}`;
}
```

In the **spawnPlayer** method in the **GameManager**, currently you're choosing a random location for the player. Move this logic into the constructor of the **PlayerModel** class. The constructor will also take a parameter **spawnLocations**. Set the **x** and **y** values of the player to the new random location. See the code below:

```
constructor(spawnLocations) {
    this.health = 10;
    this.maxHealth = 10;
    this.gold = 0;
    this.id = `player-${uuid.v4()}`;
    this.spawnLocations = spawnLocations;

    const location = this.spawnLocations[Math.floor(Math.random() * this.spawnLocations.length)];
    [this.x, this.y] = location;
}
```



The syntax in line 9 above is a shorthand to allow you to pull multiple values out of an array and set them to a variable. Here you're saying `this.x` is equal to the first element of the array, and `this.y` is the second element. This is called a destructuring assignment. See the additional notes for more information on this.

Currently, in the **spawnPlayer** method in the **GameManager**, you're just emitting a scene with the location of the player. Instead, you will want to create a new instance of the **PlayerModel** first to create a **player** object. Next, add this **player** to the **players** object. Change the emitted event to pass the **player** object instead of the **location**. See the code below:

```
spawnPlayer() {
  const player = new PlayerModel(this.playerLocations);
  this.players[player.id] = player;
  this.scene.events.emit('spawnPlayer', player);
}
```

Updating the GameScene class

In the **GameScene** class, edit the logic at receiving the **spawnPlayer** event in the **createGameManager** method. Instead of a location, the event will pass the **playerObject**, which will be passed into the **createPlayer** method. See the code below (lines 2-5):

```
createGameManager() {
  this.events.on('spawnPlayer', (playerObject) => {
    this.createPlayer(playerObject);
    this.addCollisions();
  });

  this.events.on('chestSpawned', (chest) => {
    this.spawnChest(chest);
  });

  this.events.on('monsterSpawned', (monster) => {
    this.spawnMonster(monster);
  });

  this.events.on('monsterRemoved', (monsterId) => {
    this.monsters.getChildren().forEach((monster) => {
      if (monster.id === monsterId) {
        monster.makeInactive();
      }
    });
  });

  this.events.on('updateMonsterHealth', (monsterId, health) => {
    this.monsters.getChildren().forEach((monster) => {
      if (monster.id === monsterId) {
        monster.updateHealth(health);
      }
    });
  });
}
```




Next, update the **createPlayer** method in **GameScene** to take **playerObject** as the argument. Then, change the location of the player container by referencing **playerObject.x** and **playerObject.y**. Pass the **health**, **maxHealth**, and **id** values as well. See the code below:

```
createPlayer(playerObject) {  
    this.player = new PlayerContainer(  
        this,  
        playerObject.x * 2,  
        playerObject.y * 2,  
        'characters',  
        0,  
        playerObject.health,  
        playerObject.maxHealth,  
        playerObject.id,  
    );  
}
```

Updating the PlayerContainer class

To support the above change made in the **GameScene** class, you will add a few new parameters **health**, **maxHealth**, and **id** to the constructor of the **PlayerContainer** class. Set the properties of the constructor with the appropriate parameters. See the code below:

```
constructor(scene, x, y, key, frame, health, maxHealth, id) {  
    super(scene, x, y);  
    this.scene = scene; // the scene this container will be added to  
    this.velocity = 160; // the velocity when moving our player  
    this.currentDirection = Direction.RIGHT;  
    this.playerAttacking = false;  
    this.flipX = true;  
    this.swordHit = false;  
  
    // update the new properties  
    this.health = health;  
    this.maxHealth = maxHealth;  
    this.id = id;  
  
    // set a size on the container  
    this.setSize(64, 64);  
    // enable physics  
    this.scene.physics.world.enable(this);  
    // collide with world bounds  
    this.body.setCollideWorldBounds(true);  
    // add the player container to our existing scene  
    this.scene.add.existing(this);  
    // have the camera follow the player  
    this.scene.cameras.main.startFollow(this);
```




```
// create the player
this.player = new Player(this.scene, 0, 0, key, frame);
this.add(this.player);

// create the weapon game object
this.weapon = this.scene.add.image(40, 0, 'items', 4);
this.scene.add.existing(this.weapon);
this.weapon.setScale(1.5);
this.scene.physics.world.enable(this.weapon);
this.add(this.weapon);
this.weapon.alpha = 0;

// create the player healthbar
this.createHealthBar();
}
```

Save and refresh the game. You will see that the player is still being created, is able to attack, and can interact with the blocked layers.

Additional notes: The destructuring assignment

The destructuring assignment is an expression that makes it possible to unpack values from arrays. Instead of assigning a variable to one element in an array, you can unpack multiple values into multiple variables at a time. Here is an example:

```
var a, b, rest;
[a, b] = [10, 20]
```

In this lesson, you will move the logic for updating the score to the **GameManager** class and update the player's gold in the **PlayerModel** class.

Currently, you have the **PlayerModel** in place and it seems to be working. The next thing to do, is to move all the logic for the score and the amount of gold from the **GameScene** class to the **PlayerModel** class. The main reason for doing this is, you want to track the information for gold in the **PlayerModel** class. Anytime the player picks up a chest or kills a monster, you will have all the related information in one location inside the **GameManager** for updating that **PlayerModel** instance. This way, it doesn't have to be updated multiple times in the **GameScene** class.

Updating the GameScene class

To do these changes, first remove the logic for setting the score in the **init** method in **GameScene** class. See the code below:

```
init() {  
    this.scene.launch('Ui');  
}
```

Next, refactor the logic for gold and score in the **collectChest** method. Remove the logic for updating the score. Also remove the logic for making the chest object inactive. When emitting the **pickUpChest** event, emit the player id in addition to the chest id. See the refactored code below:

```
collectChest(player, chest) {  
    // play gold pickup sound  
    this.goldPickupAudio.play();  
    this.events.emit('pickUpChest', chest.id, player.id);  
}
```

Updating the GameManager class

In the **GameManager** class, update the **pickUpChest** event in the **setUpEventListener** method to receive the **playerId** in addition to the **chestId**. You will need to get a reference to the amount of gold you need to give the players. For this, set the value of the gold property in the relevant chest to a new variable **gold**. Then, update the player's gold by calling a new method **updateGold**, which will take the **gold** value as its argument. Next, emit an event **updateScore** with the player's **gold** value for updating the score in the UI. Add another event **chestRemoved** to be emitted when a chest is removed. See the code below (lines 2-15):

```
setUpEventListener() {  
    this.scene.events.on('pickUpChest', (chestId, playerId) => {  
        // update the spawner  
        if (this.chests[chestId]) {  
            const { gold } = this.chests[chestId];  
  
            // updating the players gold  
            this.players[playerId].updateGold(gold);  
            this.scene.events.emit('updateScore', this.players[playerId].gold);  
        }  
    });  
}
```



```
// removing the chest
this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
this.scene.events.emit('chestRemoved', chestId);
}
});

this.scene.events.on('monsterAttacked', (monsterId) => {
  // update the spawner
  if (this.monsters[monsterId]) {
    // subtract health monster model
    this.monsters[monsterId].loseHealth();

    // check the monsters health, and if dead remove that object
    if (this.monsters[monsterId].health <= 0) {
      // removing the monster
      this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
      this.scene.events.emit('monsterRemoved', monsterId);
    } else {
      // update the monsters health
      this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);
    }
  }
});
}
```

In the above code, line 5 refers to the syntax for object destructuring. See additional notes for more information.

Back in the **GameScene** class, update the **createGameManager** method to receive the **chestRemoved** event. If the received **chestId** is in the **chests** object, remove that chest by calling the **makeInactive** method on the **chest** object. See the code below (lines 15-21):

```
createGameManager() {
  this.events.on('spawnPlayer', (playerObject) => {
    this.createPlayer(playerObject);
    this.addCollisions();
  });

  this.events.on('chestSpawned', (chest) => {
    this.spawnChest(chest);
  });

  this.events.on('monsterSpawned', (monster) => {
    this.spawnMonster(monster);
  });

  this.events.on('chestRemoved', (chestId) => {
    this.chests.getChildren().forEach((chest) => {
      if (chest.id === chestId) {
        chest.makeInactive();
      }
    });
  });
}
```

```
    });  
  });  
  
  this.events.on('monsterRemoved', (monsterId) => {  
    this.monsters.getChildren().forEach((monster) => {  
      if (monster.id === monsterId) {  
        monster.makeInactive();  
      }  
    });  
  });  
  
  this.events.on('updateMonsterHealth', (monsterId, health) => {  
    this.monsters.getChildren().forEach((monster) => {  
      if (monster.id === monsterId) {  
        monster.updateHealth(health);  
      }  
    });  
  });  
  
  this.gameManager = new GameManager(this, this.map.map.objects);  
  this.gameManager.setup();  
}
```

Updating the PlayerModel class

Implement the new method **updateGold** in the **PlayerModel** class. See the code below:

```
updateGold(gold) {  
  this.gold += gold;  
}
```

Save and refresh the game. Test the changes by picking up a model and seeing if the UI is updated correctly to show the amount of gold added.





Additional notes: Object destructuring

When you are unpacking values from an object, you can assign them to a new variable name as shown below:

```
var o = {p: 42, q: true};  
var {p: foo, q: bar} = o;
```

You can also provide a default value:

```
var {a = 10, b = 5} = {a = 3};
```



In this lesson, you will create a health bar for the player and update it to change dynamically as the player gains and loses health.

Updating the player's gold

Now that you have the **PlayerModel** integrated with the **GameManager** and the **GameScene** classes, you're going to work on enhancing the event of when a player attacks a monster. That way, the amount of gold that the monster had will belong to the player when the monster dies.

To do this, first you will update the **setUpEventListener** method in the **GameManager** class. In the **monsterAttacked** event, edit the arguments to receive **playerId** in addition to the **monsterId** for the callback function. Then, inside the **if** statement where the **health** of the monster is equal to or drops below zero, you will update the player's gold. First, get a reference as to how much gold the monster has, and set that to the variable **gold**. Then, call the **updateGold** method on the player object with the **gold** value as the argument. Next, emit an event **updateScore** along with the player's amount of gold. See the code below (lines 25-29):

```
setUpEventListener() {
  this.scene.events.on('pickUpChest', (chestId, playerId) => {
    // update the spawner
    if (this.chests[chestId]) {
      const { gold } = this.chests[chestId];

      // updating the players gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the chest
      this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
      this.scene.events.emit('chestRemoved', chestId);
    }
  });

  this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
    // update the spawner
    if (this.monsters[monsterId]) {
      // subtract health monster model
      this.monsters[monsterId].loseHealth();

      // check the monsters health, and if dead remove that object
      if (this.monsters[monsterId].health <= 0) {
        const { gold } = this.monsters[monsterId];

        //updating the player's gold
        this.players[playerId].updateGold(gold);
        this.scene.events.emit('updateScore', this.players[playerId].gold);

        // removing the monster
        this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
        this.scene.events.emit('monsterRemoved', monsterId);
      } else {
        // update the monsters health
        this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);
      }
    }
  });
}
```




```
    }  
  });  
}
```

In the **GameScene** class, update the **enemyOverlap** method to pass the **id** of the player as well when emitting the **monsterAttacked** event. See the code below:

```
enemyOverlap(player, enemy) {  
  if (this.player.playerAttacking && !this.player.swordHit) {  
    this.player.swordHit = true;  
    this.events.emit('monsterAttacked', enemy.id, player.id);  
  }  
}
```

Save the changes and refresh the game. When a monster spawns, attack it to check if the changes are working correctly.

Unfortunately, you will see that there's an Uncaught TypeError appearing in the console log and the game is not working correctly. The reason for this is, in the **enemyOverlap** method in the **GameScene** class, the **player** should not be passed to the method as an argument. Instead, it actually has to be the player's **weapon** that needs to be passed. Update that and how the player id is referenced as well when emitting the **monsterAttacked** event. See the code below:

```
enemyOverlap(weapon, enemy) {  
  if (this.player.playerAttacking && !this.player.swordHit) {  
    this.player.swordHit = true;  
    this.events.emit('monsterAttacked', enemy.id, this.player.id);  
  }  
}
```

Save changes and refresh the game again. Now when you attack a monster and the monster dies, the player's gold is updated. Pick up more chests or attack more monsters to see if the player's gold is updating properly.



Updating monsters to attack the player

To make the game a little more interesting, you will have the monsters attack the player when the player attacks them. This way it acts like they're defending themselves, and it adds a little bit of randomness to the game.

In the **GameManager** class, under the **monsterAttacked** event in **setUpEventListener** method, you previously made a reference to the amount of gold the monster has. Update this reference to include the monster's attack value as well by setting the property into a new variable **attack**. Then, add the logic to update the player's health while the monster is still alive in the **else** statement. Call a new method **updateHealth** on the player and subtract the **attack** value of the monster. See the code below:

```
setUpEventListener() {
  this.scene.events.on('pickUpChest', (chestId, playerId) => {
    // update the spawner
    if (this.chests[chestId]) {
      const { gold } = this.chests[chestId];

      // updating the players gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the chest
      this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
      this.scene.events.emit('chestRemoved', chestId);
    }
  });

  this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
    // update the spawner
    if (this.monsters[monsterId]) {
      // subtract health monster model
      this.monsters[monsterId].loseHealth();

      // check the monsters health, and if dead remove that object
      if (this.monsters[monsterId].health <= 0) {
        const { gold, attack } = this.monsters[monsterId]; // reference to attack v
```



alue

```
//updating the player's gold
this.players[playerId].updateGold(gold);
this.scene.events.emit('updateScore', this.players[playerId].gold);

// removing the monster
this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
this.scene.events.emit('monsterRemoved', monsterId);
} else {
    // update the player's health
    this.players[playerId].updateHealth(-attack);
    // update the monsters health
    this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);
}
}
});
}
```

In the **PlayerModel** class, implement the new method **updateHealth** as shown below:

```
updateHealth(health) {
    this.health += health;
    console.log(this.health);
}
```

Creating player's health bar

Now that you have a way for a monster to attack the player, you need to display the player's health bar so that you know how much health the player currently has. To do this, you will update the **PlayerContainer** class. At the bottom of the constructor of **PlayerContainer** class, call the method **createHealthBar** to create the player's health bar. See the code below (line 37):

```
constructor(scene, x, y, key, frame, health, maxHealth, id) {
    super(scene, x, y);
    this.scene = scene; // the scene this container will be added to
    this.velocity = 160; // the velocity when moving our player
    this.currentDirection = Direction.RIGHT;
    this.playerAttacking = false;
    this.flipX = true;
    this.swordHit = false;
    this.health = health;
    this.maxHealth = maxHealth;
    this.id = id;

    // set a size on the container
    this.setSize(64, 64);
    // enable physics
    this.scene.physics.world.enable(this);
    // collide with world bounds
```



```
this.body.setCollideWorldBounds(true);
// add the player container to our existing scene
this.scene.add.existing(this);
// have the camera follow the player
this.scene.cameras.main.startFollow(this);

// create the player
this.player = new Player(this.scene, 0, 0, key, frame);
this.add(this.player);

// create the weapon game object
this.weapon = this.scene.add.image(40, 0, 'items', 4);
this.scene.add.existing(this.weapon);
this.weapon.setScale(1.5);
this.scene.physics.world.enable(this.weapon);
this.add(this.weapon);
this.weapon.alpha = 0;

// create the player healthbar
this.createHealthBar();
}
```

Previously, you implemented **createHealthBar**, **updateHealthBar**, and **updateHealth** methods for the monster's health bar in **Monster.js** file. Copy all of that logic into the **PlayerContainer** class. See the code below:

```
createHealthBar() {
    this.healthBar = this.scene.add.graphics();
    this.updateHealthBar();
}

updateHealthBar() {
    this.healthBar.clear();
    this.healthBar.fillStyle(0xffffffff, 1);
    this.healthBar.fillRect(this.x, this.y - 8, 64, 5);
    this.healthBar.fillGradientStyle(0xff0000, 0xffffffff, 4);
    this.healthBar.fillRect(this.x, this.y - 8, 64 * (this.health / this.maxHealth),
5);
}

updateHealth(health) {
    this.health = health;
    this.updateHealthBar();
}
```

Save changes and test the game. You can see that the health bar for the player is being created, but it's not positioned above the player nor is it moving with the player.



Updating the position and movement of the player's health bar

To fix the positioning, you will update the **x** and **y** values of the health bar rectangles (background and gradient) in the **updateHealthBar** method in the **PlayerContainer**. This way, the health bar will be off-centered and positioned above the player's head. See the code below:

```
updateHealthBar() {
    this.healthBar.clear();
    this.healthBar.fillStyle(0xffffffff, 1);
    this.healthBar.fillRect(this.x - 32, this.y - 40, 64, 5);
    this.healthBar.fillGradientStyle(0xff0000, 0xffffffff, 4);
    this.healthBar.fillRect(this.x - 32, this.y - 40, 64 * (this.health / this.maxHealth), 5);
}
```

Now if you refresh the game, you will see that the health bar is positioned above the player's head but it is still not moving with the player. To fix this, call the **updateHealthBar** method at the bottom of the **update** method in the **PlayerContainer** so that its position can be updated. See the code below (line 58):

```
update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
    }
}
```



```
        this.player.flipX = false;
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
        this.player.flipX = true;
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }

    if (Phaser.Input.Keyboard.JustDown(cursors.space) && !this.playerAttacking) {
        this.weapon.alpha = 1;
        this.playerAttacking = true;
        this.scene.time.delayedCall(150, () => {
            this.weapon.alpha = 0;
            this.playerAttacking = false;
            this.swordHit = false;
        }, [], this);
    }

    if (this.playerAttacking) {
        if (this.weapon.flipX) {
            this.weapon.angle -= 10;
        } else {
            this.weapon.angle += 10;
        }
    } else {
        if (this.currentDirection === Direction.DOWN) {
            this.weapon.setAngle(-270);
        } else if (this.currentDirection === Direction.UP) {
            this.weapon.setAngle(-90);
        } else {
            this.weapon.setAngle(0);
        }
    }

    this.weapon.flipX = false;
    if (this.currentDirection === Direction.LEFT) {
        this.weapon.flipX = true;
    }
}

this.updateHealthBar(); // update player's health bar
}
```

Save and refresh the game. Now you can see that the health bar is moving along with the player.



However, if you try attacking a monster, you will encounter a bug in the code and will not be able to attack. To fix this, go back to the **setupEventListener** method in the **GameManager**. The issue is, under the **monsterAttacked** event, you can see that the reference to the monster's **attack** value is out of scope from the **else** statement where the player's health is being updated. Move the referencing to the **gold** and **attack** values to fix the scope. See the code below (line 5):

```
setupEventListener() {
  this.scene.events.on('pickUpChest', (chestId, playerId) => {
    // update the spawner
    if (this.chests[chestId]) {
      const { gold } = this.chests[chestId];

      // updating the players gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the chest
      this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
      this.scene.events.emit('chestRemoved', chestId);
    }
  });

  this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
    // update the spawner
    if (this.monsters[monsterId]) {
      const { gold, attack } = this.monsters[monsterId];

      // subtract health monster model
      this.monsters[monsterId].loseHealth();

      // check the monsters health, and if dead remove that object
      if (this.monsters[monsterId].health <= 0) {

        //updating the player's gold
        this.players[playerId].updateGold(gold);
        this.scene.events.emit('updateScore', this.players[playerId].gold);

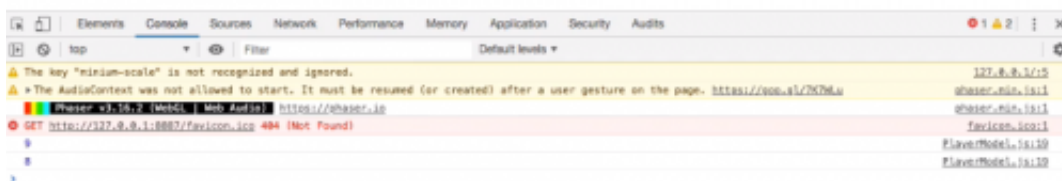
        // removing the monster
```

```

        this.spawnners[this.monsters[monsterId].spawnnerId].removeObject(monsterId);
        this.scene.events.emit('monsterRemoved', monsterId);
    } else {
        // update the player's health
        this.players[playerId].updateHealth(-attack);
        // update the monsters health
        this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);
    }
}
});
}

```

Now if you refresh the game, you will see that the health is being logged correctly to the console.



Updating the health bar to change dynamically

However, the player's health bar is not updating to display the health being decreased. To fix this issue, you will need to emit an event with the player's health. You will update the **monsterAttacked** event in the **setupEventListener** method in the **GameManager** class. After the call to the **updateHealth** method for the player, emit a new event named **updatePlayerHealth** along with the **playerId** and the player's **health** value. See the code below (line 38):

```

setupEventListener() {
    this.scene.events.on('pickUpChest', (chestId, playerId) => {
        // update the spawner

```



```
if (this.chests[chestId]) {
  const { gold } = this.chests[chestId];

  // updating the players gold
  this.players[playerId].updateGold(gold);
  this.scene.events.emit('updateScore', this.players[playerId].gold);

  // removing the chest
  this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
  this.scene.events.emit('chestRemoved', chestId);
}
});

this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
  // update the spawner
  if (this.monsters[monsterId]) {
    const { gold, attack } = this.monsters[monsterId];

    // subtract health monster model
    this.monsters[monsterId].loseHealth();

    // check the monsters health, and if dead remove that object
    if (this.monsters[monsterId].health <= 0) {

      //updating the player's gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the monster
      this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
      this.scene.events.emit('monsterRemoved', monsterId);
    } else {
      // update the player's health
      this.players[playerId].updateHealth(-attack);
      this.scene.events.emit('updatePlayerHealth', playerId, this.players[playerId].health);
      // update the monsters health
      this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);
    }
  }
});
}
```

In the **GameScene** class, update the **createGameManager** method to receive the new **updatePlayerHealth** event. When the event is received, call the **updateHealth** method on the player. See the code below (lines 40-42):

```
createGameManager() {
  this.events.on('spawnPlayer', (playerObject) => {
    this.createPlayer(playerObject);
    this.addCollisions();
  });
}
```



```
this.events.on('chestSpawned', (chest) => {
    this.spawnChest(chest);
});

this.events.on('monsterSpawned', (monster) => {
    this.spawnMonster(monster);
});

this.events.on('chestRemoved', (chestId) => {
    this.chests.getChildren().forEach((chest) => {
        if (chest.id === chestId) {
            chest.makeInactive();
        }
    });
});

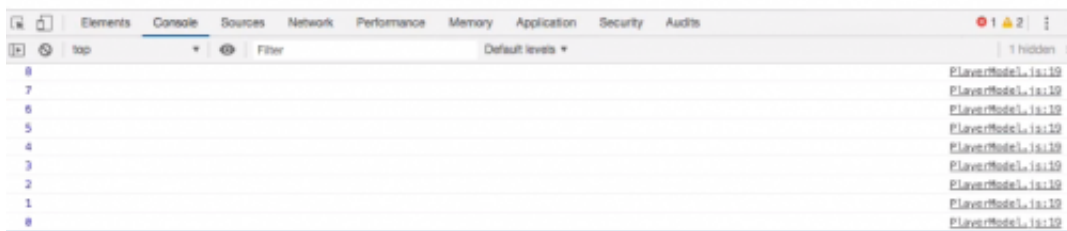
this.events.on('monsterRemoved', (monsterId) => {
    this.monsters.getChildren().forEach((monster) => {
        if (monster.id === monsterId) {
            monster.makeInactive();
        }
    });
});

this.events.on('updateMonsterHealth', (monsterId, health) => {
    this.monsters.getChildren().forEach((monster) => {
        if (monster.id === monsterId) {
            monster.updateHealth(health);
        }
    });
});

this.events.on('updatePlayerHealth', (playerId, health) => {
    this.player.updateHealth(health);
});

this.gameManager = new GameManager(this, this.map.map.objects);
this.gameManager.setup();
}
```

Save changes and refresh the game. Now, you can see that the player's health bar is decreasing appropriately when attacked.



However, you can see that the player doesn't die even though the player's health reached zero. This will be implemented in the next lesson.

Additional notes: Variable scope

JavaScript has two types of scope: Local and Global. By default, JavaScript does not have any block scope, so any variables defined inside an **if** statement will be hoisted out. However the new **let** and **const** declarations can be used to have proper block scope.



In the last lesson, you added the player's health bar. However, at the end you encountered an issue of where the player will not die even when the health bar drops below zero. In this lesson, you will implement the logic to respawn the player once their health bar reaches zero. In addition, there needs to be a consequence of the player dying. Therefore, when the player dies, they will lose half of their gold and the player will be respawned to a new, random location.

Penalizing the player's death

To implement the above, you will first make changes to the **GameManager** class. In the **setUpEventListener** method, under the **monsterAttacked** event, consider the **else** block where the player is being attacked by the monster. Here, you will add a check to see if the player's **health** is below zero. If **health** is below zero, first update the player's gold to be half of the current amount of **gold** they have by using the **updateGold** method. Pass the amount of **gold** as an integer argument in case it would become a floating point number when divided by 2, because you need it to be a round number. Next, emit the event **updateScore** to update the player's current **gold**. Then, have the player respawn. For this, you will call a new method **respawn** on the player which will be implemented later. Finally, emit an event **respawnPlayer** with the player object. Since the player will be respawned at a new location when this event is emitted, it is important to pass the full object of the player with all of the information. See the code below (lines 42-50):

```
setUpEventListener() {
  this.scene.events.on('pickUpChest', (chestId, playerId) => {
    // update the spawner
    if (this.chests[chestId]) {
      const { gold } = this.chests[chestId];

      // updating the players gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the chest
      this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
      this.scene.events.emit('chestRemoved', chestId);
    }
  });

  this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
    // update the spawner
    if (this.monsters[monsterId]) {
      const { gold, attack } = this.monsters[monsterId];

      // subtract health monster model
      this.monsters[monsterId].loseHealth();

      // check the monsters health, and if dead remove that object
      if (this.monsters[monsterId].health <= 0) {

        //updating the player's gold
        this.players[playerId].updateGold(gold);
        this.scene.events.emit('updateScore', this.players[playerId].gold);

        // removing the monster
        this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
        this.scene.events.emit('monsterRemoved', monsterId);
      } else {
```




```
// update the player's health
this.players[playerId].updateHealth(-attack);
this.scene.events.emit('updatePlayerHealth', playerId, this.players[playerId].health);
// update the monsters health
this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);

// check the player's health, if below 0 have the player respawn
if (this.players[playerId].health <= 0) {
  // update the gold the player has
  this.players[playerId].updateGold(parseInt(-this.players[playerId].gold / 2), 10);
  this.scene.events.emit('updateScore', this.players[playerId].gold);

  // respawn the player
  this.players[playerId].respawn();
  this.scene.events.emit('respawnPlayer', this.players[playerId]);
}
}
});
}
```

Respawning the player

Next, you will implement the new method **respawn** in the **PlayerModel** class. When the player respawns, the first thing you will want to do is to update their **health** to be equal to the **maxHealth** value. Then, pick a new **location** for the respawning. Generate a random location for this. See the code below:

```
respawn() {
  this.health = this.maxHealth;
  const location = this.spawnLocations[Math.floor(Math.random() * this.spawnLocations.length)];
  [this.x, this.y] = location;
}
```

In the **GameScene** class, update the **createGameManager** method to receive the new **respawnPlayer** event. At this event, call the **respawn** method on the playerObject. See the code below (lines 44-46):

```
createGameManager() {
  this.events.on('spawnPlayer', (playerObject) => {
    this.createPlayer(playerObject);
    this.addCollisions();
  });

  this.events.on('chestSpawned', (chest) => {
    this.spawnChest(chest);
  });
}
```



```
this.events.on('monsterSpawned', (monster) => {
    this.spawnMonster(monster);
});

this.events.on('chestRemoved', (chestId) => {
    this.chests.getChildren().forEach((chest) => {
        if (chest.id === chestId) {
            chest.makeInactive();
        }
    });
});

this.events.on('monsterRemoved', (monsterId) => {
    this.monsters.getChildren().forEach((monster) => {
        if (monster.id === monsterId) {
            monster.makeInactive();
        }
    });
});

this.events.on('updateMonsterHealth', (monsterId, health) => {
    this.monsters.getChildren().forEach((monster) => {
        if (monster.id === monsterId) {
            monster.updateHealth(health);
        }
    });
});

this.events.on('updatePlayerHealth', (playerId, health) => {
    this.player.updateHealth(health);
});

this.events.on('respawnPlayer', (playerObject) => {
    this.player.respawn(playerObject);
});

this.gameManager = new GameManager(this, this.map.map.objects);
this.gameManager.setup();
}
```

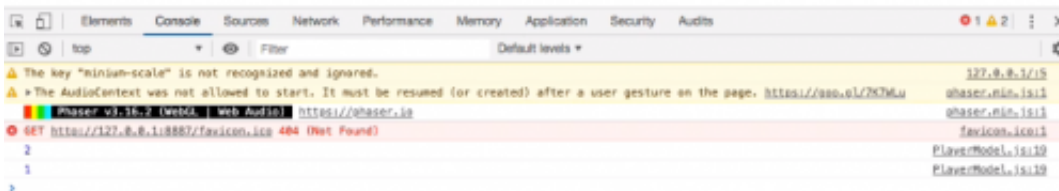
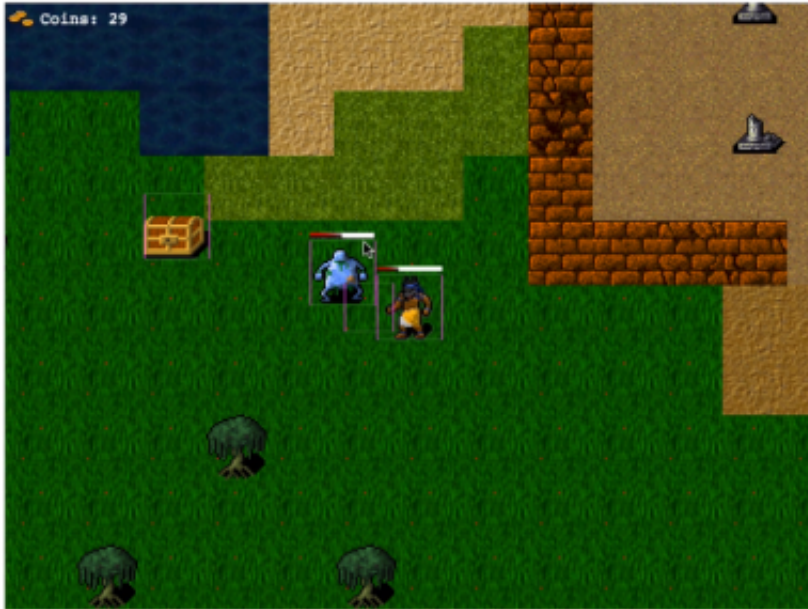
In the **PlayerContainer** class, implement the **respawn** method that acts on the player object and updates the information of the player. Within the method, first update the **health** of the player. Then, update the position of the player by calling the **setPosition** method. Call the **updateHealthBar** method next to reset the health bar graphic. See the code below:

```
respawn(playerObject) {
    this.health = playerObject.health;
    this.setPosition(playerObject.x, playerObject.y);
    this.updateHealthBar();
}
```

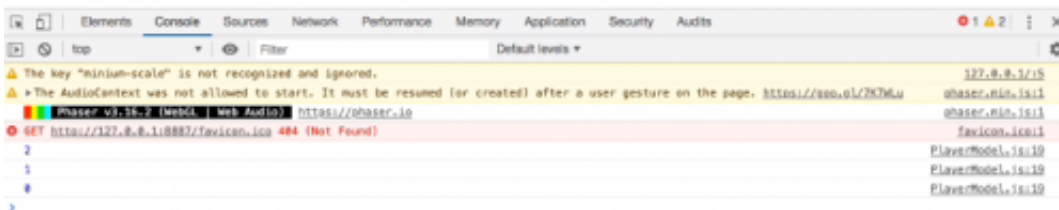


Before testing for the changes, change the default **maxHealth** value in **PlayerModel** constructor to 3 so that it is easier to test the death and respawning of the player.

Save all the changes and refresh the game. First let the player gain some gold by collecting some chests. Then, wait for a monster to spawn and attack it.



You can see above that the player has 29 gold coins, and that they should die if they attack one more time according to the console log.



Now you can see that the player has respawned at a new location with a full health bar, but gold has been decreased in half.

Regaining health when monster dies

Finally, you will have the player regain some health every time they kill a monster to increase the chance that the player will stay alive. For this, update the **setUpEventListener** method in the **GameManager** class. In the **monsterAttacked** event, you have an **if** block that checks the monster's health to remove it if dead. Here, update the player's health after removing the monster. Add 2 bonus values to the player's health by using the **updateHealth** method. See the code below (line 37):

```
setUpEventListener() {
  this.scene.events.on('pickUpChest', (chestId, playerId) => {
    // update the spawner
    if (this.chests[chestId]) {
      const { gold } = this.chests[chestId];

      // updating the players gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the chest
      this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
      this.scene.events.emit('chestRemoved', chestId);
    }
  })
}
```



```
});

this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
  // update the spawner
  if (this.monsters[monsterId]) {
    const { gold, attack } = this.monsters[monsterId];

    // subtract health monster model
    this.monsters[monsterId].loseHealth();

    // check the monsters health, and if dead remove that object
    if (this.monsters[monsterId].health <= 0) {

      //updating the player's gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

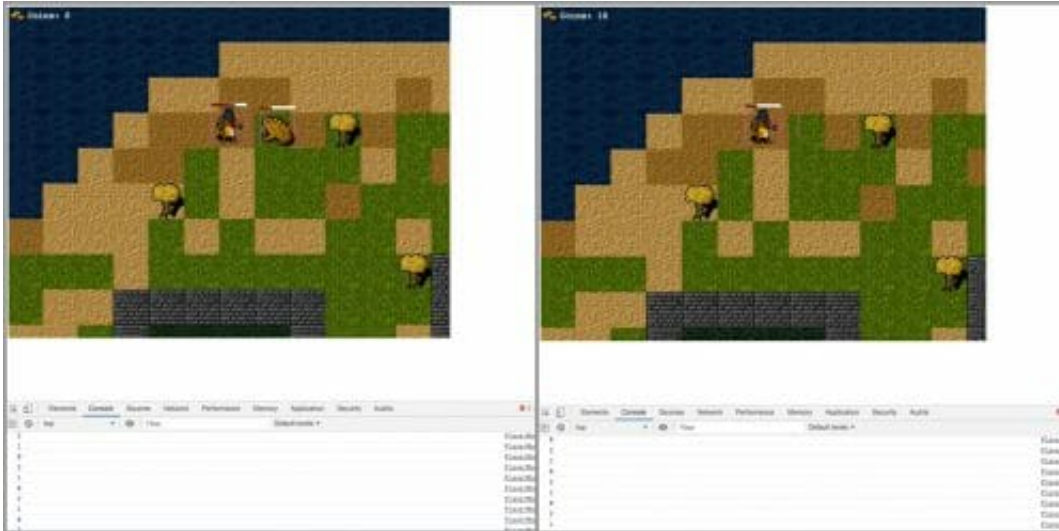
      // removing the monster
      this.spawners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
      this.scene.events.emit('monsterRemoved', monsterId);

      // add bonus health to the player
      this.players[playerId].updateHealth(2);
    } else {
      // update the player's health
      this.players[playerId].updateHealth(-attack);
      this.scene.events.emit('updatePlayerHealth', playerId, this.players[playerId].health);
      // update the monsters health
      this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);

      // check the player's health, if below 0 have the player respawn
      if (this.players[playerId].health <= 0) {
        // update the gold the player has
        this.players[playerId].updateGold(parseInt(-this.players[playerId].gold / 2), 10);
        this.scene.events.emit('updateScore', this.players[playerId].gold);

        // respawn the player
        this.players[playerId].respawn();
        this.scene.events.emit('respawnPlayer', this.players[playerId]);
      }
    }
  }
});
}
```

Save and refresh the game. Now when you attack a monster, you will see that the player gains **maxHealth** amount back when the monster dies.



However, even though the correct health is logged into the console, you can see that the health bar is not updating correctly. To fix this, go to the **monsterAttacked** event in the **setUpEventListener** method in **GameManager** class. Emit the event **updatePlayerHealth** after adding bonus health for the player when a monster dies, so that the health bar will be updated as well. See the code below (line 38):

```
setUpEventListener() {
  this.scene.events.on('pickUpChest', (chestId, playerId) => {
    // update the spawner
    if (this.chests[chestId]) {
      const { gold } = this.chests[chestId];

      // updating the players gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the chest
      this.spawners[this.chests[chestId].spawnerId].removeObject(chestId);
      this.scene.events.emit('chestRemoved', chestId);
    }
  });

  this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
    // update the spawner
    if (this.monsters[monsterId]) {
      const { gold, attack } = this.monsters[monsterId];

      // subtract health monster model
      this.monsters[monsterId].loseHealth();

      // check the monsters health, and if dead remove that object
      if (this.monsters[monsterId].health <= 0) {

        //updating the player's gold
        this.players[playerId].updateGold(gold);
        this.scene.events.emit('updateScore', this.players[playerId].gold);

        // removing the monster
```




```
this.spawnners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
this.scene.events.emit('monsterRemoved', monsterId);

// add bonus health to the player
this.players[playerId].updateHealth(2);
this.scene.events.emit('updatePlayerHealth', playerId, this.players[playerId].health);
} else {
    // update the player's health
    this.players[playerId].updateHealth(-attack);
    this.scene.events.emit('updatePlayerHealth', playerId, this.players[playerId].health);
    // update the monsters health
    this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);

    // check the player's health, if below 0 have the player respawn
    if (this.players[playerId].health <= 0) {
        // update the gold the player has
        this.players[playerId].updateGold(parseInt(-this.players[playerId].gold / 2), 10);
        this.scene.events.emit('updateScore', this.players[playerId].gold);

        // respawn the player
        this.players[playerId].respawn();
        this.scene.events.emit('respawnPlayer', this.players[playerId]);
    }
}
});
}
```

Finally, in the **PlayerModel** class, remove the console log from the **updateHealth** method. See the code below:

```
updateHealth(health) {
    this.health += health;
}
```

Extending the game

With a few small changes, you could make the game more interesting to the player. Some examples are:

- When the monster attacks the player, have the value be a random value between a min and max.
- When the player dies, you could have the gold drop on the map for the player to collect. Or you could add that gold to the monster that defeated the player.



Right now, the respawn function is not spawning the player in the correct spawn locations. This happens because our game world has twice the size of the Tiled map, so we need to adjust the coordinates to reflect that. Because of that, the player might spawn on incorrect locations such as above the water:



An easy way to fix that is to multiply the respawn location by 2 in the respawn function of PlayerModel.js. This will make sure the player respawn on the correct locations.

```
respawn() {  
    this.health = this.maxHealth;  
    const location = this.spawnLocations[Math.floor(Math.random() * this.spawnLocations.length)];  
    this.x = location[0] * 2;  
    this.y = location[1] * 2;  
}
```



In this lesson, you will be adding audio for the events of player attacking, player damage, player death, and enemy death.

Updates on last lesson

In the last lesson, you created and tested the player's health bar. For this, you set the **maxHealth** of a player to 3 in the **PlayerModel** class. Update it to 10 again before beginning today's lesson. This is the limit of health a player can have. Therefore, if it goes above 10, you will need to drop the health back down to 10. To make this change, go to **updateHealth** method in the **PlayerModel** class. Implement the logic to check if the player's health is greater than 10. If it is, set the value of health to 10. This way, the player's health cannot go over capacity. See the code below:

```
updateHealth(health) {  
    this.health += health;  
    if (this.health > 10) this.health = 10;  
}
```

Adding audio for the game

Earlier, you only had audio for when gaining gold. Now, you will add audio effects for the events of player attacking, player damage, player death, and enemy death. To do these, first open the **BootScene.js** file. In the **loadAudio** method, load the other assets for audio from **assets/audio** directory. See the code below:

```
loadAudio() {  
    this.load.audio('goldSound', ['assets/audio/Pickup.wav']);  
    this.load.audio('enemyDeath', ['assets/audio/EnemyDeath.wav']);  
    this.load.audio('playerAttack', ['assets/audio/PlayerAttack.wav']);  
    this.load.audio('playerDamage', ['assets/audio/PlayerDamage.wav']);  
    this.load.audio('playerDeath', ['assets/audio/PlayerDeath.wav']);  
}
```

Next, you will update the **createAudio** method in **GameScene** class. Similar to how the sound was configured for picking up gold, configure the sounds for the new audio clips as well. Remember to refer to the proper keys as set in **loadAudio** method above. You can have different volume levels for each event. See the code below:

```
createAudio() {  
    this.goldPickupAudio = this.sound.add('goldSound', { loop: false, volume: 0.3 });  
    this.playerAttackAudio = this.sound.add('playerAttack', { loop: false, volume: 0.  
01 });  
    this.playerDamageAudio = this.sound.add('playerDamage', { loop: false, volume: 0.  
2 });  
    this.playerDeathAudio = this.sound.add('playerDeath', { loop: false, volume: 0.2  
});  
    this.monsterDeathAudio = this.sound.add('enemyDeath', { loop: false, volume: 0.2  
});  
}
```



For implementing the logic for monster death, player death and player damage, you will be updating the **createGameManager** method in **GameScene** class.

Playing monster death audio

You will be playing the **monsterDeathAudio** when a monster is removed from the game. Add the logic to play the audio after a monster has been removed in the **monsterAttacked** event. Use the **play** method on the **monsterDeathAudio** for this. See the code snippet below:

```
this.events.on('monsterRemoved', (monsterId) => {
    this.monsters.getChildren().forEach((monster) => {
        if (monster.id === monsterId) {
            monster.makeInactive();
            this.monsterDeathAudio.play();
        }
    });
});
```

Playing player death audio

You will play the **playerDeathAudio** where you respawn the player. In the event **respawnPlayer**, play the audio before respawning the player. See the code snippet below :

```
this.events.on('respawnPlayer', (playerObject) => {
    this.playerDeathAudio.play();
    this.player.respawn(playerObject);
});
```

Playing player damage audio

Play the **playerDamageAudio** when the player's health is being updated at the **updatePlayerHealth** event. You only want to play this audio when the player's health is decreasing. Therefore, add a check to see if **health** decreased in order to play the audio. See the code snippet below:

```
this.events.on('updatePlayerHealth', (playerId, health) => {
    if (health < this.player.health) {
        this.playerDamageAudio.play();
    }
    this.player.updateHealth(health);
});
```

See the following code for the updates made to **createGameManager** method with all 3 events above:

```
createGameManager() {
    this.events.on('spawnPlayer', (playerObject) => {
```



```
        this.createPlayer(playerObject);
        this.addCollisions();
    });

    this.events.on('chestSpawned', (chest) => {
        this.spawnChest(chest);
    });

    this.events.on('monsterSpawned', (monster) => {
        this.spawnMonster(monster);
    });

    this.events.on('chestRemoved', (chestId) => {
        this.chests.getChildren().forEach((chest) => {
            if (chest.id === chestId) {
                chest.makeInactive();
            }
        });
    });

    this.events.on('monsterRemoved', (monsterId) => {
        this.monsters.getChildren().forEach((monster) => {
            if (monster.id === monsterId) {
                monster.makeInactive();
                this.monsterDeathAudio.play();    // playing monster death audio
            }
        });
    });

    this.events.on('updateMonsterHealth', (monsterId, health) => {
        this.monsters.getChildren().forEach((monster) => {
            if (monster.id === monsterId) {
                monster.updateHealth(health);
            }
        });
    });

    this.events.on('updatePlayerHealth', (playerId, health) => {
        if (health < this.player.health) {
            this.playerDamageAudio.play();    // playing player damage audio
        }
        this.player.updateHealth(health);
    });

    this.events.on('respawnPlayer', (playerObject) => {
        this.playerDeathAudio.play();    // playing player death audio
        this.player.respawn(playerObject);
    });

    this.gameManager = new GameManager(this, this.map.map.objects);
    this.gameManager.setup();
}
```



Playing player attack audio

You will play the **playerAttackAudio** every time a player attacks an enemy. Currently, all of the logic relating to player attacking is in the **PlayerContainer** class. Therefore you will pass the attack audio to the **PlayerContainer**.

To do this, first update the **PlayerContainer** constructor to take in a new parameter **attackAudio**. Then, set a new property **attackAudio** to that parameter. See the code below:

```
constructor(scene, x, y, key, frame, health, maxHealth, id, attackAudio) {
    super(scene, x, y);
    this.scene = scene;
    this.velocity = 160;
    this.currentDirection = Direction.RIGHT;
    this.playerAttacking = false;
    this.flipX = true;
    this.swordHit = false;
    this.health = health;
    this.maxHealth = maxHealth;
    this.id = id;
    this.attackAudio = attackAudio; // audio for attacking

    // set a size on the container
    this.setSize(64, 64);
    // enable physics
    this.scene.physics.world.enable(this);
    // collide with world bounds
    this.body.setCollideWorldBounds(true);
    // add the player container to our existing scene
    this.scene.add.existing(this);
    // have the camera follow the player
    this.scene.cameras.main.startFollow(this);

    // create the player
    this.player = new Player(this.scene, 0, 0, key, frame);
    this.add(this.player);

    // create the weapon game object
    this.weapon = this.scene.add.image(40, 0, 'items', 4);
    this.scene.add.existing(this.weapon);
    this.weapon.setScale(1.5);
    this.scene.physics.world.enable(this.weapon);
    this.add(this.weapon);
    this.weapon.alpha = 0;

    // create the player health bar
    this.createHealthBar();
}
```

Next, you will change the **update** method in **PlayerContainer** to play the **playerAttackAudio** when the space bar is being pressed down for attacking. See the code below (line 29):

```
update(cursors) {
```




```
this.body.setVelocity(0);

if (cursors.left.isDown) {
    this.body.setVelocityX(-this.velocity);
    this.currentDirection = Direction.LEFT;
    this.weapon.setPosition(-40, 0);
    this.player.flipX = false;
} else if (cursors.right.isDown) {
    this.body.setVelocityX(this.velocity);
    this.currentDirection = Direction.RIGHT;
    this.weapon.setPosition(40, 0);
    this.player.flipX = true;
}

if (cursors.up.isDown) {
    this.body.setVelocityY(-this.velocity);
    this.currentDirection = Direction.UP;
    this.weapon.setPosition(0, -40);
} else if (cursors.down.isDown) {
    this.body.setVelocityY(this.velocity);
    this.currentDirection = Direction.DOWN;
    this.weapon.setPosition(0, 40);
}

if (Phaser.Input.Keyboard.JustDown(cursors.space) && !this.playerAttacking) {
    this.weapon.alpha = 1;
    this.playerAttacking = true;
    this.attackAudio.play();
    this.scene.time.delayedCall(150, () => {
        this.weapon.alpha = 0;
        this.playerAttacking = false;
        this.swordHit = false;
    }, [], this);
}

if (this.playerAttacking) {
    if (this.weapon.flipX) {
        this.weapon.angle -= 10;
    } else {
        this.weapon.angle += 10;
    }
} else {
    if (this.currentDirection === Direction.DOWN) {
        this.weapon.setAngle(-270);
    } else if (this.currentDirection === Direction.UP) {
        this.weapon.setAngle(-90);
    } else {
        this.weapon.setAngle(0);
    }
}

this.weapon.flipX = false;
if (this.currentDirection === Direction.LEFT) {
    this.weapon.flipX = true;
}
}
```



```
    this.updateHealthBar();  
}
```

Since you updated the **PlayerContainer** constructor to take in a new parameter for the audio, creation of a player needs to be updated. To do this, go to where the **PlayerContainer** instance is being created in the **GameScene** class, and pass the **playerAttackAudio**. See the code below:

```
createPlayer(playerObject) {  
    this.player = new PlayerContainer(  
        this,  
        playerObject.x * 2,  
        playerObject.y * 2,  
        'characters',  
        0,  
        playerObject.health,  
        playerObject.maxHealth,  
        playerObject.id,  
        this.playerAttackAudio,  
    );  
}
```

Save and refresh the game to test changes. If you press the space bar, you should hear the audio for attacking. Picking up a chest should trigger the pickup sound. If a player takes a hit, the audio for the player being attacked should be played. If a monster dies, it will make a noise as well.

Additional note: Audio game objects

- To support multiple browsers, when you load an audio asset you can provide an array of asset files to load. Phaser will automatically use the assets that are supported in the player's browser.
- By default, when you play an Audio Game Asset, Phaser will only play that asset one time.



In this lesson, you will implement movement of the monsters in the game.

Updating MonsterModel class

Currently, a monster in the game would spawn in one location. They do not move, and would attack the player only when attacked. This is very basic. To upgrade the actions of the game, you will update the enemies to allow them to move around the map, and to move to random positions. That way, as the monsters move around, the player will have to chase them down and attack them.

To do this, create a new method called **move** in the **MonsterModel** class. In this method, you will allow the monster to move to one of the eight squares around it. For this, generate a random number between 1 and 8 that will be stored in the **randomPosition** variable. Then, move the monster according to that number. Create a **switch** statement for the **randomPosition** variable, and have 8 cases for each of the numbers. For each of the cases, update the **x** and/or **y** positions of the monster by increasing or decreasing the distance with a certain number of pixels. Note that both coordinates do not have to be updated at every time. See the code below:

```
move() {
    const randomPosition = randomNumber(1, 8);

    switch (randomPosition) {
        case 1:
            this.x += 32;
            break;
        case 2:
            this.x -= 32;
            break;
        case 3:
            this.y += 32;
            break;
        case 4:
            this.y -= 32;
            break;
        case 5:
            this.x += 32;
            this.y += 32;
            break;
        case 6:
            this.x += 32;
            this.y -= 32;
            break;
        case 7:
            this.x -= 32;
            this.y += 32;
            break;
        case 8:
            this.x -= 32;
            this.y -= 32;
            break;
        default:
            break;
    }
}
```



Updating Spawner class

Next, you need to have a way to call the **move** method every so often. To allow this, go to the **Spawner** class, and add a new method called **moveMonsters**. In this method, create an interval of 1 second. And for each of the monsters created, call the **move** method on them. See the code below:

```
moveMonsters() {
  this.moveMonsterInterval = setInterval(() => {
    this.objectsCreated.forEach((monster) => {
      monster.move();
    });
  }, 1000);
}
```

Next, in the **start** method of the **Spawner** class, add a check for the type of object. If it's a monster, you will call the interval implemented above for moving monsters. See the code below:

```
start() {
  this.interval = setInterval(() => {
    if (this.objectsCreated.length < this.limit) {
      this.spawnObject();
    }
  }, this.spawnInterval);
  if (this.objectType === SpawnerType.MONSTER) this.moveMonsters();
}
```

Additional Notes: JS switch statement

Switch statements are beneficial when you are comparing multiple possible conditions of an expression.

If you omit a **break** statement at the end of a case statement, the next case statement will be run regardless if the criteria was met.

In this lesson, you will update the monster movements and notify the **GameManager** of movement.

Updating the GameManager class

You need a way to have the **Spawner** notify the **GameManager** that the monsters have moved. For this, create a new method **moveMonsters** in the **GameManager** class. In this method, you will emit a new event called **monsterMovement**. Instead of sending one particular monster with the event, send the whole array of monsters. This way, you can update all of them at once. See the code below:

```
moveMonsters() {  
    this.scene.events.emit('monsterMovement', this.monsters);  
}
```

Then, this method needs to be passed to the **Spawner** class. For this, update the **setupSpawners** method in **GameManager** by passing a new argument with the **moveMonsters** method where you are creating monster spawners. See the code below (line 33):

```
setupSpawners() {  
    const config = {  
        spawnInterval: 3000,  
        limit: 3,  
        spawnerType: SpawnerType.CHEST,  
        id: '',  
    };  
    let spawner;  
  
    // create chest spawners  
    Object.keys(this.chestLocations).forEach((key) => {  
        config.id = `chest-${key}`;  
  
        spawner = new Spawner(  
            config,  
            this.chestLocations[key],  
            this.addChest.bind(this),  
            this.deleteChest.bind(this)  
        );  
        this.spawners[spawner.id] = spawner;  
    });  
  
    // create monster spawners  
    Object.keys(this.monsterLocations).forEach((key) => {  
        config.id = `monster-${key}`;  
        config.spawnerType = SpawnerType.MONSTER;  
  
        spawner = new Spawner(  
            config,  
            this.monsterLocations[key],  
            this.addMonster.bind(this),  
            this.deleteMonster.bind(this),  
            this.moveMonsters.bind(this),  
        );  
        this.spawners[spawner.id] = spawner;  
    });  
}
```

```
});  
}
```

Updating the Spawner class

In the constructor of the **Spawner** class, add a new parameter **moveObjects**. Store a reference to **moveObjects** within the constructor appropriately. See the code below:

```
constructor(config, spawnLocations, addObject, deleteObject, moveObjects) {  
    this.id = config.id;  
    this.spawnInterval = config.spawnInterval;  
    this.limit = config.limit;  
    this.objectType = config.spawnerType;  
    this.spawnLocations = spawnLocations;  
    this.addObject = addObject;  
    this.deleteObject = deleteObject;  
    this.moveObjects = moveObjects; // reference to moveObjects  
  
    this.objectsCreated = [];  
  
    this.start();  
}
```

Next, update the **moveMonsters** method in **Spawner** class. Within the interval, call the **moveObjects** method after the monsters have been moved. See the code below:

```
moveMonsters() {  
    this.moveMonsterInterval = setInterval(() => {  
        this.objectsCreated.forEach((monster) => {  
            monster.move();  
        });  
  
        this.moveObjects();  
    }, 1000);  
}
```

Updating the GameScene class

You will need to listen to the new event **monsterMovement** emitted by the **GameManager** in the **GameScene** class. Update the **createGameManager** method to receive the event and act upon it. For all the children of the **monsters** game object, loop through all of the **monsterId** keys. If the IDs of the monsters match, move that monster by calling the **physics.moveToObject** method. You will pass three arguments to this method; the game object that you want to move, an object that has the **x** and **y** coordinates of where you want the subject to go to, and the velocity you want the object to move towards that destination. See the code below (lines 40-49):

```
createGameManager() {  
    this.events.on('spawnPlayer', (playerObject) => {  
        this.createPlayer(playerObject);  
    });  
}
```




```
        this.addCollisions();
    });

    this.events.on('chestSpawned', (chest) => {
        this.spawnChest(chest);
    });

    this.events.on('monsterSpawned', (monster) => {
        this.spawnMonster(monster);
    });

    this.events.on('chestRemoved', (chestId) => {
        this.chests.getChildren().forEach((chest) => {
            if (chest.id === chestId) {
                chest.makeInactive();
            }
        });
    });

    this.events.on('monsterRemoved', (monsterId) => {
        this.monsters.getChildren().forEach((monster) => {
            if (monster.id === monsterId) {
                monster.makeInactive();
                this.monsterDeathAudio.play();
            }
        });
    });

    this.events.on('updateMonsterHealth', (monsterId, health) => {
        this.monsters.getChildren().forEach((monster) => {
            if (monster.id === monsterId) {
                monster.updateHealth(health);
            }
        });
    });

    // move the monster game object
    this.events.on('monsterMovement', (monsters) => {
        this.monsters.getChildren().forEach((monster) => {
            Object.keys(monsters).forEach((monsterId) => {
                if (monster.id === monsterId) {
                    this.physics.moveToObject(monster, monsters[monsterId], 40);
                }
            });
        });
    });

    this.events.on('updatePlayerHealth', (playerId, health) => {
        if (health < this.player.health) {
            this.playerDamageAudio.play();
        }
        this.player.updateHealth(health);
    });

    this.events.on('respawnPlayer', (playerObject) => {
```

```
        this.playerDeathAudio.play();
        this.player.respawn(playerObject);
    });

    this.gameManager = new GameManager(this, this.map.map.objects);
    this.gameManager.setup();
}
```

Calling the **setPosition** method would only have forcefully moved the monster to a new location. Therefore, **physics.moveToObject** is used instead so that when the game object moves it will use physics to actually move that object to the location.

Save and refresh the game. Wait for monsters to spawn. You will see that now the monsters are moving.



However, you can see that the health bar of the monsters are not moving with them.

Fixing scaling issues

You will also notice that the monster movement is a little off. The reason for this is, in the **move** method in **MonsterModel** class, you are expecting the tile size to be 32 when you have actually scaled everything by two. The monster actually needs to move 64 pixels instead of 32. However, you will run into an issue as the **x** and **y** values passed to the **MonsterModel** constructor when first spawning a monster are not multiplied.

To fix this, first remove the scaling for **monsterObject.x** and **monsterObject.y** in the **spawnMonster** method in the **GameScene** class. Do the same when you call **setPosition** method within the **spawnMonster** method. See the code below:

```
spawnMonster(monsterObject) {
    let monster = this.monsters.getFirstDead();
    if (!monster) {
        monster = new Monster(
            this,
            monsterObject.x,    // remove scaling
            monsterObject.y,    // remove scaling
        );
    }
}
```



```
        'monsters',
        monsterObject.frame,
        monsterObject.id,
        monsterObject.health,
        monsterObject.maxHealth,
    );
    // add monster to monsters group
    this.monsters.add(monster);
} else {
    monster.id = monsterObject.id;
    monster.health = monsterObject.health;
    monster.maxHealth = monsterObject.maxHealth;
    monster.setTexture('monsters', monsterObject.frame);
    monster.setPosition(monsterObject.x, monsterObject.y); // pass unscaled arguments
    monster.makeActive();
}
```

Then, in the **MonsterModel** constructor, scale the **x** and **y** values by 2. See the code below:

```
constructor(x, y, gold, spawnerId, frame, health, attack) {
    this.id = `${spawnerId}-${uuid.v4()}`;
    this.spawnerId = spawnerId;
    this.x = x * 2; // scale the position
    this.y = y * 2; // scale the position
    this.gold = gold;
    this.frame = frame;
    this.health = health;
    this.maxHealth = health;
    this.attack = attack;
}
```

Next, in the **move** method in the **MonsterModel** class, define a new variable **distance** and set it to 64. For all the **switch** cases of **randomPosition**, update the amount of pixels to move with the **distance** variable. See the code below:

```
move() {
    const randomPosition = randomNumber(1, 8);
    const distance = 64;

    switch (randomPosition) {
        case 1:
            this.x += distance;
            break;
        case 2:
            this.x -= distance;
            break;
        case 3:
            this.y += distance;
            break;
    }
}
```



```
    case 4:
        this.y -= distance;
        break;
    case 5:
        this.x += distance;
        this.y += distance;
        break;
    case 6:
        this.x += distance;
        this.y -= distance;
        break;
    case 7:
        this.x -= distance;
        this.y += distance;
        break;
    case 8:
        this.x -= distance;
        this.y -= distance;
        break;
    default:
        break;
}
}
```

Updating monster health bar

To fix the issue of health bars not moving with the monsters, you will update the **Monster** class. First, add a new method **update** and call **updateHealthBar** method within it. See the code below:

```
update() {
    this.updateHealthBar();
}
```

To use the **update** method, you can use the Phaser group. To do this, update the **createGroups** method in the **GameScene** class. After creating a monster group, set the **runChildUpdate** property of the **monsters** group to **true**. See the code below:

```
createGroups() {
    // create a chest group
    this.chests = this.physics.add.group();
    // create a monster group
    this.monsters = this.physics.add.group();
    this.monsters.runChildUpdate = true;
}
```

A benefit of having the monsters in a group is, you will be able to use this **runChildUpdate** property that Phaser has. This property is set to **false** by default. However, if there is an **update** method defined on that class, Phaser will automatically run that method for you if you set **runChildUpdate** property to **true**.

Save and refresh the game. Now, the monsters should be moving and their health bars should be following along with them.



Additional Notes: Phaser Physics `moveToObject`

In addition to using `setPosition()`, we can use `this.physics.moveToObject()` to move game objects.

While `setPosition()` will move a game object to the provided position, Phaser will use physics to move the game object to the provided position when you use `this.physics.moveToObject()`.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

utils.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/game_manager

The code establishes utilities used in the game. The 'SpawnerType' is an object representing types of entities that can be spawned. The 'randomNumber' function generates a random number within a given range. The 'getTiledProperty' function searches for and returns a specified property of a given object.

```
const SpawnerType = {
  MONSTER: 'MONSTER',
  CHEST: 'CHEST',
};

function randomNumber(min, max) {
  return Math.floor(Math.random() * (max - min)) + min;
}

function getTiledProperty(obj, property_name) {
  for (var property_index = 0; property_index < obj.properties.length; property_index += 1) {
    var property = obj.properties[property_index];
    if (property.name == property_name) {
      return property.value;
    }
  }
}
```

GameManager.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/game_manager

This code outlines the construction of a game manager class for a game using the Phaser framework. The GameManager handles the initialization of game entities such as spawners, chests, monsters, and players, the parsing of map data, and the setup of event listeners. It also manages various actions like spawning player, adding or deleting chests and monsters, and monster movement.

```
class GameManager {
  constructor(scene, mapData) {
    this.scene = scene;
    this.mapData = mapData;

    this.spawners = {};
    this.chests = {};
    this.monsters = {};
  }
}
```




```
this.players = {};  
  
this.playerLocations = [];  
this.chestLocations = {};  
this.monsterLocations = {};  
}  
  
setup() {  
  this.parseMapData();  
  this.setupEventListener();  
  this.setupSpawners();  
  this.spawnPlayer();  
}  
  
parseMapData() {  
  this.mapData.forEach((layer) => {  
    if (layer.name === 'player_locations') {  
      layer.objects.forEach((obj) => {  
        this.playerLocations.push([obj.x + (obj.width / 2), obj.y - (obj.height / 2  
    ]]);  
      });  
    } else if (layer.name === 'chest_locations') {  
      layer.objects.forEach((obj) => {  
        var spawner = getTiledProperty(obj, 'spawner');  
        if (this.chestLocations[spawner]) {  
          this.chestLocations[spawner].push([obj.x + (obj.width / 2), obj.y - (obj.  
height / 2)]);  
        } else {  
          this.chestLocations[spawner] = [[obj.x + (obj.width / 2), obj.y - (obj.he  
ight / 2)]];  
        }  
      });  
    } else if (layer.name === 'monster_locations') {  
      layer.objects.forEach((obj) => {  
        var spawner = getTiledProperty(obj, 'spawner');  
        if (this.monsterLocations[spawner]) {  
          this.monsterLocations[spawner].push([obj.x, obj.y]);  
        } else {  
          this.monsterLocations[spawner] = [[obj.x, obj.y]];  
        }  
      });  
    }  
  });  
}  
  
setupEventListener() {  
  this.scene.events.on('pickUpChest', (chestId, playerId) => {  
    // update the spawner  
    if (this.chests[chestId]) {  
      const { gold } = this.chests[chestId];  
  
      // updating the players gold  
      this.players[playerId].updateGold(gold);  
      this.scene.events.emit('updateScore', this.players[playerId].gold);  
    }  
  });  
}
```



```
// removing the chest
this.spawnners[this.chests[chestId].spawnerId].removeObject(chestId);
this.scene.events.emit('chestRemoved', chestId);
}
});

this.scene.events.on('monsterAttacked', (monsterId, playerId) => {
  // update the spawner
  if (this.monsters[monsterId]) {
    const { gold, attack } = this.monsters[monsterId];

    // subtract health monster model
    this.monsters[monsterId].loseHealth();

    // check the monsters health, and if dead remove that object
    if (this.monsters[monsterId].health <= 0) {
      // updating the players gold
      this.players[playerId].updateGold(gold);
      this.scene.events.emit('updateScore', this.players[playerId].gold);

      // removing the monster
      this.spawnners[this.monsters[monsterId].spawnerId].removeObject(monsterId);
      this.scene.events.emit('monsterRemoved', monsterId);

      // add bonus health to the player
      this.players[playerId].updateHealth(2);
      this.scene.events.emit('updatePlayerHealth', playerId, this.players[playerId].health);
    } else {
      // update the players health
      this.players[playerId].updateHealth(-attack);
      this.scene.events.emit('updatePlayerHealth', playerId, this.players[playerId].health);

      // update the monsters health
      this.scene.events.emit('updateMonsterHealth', monsterId, this.monsters[monsterId].health);

      // check the player's health, if below 0 have the player respawn
      if (this.players[playerId].health <= 0) {
        // update the gold the player has
        this.players[playerId].updateGold(parseInt(-this.players[playerId].gold / 2), 10);
        this.scene.events.emit('updateScore', this.players[playerId].gold);

        // respawn the player
        this.players[playerId].respawn();
        this.scene.events.emit('respawnPlayer', this.players[playerId]);
      }
    }
  }
});

setupSpawners() {
```



```
const config = {
  spawnInterval: 3000,
  limit: 3,
  spawnerType: SpawnerType.CHEST,
  id: '',
};
let spawner;

// create chest spawners
Object.keys(this.chestLocations).forEach((key) => {
  config.id = `chest-${key}`;

  spawner = new Spawner(
    config,
    this.chestLocations[key],
    this.addChest.bind(this),
    this.deleteChest.bind(this)
  );
  this.spawners[spawner.id] = spawner;
});

// create monster spawners
Object.keys(this.monsterLocations).forEach((key) => {
  config.id = `monster-${key}`;
  config.spawnerType = SpawnerType.MONSTER;

  spawner = new Spawner(
    config,
    this.monsterLocations[key],
    this.addMonster.bind(this),
    this.deleteMonster.bind(this),
    this.moveMonsters.bind(this),
  );
  this.spawners[spawner.id] = spawner;
});
}

spawnPlayer() {
  const player = new PlayerModel(this.playerLocations);
  this.players[player.id] = player;
  this.scene.events.emit('spawnPlayer', player);
}

addChest(chestId, chest) {
  this.chests[chestId] = chest;
  this.scene.events.emit('chestSpawned', chest);
}

deleteChest(chestId) {
  delete this.chests[chestId];
}

addMonster(monsterId, monster) {
  this.monsters[monsterId] = monster;
  this.scene.events.emit('monsterSpawned', monster);
}
```

```
}

deleteMonster(monsterId) {
    delete this.monsters[monsterId];
}

moveMonsters() {
    this.scene.events.emit('monsterMovement', this.monsters);
}
}
```

MonsterModel.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/game_manager

This code defines a 'MonsterModel' class for a game. The class includes the properties and behaviors of a monster, such as its id, location, gold amount, health, attack strength, frame, and actions for losing health and moving in random directions.

```
class MonsterModel {
    constructor(x, y, gold, spawnerId, frame, health, attack) {
        this.id = `${spawnerId}-${uuid.v4()}`;
        this.spawnerId = spawnerId;
        this.x = x * 2;
        this.y = y * 2;
        this.gold = gold;
        this.frame = frame;
        this.health = health;
        this.maxHealth = health;
        this.attack = attack;
    }

    loseHealth() {
        this.health -= 1;
    }

    move() {
        const randomPosition = randomNumber(1, 8);
        const distance = 64;

        switch (randomPosition) {
            case 1:
                this.x += distance;
                break;
            case 2:
                this.x -= distance;
                break;
            case 3:
                this.y += distance;
                break;
            case 4:
                this.y -= distance;
                break;
        }
    }
}
```



```
        case 5:
            this.x += distance;
            this.y += distance;
            break;
        case 6:
            this.x += distance;
            this.y -= distance;
            break;
        case 7:
            this.x -= distance;
            this.y += distance;
            break;
        case 8:
            this.x -= distance;
            this.y -= distance;
            break;
        default:
            break;
    }
}
```

PlayerModel.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/game_manager

This code creates a PlayerModel class for a game. It defines the player's initial states, such as health, gold, and location. The methods within the class allow for updating the player's gold and health, and allows for the player to respawn(changing location and resetting health) within the game.

```
class PlayerModel {
    constructor(spawnLocations) {
        this.health = 10;
        this.maxHealth = 10;
        this.gold = 0;
        this.id = `player-${uuid.v4()}`;
        this.spawnLocations = spawnLocations;

        const location = this.spawnLocations[Math.floor(Math.random() * this.spawnLocations.length)];
        [this.x, this.y] = location;
    }

    updateGold(gold) {
        this.gold += gold;
    }

    updateHealth(health) {
        this.health += health;
        if (this.health > 10) this.health = 10;
    }

    respawn() {
```



```
        this.health = this.maxHealth;
        const location = this.spawnLocations[Math.floor(Math.random() * this.spawnLocations.length)];
        [this.x, this.y] = location;
    }
}
```

ChestModel.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/game_manager

This code creates a ChestModel class for a game. It sets the initial properties of a chest, like its identifier, spawner ID, coordinates, and the amount of gold it holds. Each chest is created in a specific location and contains a certain quantity of gold.

```
class ChestModel {
    constructor(x, y, gold, spawnerId) {
        this.id = `${spawnerId}-${uuid.v4()}`;
        this.spawnerId = spawnerId;
        this.x = x;
        this.y = y;
        this.gold = gold;
    }
}
```

Spawner.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/game_manager

This code defines a 'Spawner' class used in a game. This class controls the creation (spawning) of game objects: monsters or chests. It manages objects' creation at random locations, their movement (for monsters), and deletion. Various parameters such as spawn interval, limit on objects, and the type of object to spawn are defined through the constructor.

```
class Spawner {
    constructor(config, spawnLocations, addObject, deleteObject, moveObjects) {
        this.id = config.id;
        this.spawnInterval = config.spawnInterval;
        this.limit = config.limit;
        this.objectType = config.spawnerType;
        this.spawnLocations = spawnLocations;
        this.addObject = addObject;
        this.deleteObject = deleteObject;
        this.moveObjects = moveObjects;

        this.objectsCreated = [];

        this.start();
    }

    start() {
```




```
this.interval = setInterval(() => {
    if (this.objectsCreated.length < this.limit) {
        this.spawnObject();
    }
}, this.spawnInterval);
if (this.objectType === SpawnerType.MONSTER) this.moveMonsters();
}

spawnObject() {
    if (this.objectType === SpawnerType.CHEST) {
        this.spawnChest();
    } else if (this.objectType === SpawnerType.MONSTER) {
        this.spawnMonster();
    }
}

spawnChest() {
    const location = this.pickRandomLocation();
    const chest = new ChestModel(location[0], location[1], randomNumber(10, 20), this
.id);
    this.objectsCreated.push(chest);
    this.addObject(chest.id, chest);
}

spawnMonster() {
    const location = this.pickRandomLocation();
    const monster = new MonsterModel(
        location[0],
        location[1],
        randomNumber(10, 20),
        this.id,
        randomNumber(0, 20),
        randomNumber(3, 5),
        1,
    );
    this.objectsCreated.push(monster);
    this.addObject(monster.id, monster);
}

pickRandomLocation() {
    const location = this.spawnLocations[Math.floor(Math.random() * this.spawnLocations.length)];
    const invalidLocation = this.objectsCreated.some((obj) => {
        if (obj.x === location[0] && obj.y === location[1]) {
            return true;
        }
        return false;
    });
    if (invalidLocation) return this.pickRandomLocation();
    return location;
}

removeObject(id) {
    this.objectsCreated = this.objectsCreated.filter(obj => obj.id !== id);
}
```



```
        this.deleteObject(id);
    }

    moveMonsters() {
        this.moveMonsterInterval = setInterval(() => {
            this.objectsCreated.forEach((monster) => {
                monster.move();
            });

            this.moveObjects();
        }, 1000);
    }
}
```

Map.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/classes

This JavaScript code defines a `Map` class for a tiled game world. It includes a constructor and a `createMap` method that sets up the tile map, background and blocked layers, and adjusts the physics world boundaries and camera bounds to the map's dimensions. The class provides capabilities to create and arrange tiles, map layers, and set collision properties.

```
class Map {
    constructor(scene, key, tileSetName, bgLayerName, blockedLayerName) {
        this.scene = scene; // the scene this map belongs to
        this.key = key; // Tiled JSON file key name
        this.tileSetName = tileSetName; // Tiled Tileset image key name
        this.bgLayerName = bgLayerName; // the name of the layer created in tiled for the
        map background
        this.blockedLayerName = blockedLayerName; // the name of the layer created in tiled for the blocked areas

        this.createMap();
    }

    createMap() {
        // create the tile map
        this.map = this.scene.make.tilemap({ key: this.key });

        // add the tileset image to our map
        this.tiles = this.map.addTilesetImage(this.tileSetName, this.tileSetName, 32, 32,
        1, 2);

        // create our background
        this.backgroundLayer = this.map.createLayer(this.bgLayerName, this.tiles, 0, 0);
        this.backgroundLayer.setScale(2);

        // create blocked layer
        this.blockedLayer = this.map.createLayer(this.blockedLayerName, this.tiles, 0, 0)
        ;
        this.blockedLayer.setScale(2);
        this.blockedLayer.setCollisionByExclusion([-1]);
    }
}
```



```
// update the world bounds
this.scene.physics.world.bounds.width = this.map.widthInPixels * 2;
this.scene.physics.world.bounds.height = this.map.heightInPixels * 2;

// limit the camera to the size of our map
this.scene.cameras.main.setBounds(0, 0, this.map.widthInPixels * 2, this.map.heightInPixels * 2);
}
```

Monster.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/classes

This JavaScript code defines a `Monster` class, a subclass of `Phaser.Physics.Arcade.Image`. It includes methods for creating a monster instance with health attributes and a health bar, updating health and health bar, and controlling monster's active status in a Phaser game environment. Health and visibility states are managed, while physics, scaling, and collision settings are configured in its constructor.

```
class Monster extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame, id, health, maxHealth) {
    super(scene, x, y, key, frame);
    this.scene = scene;
    this.id = id;
    this.health = health;
    this.maxHealth = maxHealth;

    // enable physics
    this.scene.physics.world.enable(this);
    // set immovable if another object collides with our monster
    this.setImmovable(false);
    // scale our monster
    this.setScale(2);
    // collide with world bounds
    this.setCollideWorldBounds(true);
    // add the monster to our existing scene
    this.scene.add.existing(this);
    // update the origin
    this.setOrigin(0);

    this.createHealthBar();
  }

  createHealthBar() {
    this.healthBar = this.scene.add.graphics();
    this.updateHealthBar();
  }

  updateHealthBar() {
    this.healthBar.clear();
    this.healthBar.fillStyle(0xffffffff, 1);
```



```
        this.healthBar.fillRect(this.x, this.y - 8, 64, 5);
        this.healthBar.fillGradientStyle(0xff0000, 0xffffffff, 4);
        this.healthBar.fillRect(this.x, this.y - 8, 64 * (this.health / this.maxHealth),
5);
    }

    updateHealth(health) {
        this.health = health;
        this.updateHealthBar();
    }

    makeActive() {
        this.setActive(true);
        this.setVisible(true);
        this.body.checkCollision.none = false;
        this.updateHealthBar();
    }

    makeInactive() {
        this.setActive(false);
        this.setVisible(false);
        this.body.checkCollision.none = true;
        this.healthBar.clear();
    }

    update() {
        this.updateHealthBar();
    }
}
```

Chest.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/classes

This JavaScript code outlines a `Chest` class, a subclass of `Phaser.Physics.Arcade.Image`, which represents a chest game object in a Phaser game. The class has methods to set the chest active or inactive, and a constructor setting the scene, position, image keys, coins contained, and enabling physics and scaling.

```
class Chest extends Phaser.Physics.Arcade.Image {
    constructor(scene, x, y, key, frame, coins, id) {
        super(scene, x, y, key, frame);
        this.scene = scene; // the scene this game object will be added to
        this.coins = coins; // the amount of coins this chest contains
        this.id = id;

        // enable physics
        this.scene.physics.world.enable(this);
        // add the player to our existing scene
        this.scene.add.existing(this);
        // scale the chest game object
        this.setScale(2);
    }
}
```

```
makeActive() {
    this.setActive(true);
    this.setVisible(true);
    this.body.checkCollision.none = false;
}

makeInactive() {
    this.setActive(false);
    this.setVisible(false);
    this.body.checkCollision.none = true;
}
}
```

UIButton.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/classes

This JavaScript code creates a `UIButton` class inheriting from Phaser's `GameObjects.Container`, used for creating interactive buttons within a game. It accepts parameters for scene, position, button images, text, and an action callback. The `createButton` method helps in creating and customizing the button along with functionality to handle hover states and clicks.

```
class UIButton extends Phaser.GameObjects.Container {
    constructor(scene, x, y, key, hoverKey, text, targetCallback) {
        super(scene, x, y);
        this.scene = scene; // the scene this container will be added to
        this.x = x; // the x position of our container
        this.y = y; // the y position of our container
        this.key = key; // the background image of our button
        this.hoverKey = hoverKey; // the image that will be displayed when the player hovers over the button
        this.text = text; // the text that will be displayed on the button
        this.targetCallback = targetCallback; // the callback function that will be called when the player clicks the button

        // create our Ui Button
        this.createButton();
        // add this container to our Phaser Scene
        this.scene.add.existing(this);
    }

    createButton() {
        // create play game button
        this.button = this.scene.add.image(0, 0, 'button1');
        // make button interactive
        this.button.setInteractive();
        // scale the button
        this.button.setScale(1.4);

        // create the button text
        this.buttonText = this.scene.add.text(0, 0, this.text, { fontSize: '26px', fill: '#fff' });
    }
}
```



```
// center the button text inside the Ui button
Phaser.Display.Align.In.Center(this.buttonText, this.button);

// add the two game objects to our container
this.add(this.button);
this.add(this.buttonText);

// listen for events
this.button.on('pointerdown', () => {
  this.targetCallback();
});

this.button.on('pointerover', () => {
  this.button.setTexture(this.hoverKey);
});

this.button.on('pointerout', () => {
  this.button.setTexture(this.key);
});
}
```

Player.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/classes/player

This code constructs a `Player` class in JavaScript, which is a subclass of `Phaser.Physics.Arcade.Image`. The class constructs a player object in a Phaser game, activating physics for it, marking it immovable upon collision, scaling it and placing it in the supplied scene.

```
class Player extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);
    this.scene = scene; // the scene this container will be added to

    // enable physics
    this.scene.physics.world.enable(this);
    // set immovable if another object collides with our player
    this.setImmovable(true);
    // scale our player
    this.setScale(2);
    // add the player to our existing scene
    this.scene.add.existing(this);
  }
}
```

Player.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/classes/player

This code creates a `PlayerContainer` class that extends `Phaser.GameObjects.Container`, meant to



construct a player character in a game. It defines player attributes like health, attack state, direction of movement, and an equipped weapon. Methods are provided for creating a health bar, updating health and health bar, and player actions like moving and attacking. The update method handles inputs for movement and attacks.

```
const Direction = {
  RIGHT: 'RIGHT',
  LEFT: 'LEFT',
  UP: 'UP',
  DOWN: 'DOWN',
};

class PlayerContainer extends Phaser.GameObjects.Container {
  constructor(scene, x, y, key, frame, health, maxHealth, id, attackAudio) {
    super(scene, x, y);
    this.scene = scene; // the scene this container will be added to
    this.velocity = 160; // the velocity when moving our player
    this.currentDirection = Direction.RIGHT;
    this.playerAttacking = false;
    this.flipX = true;
    this.swordHit = false;
    this.health = health;
    this.maxHealth = maxHealth;
    this.id = id;
    this.attackAudio = attackAudio;

    // set a size on the container
    this.setSize(64, 64);
    // enable physics
    this.scene.physics.world.enable(this);
    // collide with world bounds
    this.body.setCollideWorldBounds(true);
    // add the player container to our existing scene
    this.scene.add.existing(this);
    // have the camera follow the player
    this.scene.cameras.main.startFollow(this);

    // create the player
    this.player = new Player(this.scene, 0, 0, key, frame);
    this.add(this.player);

    // create the weapon game object
    this.weapon = this.scene.add.image(40, 0, 'items', 4);
    this.scene.add.existing(this.weapon);
    this.weapon.setScale(1.5);
    this.scene.physics.world.enable(this.weapon);
    this.add(this.weapon);
    this.weapon.alpha = 0;

    // create the player healthbar
    this.createHealthBar();

    this.key = this.scene.input.keyboard.addKey(Phaser.Input.Keyboard.KeyCodes.CTRL
  );
}
```




```
createHealthBar() {
    this.healthBar = this.scene.add.graphics();
    this.updateHealthBar();
}

updateHealthBar() {
    this.healthBar.clear();
    this.healthBar.fillStyle(0xffffffff, 1);
    this.healthBar.fillRect(this.x - 32, this.y - 40, 64, 5);
    this.healthBar.fillGradientStyle(0xff0000, 0xffffffff, 4);
    this.healthBar.fillRect(this.x - 32, this.y - 40, 64 * (this.health / this.maxHealth), 5);
}

updateHealth(health) {
    this.health = health;
    this.updateHealthBar();
}

respawn(playerObject) {
    this.health = playerObject.health;
    this.setPosition(playerObject.x, playerObject.y);
    this.updateHealthBar();
}

update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
        this.body.setVelocityX(-this.velocity);
        this.currentDirection = Direction.LEFT;
        this.weapon.setPosition(-40, 0);
        this.player.flipX = false;
    } else if (cursors.right.isDown) {
        this.body.setVelocityX(this.velocity);
        this.currentDirection = Direction.RIGHT;
        this.weapon.setPosition(40, 0);
        this.player.flipX = true;
    }

    if (cursors.up.isDown) {
        this.body.setVelocityY(-this.velocity);
        this.currentDirection = Direction.UP;
        this.weapon.setPosition(0, -40);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
        this.currentDirection = Direction.DOWN;
        this.weapon.setPosition(0, 40);
    }

    if (Phaser.Input.Keyboard.JustDown(cursors.space) && !this.playerAttacking) {
        this.weapon.alpha = 1;
        this.playerAttacking = true;
        this.attackAudio.play();
    }
}
```



```
        this.scene.time.delayedCall(150, () => {
            this.weapon.alpha = 0;
            this.playerAttacking = false;
            this.swordHit = false;
        }, [], this);
    }

    if (this.playerAttacking) {
        if (this.weapon.flipX) {
            this.weapon.angle -= 10;
        } else {
            this.weapon.angle += 10;
        }
    } else {
        if (this.currentDirection === Direction.DOWN) {
            this.weapon.setAngle(-270);
        } else if (this.currentDirection === Direction.UP) {
            this.weapon.setAngle(-90);
        } else {
            this.weapon.setAngle(0);
        }
    }

    this.weapon.flipX = false;
    if (this.currentDirection === Direction.LEFT) {
        this.weapon.flipX = true;
    }
}

this.updateHealthBar();
}
```

BootScene.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/scenes

This `BootScene` class, which is a subclass of `Phaser.Scene`, handles preloading images, spritesheets, audio files, and a tilemap for a game, then transitions to a `Title` scene. Each resource category has a separate loading method. Preloads occur in the `preload()` method, and the scene transition happens in the `create()` method.

```
class BootScene extends Phaser.Scene {
    constructor() {
        super('Boot');
    }

    preload() {
        // load images
        this.loadImages();
        // load spritesheets
        this.loadSpriteSheets();
        // load audio
        this.loadAudio();
    }
}
```

```
// load tilemap
this.loadTileMap();
}

loadImages() {
    this.load.image('button1', 'assets/images/ui/blue_button01.png');
    this.load.image('button2', 'assets/images/ui/blue_button02.png');
    // load the map tileset image
    this.load.image('background', 'assets/level/background-extruded.png');
}

loadSpriteSheets() {
    this.load.spritesheet('items', 'assets/images/items.png', { frameWidth: 32, frame
Height: 32 });
    this.load.spritesheet('characters', 'assets/images/characters.png', { frameWidth:
32, frameHeight: 32 });
    this.load.spritesheet('monsters', 'assets/images/monsters.png', { frameWidth: 32,
frameHeight: 32 });
}

loadAudio() {
    this.load.audio('goldSound', ['assets/audio/Pickup.wav']);
    this.load.audio('enemyDeath', ['assets/audio/EnemyDeath.wav']);
    this.load.audio('playerAttack', ['assets/audio/PlayerAttack.wav']);
    this.load.audio('playerDamage', ['assets/audio/PlayerDamage.wav']);
    this.load.audio('playerDeath', ['assets/audio/PlayerDeath.wav']);
}

loadTileMap() {
    // map made with Tiled in JSON format
    this.load.tilemapTiledJSON('map', 'assets/level/large_level2.json');
}

create() {
    this.scene.start('Title');
}
}
```

GameScene.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/scenes

This `GameScene` class extends `Phaser.Scene` and provides the core logic for a game. It manages player and enemy creation, input handling, audio, and game assets spawning. Within this class, methods are assigned to handle actions such as collecting chests, attacking enemies, and responding to collision. It also covers creating and updating health bars and gameplay events. Game initialization occurs in the `create()` method and then the `update()` method runs continuously in the game loop.

```
class GameScene extends Phaser.Scene {
    constructor() {
        super('Game');
    }
}
```



```
init() {
    this.scene.launch('Ui');
}

create() {
    this.createMap();
    this.createAudio();
    this.createGroups();
    this.createInput();

    this.createGameManager();
}

update() {
    if (this.player) this.player.update(this.cursors);
}

createAudio() {
    this.goldPickupAudio = this.sound.add('goldSound', { loop: false, volume: 0.3 });
    this.playerAttackAudio = this.sound.add('playerAttack', { loop: false, volume: 0.
01 });
    this.playerDamageAudio = this.sound.add('playerDamage', { loop: false, volume: 0.
2 });
    this.playerDeathAudio = this.sound.add('playerDeath', { loop: false, volume: 0.2
});
    this.monsterDeathAudio = this.sound.add('enemyDeath', { loop: false, volume: 0.2
});
}

createPlayer(playerObject) {
    this.player = new PlayerContainer(
        this,
        playerObject.x * 2,
        playerObject.y * 2,
        'characters',
        0,
        playerObject.health,
        playerObject.maxHealth,
        playerObject.id,
        this.playerAttackAudio,
    );
}

createGroups() {
    // create a chest group
    this.chests = this.physics.add.group();
    // create a monster group
    this.monsters = this.physics.add.group();
    this.monsters.runChildUpdate = true;
}

spawnChest(chestObject) {
    let chest = this.chests.getFirstDead();
    if (!chest) {
```



```
        chest = new Chest(this, chestObject.x * 2, chestObject.y * 2, 'items', 0, chestObject.gold, chestObject.id);
        // add chest to chests group
        this.chests.add(chest);
    } else {
        chest.coins = chestObject.gold;
        chest.id = chestObject.id;
        chest.setPosition(chestObject.x * 2, chestObject.y * 2);
        chest.makeActive();
    }
}

spawnMonster(monsterObject) {
    let monster = this.monsters.getFirstDead();
    if (!monster) {
        monster = new Monster(
            this,
            monsterObject.x,
            monsterObject.y,
            'monsters',
            monsterObject.frame,
            monsterObject.id,
            monsterObject.health,
            monsterObject.maxHealth,
        );
        // add monster to monsters group
        this.monsters.add(monster);
        monster.setCollideWorldBounds(true);
    } else {
        monster.id = monsterObject.id;
        monster.health = monsterObject.health;
        monster.maxHealth = monsterObject.maxHealth;
        monster.setTexture('monsters', monsterObject.frame);
        monster.setPosition(monsterObject.x, monsterObject.y);
        monster.makeActive();
    }
}

createInput() {
    this.cursors = this.input.keyboard.createCursorKeys();
}

addCollisions() {
    // check for collisions between the player and the tiled blocked layer
    this.physics.add.collider(this.player, this.map.blockedLayer);
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, this);
    ;
    // check for collisions between the monster group and the tiled blocked layer
    this.physics.add.collider(this.monsters, this.map.blockedLayer);
    // check for overlaps between the player's weapon and monster game objects
    this.physics.add.overlap(this.player.weapon, this.monsters, this.enemyOverlap, null, this);
}
```



```
enemyOverlap(weapon, enemy) {
  if (this.player.playerAttacking && !this.player.swordHit) {
    this.player.swordHit = true;
    this.events.emit('monsterAttacked', enemy.id, this.player.id);
  }
}

collectChest(player, chest) {
  // play gold pickup sound
  this.goldPickupAudio.play();
  this.events.emit('pickUpChest', chest.id, player.id);
}

createMap() {
  // create map
  this.map = new Map(this, 'map', 'background', 'background', 'blocked');
}

createGameManager() {
  this.events.on('spawnPlayer', (playerObject) => {
    this.createPlayer(playerObject);
    this.addCollisions();
  });

  this.events.on('chestSpawned', (chest) => {
    this.spawnChest(chest);
  });

  this.events.on('monsterSpawned', (monster) => {
    this.spawnMonster(monster);
  });

  this.events.on('chestRemoved', (chestId) => {
    this.chests.getChildren().forEach((chest) => {
      if (chest.id === chestId) {
        chest.makeInactive();
      }
    });
  });

  this.events.on('monsterRemoved', (monsterId) => {
    this.monsters.getChildren().forEach((monster) => {
      if (monster.id === monsterId) {
        monster.makeInactive();
        this.monsterDeathAudio.play();
      }
    });
  });

  this.events.on('updateMonsterHealth', (monsterId, health) => {
    this.monsters.getChildren().forEach((monster) => {
      if (monster.id === monsterId) {
        monster.updateHealth(health);
      }
    });
  });
}
```



```
});

this.events.on('monsterMovement', (monsters) => {
  this.monsters.getChildren().forEach((monster) => {
    Object.keys(monsters).forEach((monsterId) => {
      if (monster.id === monsterId) {
        this.physics.moveToObject(monster, monsters[monsterId], 40);
      }
    });
  });
});

this.events.on('updatePlayerHealth', (playerId, health) => {
  if (health < this.player.health) {
    this.playerDamageAudio.play();
  }
  this.player.updateHealth(health);
});

this.events.on('respawnPlayer', (playerObject) => {
  this.playerDeathAudio.play();
  this.player.respawn(playerObject);
});

this.gameManager = new GameManager(this, this.map.map.objects);
this.gameManager.setup();
}
```

TitleScene.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/scenes

The `TitleScene` class extends `Phaser.Scene` and is used for creating a game's title screen. It has a `create` method that displays a title text and a play button, the latter created using custom `UIButton` class. Clicking the play button triggers the start of the game scene.

```
class TitleScene extends Phaser.Scene {
  constructor() {
    super('Title');
  }

  create() {
    // create title text
    this.titleText = this.add.text(this.scale.width / 2, this.scale.height / 2, 'Zenva MMORPG', {
      fontSize: '64px', fill: '#fff'
    });
    this.titleText.setOrigin(0.5);

    // create the Play game button
    this.startGameButton = new UIButton(this, this.scale.width / 2, this.scale.height * 0.65, 'button1', 'button2', 'Start', this.startScene.bind(this, 'Game'));
  }
}
```




```
startScene(targetScene) {  
    this.scene.start(targetScene);  
}  
}
```

UiScene.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/scenes

The `UiScene` class extends Phaser's `Scene` and represents the user interface of the game. Here, references to the game scene are created and necessary UI elements like the score text and icon are set up. There's also a `setupEvents` method listening for `updateScore` events, updating the score text whenever received from the game scene.

```
class UiScene extends Phaser.Scene {  
    constructor() {  
        super('Ui');  
    }  
  
    init() {  
        // grab a reference to the game scene  
        this.gameScene = this.scene.get('Game');  
    }  
  
    create() {  
        this.setupUiElements();  
        this.setupEvents();  
    }  
  
    setupUiElements() {  
        // create the score text game object  
        this.scoreText = this.add.text(35, 8, 'Coins: 0', { fontSize: '16px', fill: '#fff' });  
        // create coin icon  
        this.coinIcon = this.add.image(15, 15, 'items', 3);  
    }  
  
    setupEvents() {  
        // listen for the updateScore event from the game scene  
        this.gameScene.events.on('updateScore', (score) => {  
            this.scoreText.setText(`Coins: ${score}`);  
        });  
    }  
}
```

TestScene.js

Found in Project/Phaser_RPG-CourseFiles/assets/js/scenes

The `TestScene` class, a subclass of `Phaser.Scene`, is a testing sandbox. It's used for studying the behavior of a `Monster` game object, specifically how it moves. The `create` method sets up the

physics world and a monster instance. The `update` method continually logs the monster's position for observation.

```
class TestScene extends Phaser.Scene {
  constructor() {
    super('Test');
  }

  create() {
    this.physics.world.bounds.width = 1000;
    this.physics.world.bounds.height = 1000;

    this.monster = new Monster(this,
      900,
      900,
      'monsters',
      0,
      0,
      10,
      10);
    this.physics.moveTo(this.monster, 2000, 2000);
  }

  update() {
    console.log(this.monster.x);
    console.log(this.monster.y);
  }
}
```

main.js

Found in Project/Phaser_RPG-CourseFiles/assets/js

This JavaScript code initiates a Phaser game using a configuration object. It sets game dimensions, physics engine, rendering settings and lists game scenes in order. It includes `BootScene`, `TitleScene`, `GameScene`, and `UiScene`. Then it creates a new `Phaser.Game` instance, launching the game with the given configuration.

```
var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  scene: [
    BootScene,
    TitleScene,
    GameScene,
    UiScene,
  ],
  physics: {
    default: 'arcade',
    arcade: {
      debug: false,
      gravity: {
```

```
        y: 0,
    },
},
},
pixelArt: true,
roundPixels: true,
};

var game = new Phaser.Game(config);
```

index.html

Found in Project/Phaser_RPG-CourseFiles

This HTML file is the home page for the “Zenva MMORPG” game. It loads in the Phaser library, utilities, game manager, models, player classes, game entities, and the game scenes from multiple JavaScript files. The game content is housed within a div with the id “phaser-game”. The final script tag in the file loads the game’s main JavaScript file.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="user-scalable=0, initial-scale=1, minium-
scale=1, maximum-scale=1, width=device-width, minimal-ui=1">
    <title>Zenva MMORPG</title>
  </head>
  <body>
    <div id="phaser-game"></div>
    <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/node-
uuid/1.4.8/uuid.min.js"></script>
    <script src="assets/js/game_manager/utils.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/GameManager.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/MonsterModel.js" charset="utf-8"></script>
  >
    <script src="assets/js/game_manager/PlayerModel.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/ChestModel.js" charset="utf-8"></script>
    <script src="assets/js/game_manager/Spawner.js" charset="utf-8"></script>
    <script src="assets/js/classes/player/Player.js" charset="utf-8"></script>
    <script src="assets/js/classes/player/PlayerContainer.js" charset="utf-8"></s
cript>
    <script src="assets/js/classes/Monster.js" charset="utf-8"></script>
    <script src="assets/js/classes/Chest.js" charset="utf-8"></script>
    <script src="assets/js/classes/UiButton.js" charset="utf-8"></script>
    <script src="assets/js/classes/Map.js" charset="utf-8"></script>
    <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
    <script src="assets/js/main.js" charset="utf-8"></script>
  </body>
</html>
```